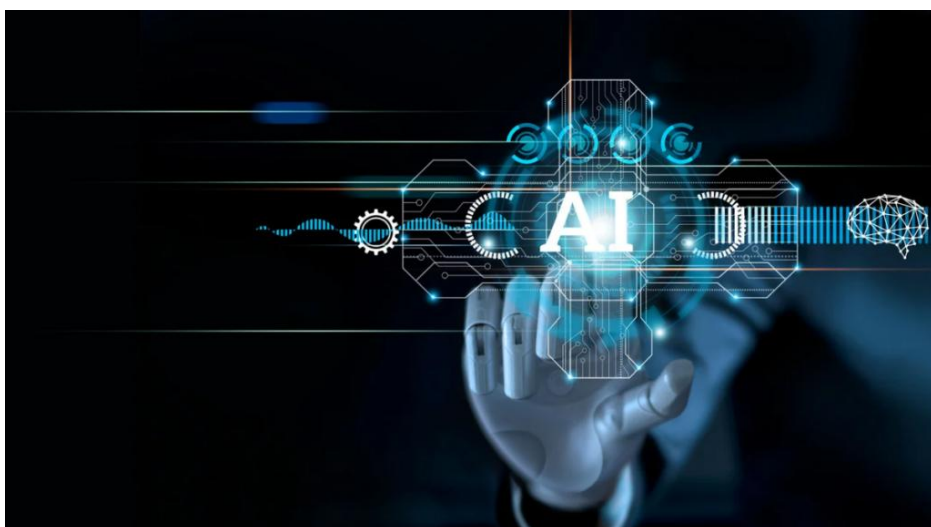




République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université Djillali Liabès de Sidi Bel Abbès
Faculté de Génie Electrique



Polycopié de cours

Techniques de l'intelligence Artificielle

Elaboré par
Dr. Khayreddine SAIDI
Maître de conférences classe A

Ce cours est destiné aux étudiants de
deuxième année Master
Electrotechnique
Spécialité : Commande électrique

Année universitaire : 2025/2026

Avant propos

Le présent polycopié, intitulé « Techniques d'Intelligence Artificielle », a été élaboré afin de constituer un support pédagogique adapté aux étudiants de deuxième année Master en Électrotechnique, spécialité Commande Électrique. Il a également été pensé pour être utile aux étudiants de deuxième année Master en Automatique, compte tenu des nombreuses similitudes et interactions entre ces deux disciplines dans le domaine du contrôle et de l'optimisation des systèmes complexes.

L'objectif principal de ce document est de fournir une vision claire et progressive des principales approches en intelligence artificielle appliquées à l'ingénierie. Le contenu a été conçu pour permettre à l'étudiant de saisir à la fois les fondements théoriques et les méthodes pratiques, tout en offrant des exemples concrets d'utilisation dans le domaine de la commande des systèmes dynamiques.

La structure de ce document reflète une progression pédagogique allant des concepts fondamentaux aux architectures hybrides et à leurs applications :

- Le premier chapitre dresse une vue d'ensemble du « Soft Computing », posant le cadre philosophique et méthodologique des techniques qui seront détaillées.
- Les deuxième et troisième chapitres présentent les deux piliers que sont la Logique Floue, pour le traitement des connaissances imprécises et le raisonnement linguistique, et les Réseaux de Neurones Artificiels, pour l'apprentissage à partir de données et l'approximation de fonctions.
- Le quatrième chapitre, consacré aux Réseaux Neuro-Flous, illustre la puissance de l'hybridation en combinant la capacité d'apprentissage des réseaux de neurones avec la transparence et le raisonnement sémantique de la logique floue.
- Le cinquième chapitre aborde les Techniques d'Optimisation Métaheuristiques, telles que les Algorithmes Génétiques et l'Optimisation par Essaims de Particules (PSO), indispensables pour résoudre des problèmes d'optimisation complexes, multivariables et non convexes, souvent rencontrés dans la conception de lois de commande ou le réglage de paramètres.
- Le sixième chapitre introduit les concepts de Probabilité et de Raisonnement Probabiliste, fournissant le socle mathématique pour raisonner dans l'incertain et faire face au bruit et à l'aléa.
- Enfin, le septième chapitre présente les Systèmes Experts, qui matérialisent la capture et l'exploitation automatisée de l'expertise humaine, avec des applications en aide à la décision.

Chaque chapitre a été conçu pour allier rigueur académique et accessibilité, afin de favoriser une meilleure assimilation des notions. Le lecteur y trouvera non seulement des explications théoriques, mais également des exemples et des schémas permettant d'ancrer les concepts dans le domaine de la commande de systèmes en électrotechniques et en automatique.

Il est important de souligner que ce polycopié n'a pas la prétention d'épuiser la richesse et la diversité des techniques d'intelligence artificielle. Il constitue néanmoins une base solide pour aborder ce domaine en pleine évolution et ouvre la voie à des approfondissements plus spécialisés. Enfin, ce travail est le fruit d'un effort de synthèse visant à rendre accessible un ensemble de connaissances dispersées dans la littérature. L'auteur espère que ce support contribuera à enrichir la formation des étudiants, à susciter leur intérêt pour l'intelligence artificielle et à leur fournir des outils conceptuels et pratiques utiles pour leurs projets académiques et professionnels.

Table des matières

Avant propos.....	i
-------------------	---

Introduction générale	- 1 -
-----------------------------	-------

Chapitre 01 : Généralités sur le "soft computing"

1.1. Introduction	- 2 -
1.2. Définition du soft computing	- 3 -
1.3. Soft Computing vs Hard Computing.....	- 3 -
1.4. Applications du Soft Computing.....	- 4 -
1.5. Composants du Soft Computing	- 5 -

Chapitre 02 : Logique floue et ses applications

2.1. Introduction	- 6 -
2.2. Qu'est-ce que la logique floue ?.....	- 6 -
2.3. Un peu d'histoire	- 7 -
2.4. Quelques applications de la logique floue.....	- 8 -
2.5. Théorie des sous-ensembles flous	- 8 -
2.5.1. Définitions.....	- 9 -
2.5.2. Caractéristiques des fonctions d'appartenance	- 11 -
2.5.3. Probabilités et logique floue	- 13 -
2.5.4. Opérateurs en logique floue	- 13 -
2.6. La commande par logique floue.....	- 16 -
2.6.1. Pourquoi un contrôle floue ?.....	- 16 -
2.6.2. Structures du régulateur flou.....	- 16 -
2.6.3. Fuzzification	- 17 -
2.6.4. Base de connaissances	- 24 -
2.6.5. Mécanisme d'inférence	- 25 -
2.6.6. Défuzzification.....	- 34 -
2.6.7. Différents types de contrôleurs flous	- 36 -
2.6.8. Construction des différentes classes des contrôleurs flous	- 38 -
2.6.9. Organigramme de la conception d'un contrôleur flou	- 41 -
2.7. Avantages et inconvénients de la commande par logique floue	- 42 -
2.8. Conclusion.....	- 43 -

Chapitre 03 : Réseaux de neurones artificiels

3.1. Introduction	- 44 -
-------------------------	--------

3.2. Aspect historique.....	- 46 -
3.3. Le neurone biologique.....	- 47 -
3.4. Le neurone formel	- 49 -
3.5. Description mathématique.....	- 51 -
3.6. Les réseaux de neurones artificiels.....	- 53 -
3.6.1. Différentes architectures des réseaux de neurones artificiels	- 53 -
3.6.1.1. Les réseaux de neurones à propagation avant (Feedforward ou non bouclés).....	- 54 -
3.6.1.2. Réseaux de neurones à connexions locales.....	- 56 -
3.6.1.3. Les réseaux de neurones récurrents (feedback network ou bouclés).....	- 56 -
3.7. Quelques modèles des RNA.....	- 57 -
3.7.1. Le réseau de Kohonen.....	- 57 -
3.7.2. Le réseau de Hopfield	- 58 -
3.7.3. Le réseau ART	- 58 -
3.7.4. Le Perceptron	- 59 -
3.7.5. Le Perceptron multicouche	- 60 -
3.7.6. Réseau ADALINE	- 60 -
3.8. L'apprentissage	- 61 -
3.8.1.1. La classification	- 63 -
3.8.1.2. La régression	- 64 -
3.8.2. Apprentissage non supervisé.....	- 64 -
3.8.3. Apprentissage par renforcement	- 66 -
3.9. Algorithmes d'apprentissage.....	- 67 -
3.9.1. La règle de Hebb	- 67 -
3.9.2. La règle du perceptron	- 68 -
3.9.2.1. Exemple d'apprentissage par l'algorithme du Perceptron	- 70 -
3.9.3. La règle de Windrow-Hoff (la règle delta)	- 72 -
3.9.4. Apprentissage compétitif	- 73 -
3.9.4. Algorithme de la retro-propagation du gradient d'erreur (Back-propagation).....	- 73 -
3.10. Mise en œuvre du réseau de neurone multicouche	- 76 -
3.11. Domaines d'application des Réseaux de Neurones.....	- 77 -
3.12. Conclusion.....	- 78 -

Chapitre 04 : Réseaux neuro-flous

4.1. Introduction	- 79 -
4.2. Définition des réseaux neuro-flous	- 79 -

4.3. Techniques de combinaison neuro-floues	- 80 -
4.3.1. Réseaux flous neuronaux	- 81 -
4.3.2. Systèmes neuro-flous coopératifs	- 81 -
4.3.3. Systèmes neuro-flous concurrents	- 82 -
4.3.4. Systèmes neuro-flous hybrides	- 83 -
4.4. Quelques modèles neuro-flous	- 83 -
4.4.1. ANFIS (Adaptive-Network-based Fuzzy Inference System)	- 83 -
4.4.1.1. Apprentissage de l'ANFIS	- 86 -
4.4.2. NEFCLASS (Neuro-Fuzzy CLASSification)	- 86 -
4.4.3. NEFCON (Neuro-Fuzzy Controller)	- 87 -
4.4.4. NEFPROX (Neuro Fuzzy function apPROXimator).....	- 88 -
4.5. Conclusion.....	- 88 -

Chapitre 05 : Techniques d'optimisation métaheuristiques

5.1. Introduction	- 89 -
5.2. Optimisation	- 90 -
5.2.1. Définition	- 90 -
5.2.2. Problème d'optimisation.....	- 91 -
5.2.3. Méthodes d'optimisation	- 91 -
5.2.3.1. Méthodes déterministes (classiques).....	- 92 -
5.2.3.2. Méthodes non-déterministes (Métaheuristiques)	- 92 -
5.3. Les algorithmes génétiques	- 94 -
5.3.1. Terminologie et concepts de base des algorithmes génétiques.....	- 95 -
5.3.1.1. Génotype et phénotype.....	- 95 -
5.3.1.2. La population	- 96 -
5.3.1.3. Les individus	- 96 -
5.3.1.4. Les chromosomes.....	- 96 -
5.3.1.5. Les gènes.....	- 96 -
5.3.2. Méthodologie adoptée.....	- 97 -
5.3.2.1. Le codage	- 99 -
5.3.2.2. La fonction objectif (Fitness).....	- 100 -
5.3.2.3. Générer une population initiale.....	- 100 -
5.3.2.4. L'opérateur de sélection.....	- 101 -
5.3.2.5. L'opérateur de croisement	- 102 -

5.3.2.6. L'opérateur de mutation.....	- 104 -
5.3.2.7. L'opérateur d'élitisme.....	- 105 -
5.3.2.8. Test d'arrêt	- 105 -
5.3.3. Paramètres de l'algorithme génétique.....	- 105 -
5.3.4. Exemple de résolution à base d'algorithme génétique.....	- 105 -
5.4. Optimisation par Essaim de particules (PSO)	- 110 -
5.4.1. Origine et inspiration du PSO	- 111 -
5.4.2. Topologies de voisinage	- 112 -
5.4.3. Principe de fonctionnement de l'algorithme du PSO.....	- 113 -
5.4.3.1. Formalisation	- 114 -
5.4.3.2. Choix des paramètres de l'algorithme	- 116 -
5.5. Conclusion.....	- 118 -

Chapitre 06 : Probabilité et raisonnement probabiliste

6.1. Introduction	- 119 -
6.2. Raisonnement probabiliste	- 119 -
6.2.1. Théorie des probabilités en intelligence artificielle	- 120 -
6.2.2. Rappels sur des notions de probabilités	- 120 -
6.2.3. Exemple illustratif.....	- 121 -
6.3. Les réseaux bayésiens	- 126 -
6.3.1. Définition	- 126 -
6.3.2. Exemple illustratif des RB	- 127 -
6.3.3. Indépendance conditionnelle et d-séparation.....	- 130 -
6.3.4. Construction d'un réseau bayésien	- 131 -
6.3.4.1. Apprentissage de la structure	- 132 -
6.3.4.2. Apprentissage des paramètres.....	- 132 -
6.3.5. Inférence dans les réseaux bayésiens	- 132 -
6.5. Conclusion.....	- 133 -

Chapitre 07 : Systèmes experts et leurs applications

7.1. Introduction	- 135 -
7.2. Définition d'un système expert	- 135 -
7.3. Quelques notions fondamentales.....	- 136 -
7.3.1. Les faits	- 136 -
7.3.2. Les règles	- 137 -

7.3.3. Le diagnostic	- 138 -
7.4. Architecture d'un système expert.....	- 138 -
7.4.1. La base de connaissances	- 139 -
7.4.2. Le moteur d'inférence.....	- 139 -
7.4.2.1. Les caractéristiques d'un moteur d'inférence	- 139 -
7.4.2.2. Stratégie et raisonnement du moteur d'inférence	- 140 -
7.5. Méthodologie de réalisation d'un système expert.....	- 142 -
7.6. Avantages et inconvénients d'un système expert.....	- 143 -
7.6.1. Avantages d'un système expert.....	- 143 -
7.6.2. Inconvénients d'un système expert	- 144 -
7.7. Quelques domaines d'applications des systèmes experts	- 144 -
7.8. Exemple d'un système expert	- 145 -
7.9. Conclusion.....	- 145 -
Conclusion générale	- 145 -
Références bibliographiques	- 145 -

Introduction générale

L'intelligence artificielle (IA, ou AI en anglais pour Artificial Intelligence) est l'un des sujets qui attirent beaucoup d'attention ces dernières années et qui affectent notre époque. Rarement une évolution technologique n'aura engendré autant d'opportunités de résolutions de problèmes, autant de changements dans les usages, autant d'intérêts.

Pourtant, il ne s'agit absolument pas d'une rupture technologique. L'intelligence artificielle s'inscrit dans la continuité de l'informatique dont la puissance de calcul ne cesse de croître, augmentée par la disponibilité de grandes masses de données que le monde Internet sait agréger.

L'intelligence artificielle ne se résume pas à résoudre des problèmes abstraits ; elle permet maintenant aux voitures de rouler sans conducteurs, aux robots de devenir de plus en plus autonomes, aux médecins de faire des diagnostics plus fins, aux avocats de faire des contrats plus précis. Après l'information, puis la connaissance, c'est maintenant au tour de l'expertise d'être disponible partout, accessible à tous. Sa rareté, qui jusqu'ici a été source d'un profit légitime mais considérable, est en passe de se transformer en abondance. Seuls les experts les plus pointus, qui auront compris comment tirer parti de la nouveauté qu'apporte l'intelligence artificielle, survivront.

Le terme « intelligence artificielle » voit le jour au cours de l'année 1956, où plusieurs scientifiques de très haut rang se sont réunis pendant des semaines dans une conférence à Dartmouth, généralement considérée comme fondatrice du domaine de l'intelligence artificielle. Parmi eux se trouvaient John McCarthy (inventeur du terme « Intelligence Artificielle ») et Marvin Minsky qui donna la première définition de l'IA : “Construction de programmes informatiques qui s'adonnent à des tâches qui sont, pour l'instant, accomplies de façon plus satisfaisantes par des êtres humains car elles demandent des processus mentaux de haut niveau tels que l'apprentissage perceptuel, l'organisation de la mémoire et le raisonnement critique.”.

Nous pouvons donc donner une définition simple pour L'IA : “ elle consiste à mettre en œuvre un certain nombre de techniques visant à permettre aux machines d'imiter d'une certaine manière l'intelligence humaine ”. Actuellement, L'IA se retrouve implémentée dans un nombre grandissant de domaines d'application : les jeux, la planification, les systèmes à base de connaissances, la traduction automatique, diagnostic médical, navigation autonome (voitures, robots, drones, avions, etc.), fouille de données, identification vocale ou visuelle, traitement d'images, etc

Chapitre 01

Généralités sur le "soft computing"

1.1. Introduction

Un des problèmes rencontrés lors de la conception des systèmes de commande traditionnels est que les systèmes complexes ne peuvent être décrits avec précision par des modèles mathématiques et sont donc difficiles à contrôler à l'aide de ces méthodes existantes. Le soft computing (SC), une approche innovante de la construction de systèmes intelligents, vient d'entrer en lumière. Cette approche a été introduite par L.A. Zadeh (qui est connu comme le pionnier de la logique floue) dans les années quatre-vingt-dix, comme un moyen de construction des systèmes intelligents répondant à des obligations d'efficacité, de robustesse, de facilité d'implémentation et d'optimisation de coûts temporels, énergétiques, financiers, etc. Mais aussi, en tenant en compte la composante humaine généralement présente dans les systèmes, puisqu'il traite de la vérité partielle, de l'incertitude et de l'approximation pour résoudre des problèmes complexes. L.A. Zadeh a donné la définition suivante « le principe directeur du soft computing est d'exploiter la tolérance à l'imprécision, à l'incertitude et à la vérité partielle pour atteindre la maniabilité, la robustesse, un faible coût des solutions et un meilleur rapport avec la réalité ». En raison de ses caractéristiques telles que le contrôle intelligent, la programmation non linéaire, l'optimisation et le soutien à la prise de décision, le soft computing est devenu populaire et a attiré l'intérêt des chercheurs et des scientifiques dans différents domaines.

Il est difficile de contrôler la complexité croissante des machines modernes en utilisant des techniques de commande traditionnelles. Par exemple, de nombreux systèmes non linéaires, variant dans le temps et présentant de grands retards ne peuvent être facilement contrôlés et stabilisés à l'aide de techniques traditionnelles. Une des raisons de cette difficulté est l'absence d'un modèle précis qui décrit le système. Le soft computing s'avère être un moyen efficace pour contrôler des systèmes complexes.

Le soft computing n'est pas une méthode unique, mais plutôt une combinaison de plusieurs méthodes, telles que la logique floue, les réseaux de neurones, les algorithmes génétiques, etc. Toutes ces méthodes ne sont pas concurrentielles, mais se complètent mutuellement et peuvent être utilisées ensemble pour résoudre un problème donné. On peut dire que le soft computing vise à résoudre des problèmes complexes en exploitant l'imprécision et l'incertitude dans les processus décisionnels.

1.2. Définition du soft computing

C'est une approche issue en premier lieu de l'informatique qui imite la remarquable capacité de l'esprit humain à raisonner et à apprendre dans un environnement d'incertitude et d'imprécision. Elle se caractérise par l'utilisation de solutions imprécises à des tâches de calculs difficiles telles que la solution de problèmes complexes non paramétriques pour lesquels une solution exacte ne peut être dérivée.

Le terme Soft Computing a été décrit par Lotfi A Zadeh, comme suit : « Le soft Computing est une collection de méthodologies qui visent à exploiter la tolérance pour l'imprécision et l'incertitude pour réaliser la robustesse, et un coût de solution minimal ». Ses principaux constituants sont logique floue, les réseaux de neurones, et le raisonnement probabiliste. Le soft Computing est susceptible de jouer un rôle de plus en plus important dans beaucoup de domaines d'application, y compris la technologie de la programmation. Le modèle de rôle pour le soft computing est l'esprit humain.

1.3. Soft Computing vs Hard Computing

Il existe deux approches de calcul pour la résolution des problèmes : Hard computing et soft computing. Le Hard computing (calcul traditionnel) repose sur les principes de précision, de certitude et d'inflexibilité. Il faut un modèle mathématique pour résoudre les problèmes. Il traite des modèles précis des problèmes à résoudre. L'approche de solution comprend les composants du raisonnement symbolique et logique et des modèles de la recherche numériques traditionnelles. Le soft computing n'assume pas un modèle précis mais un modèle approximatif du problème et l'approche de solution comprend les composants du raisonnement approximatif. La figure 1.1 montre le principe de la résolution de problèmes basée sur le Hard computing et le Soft computing.

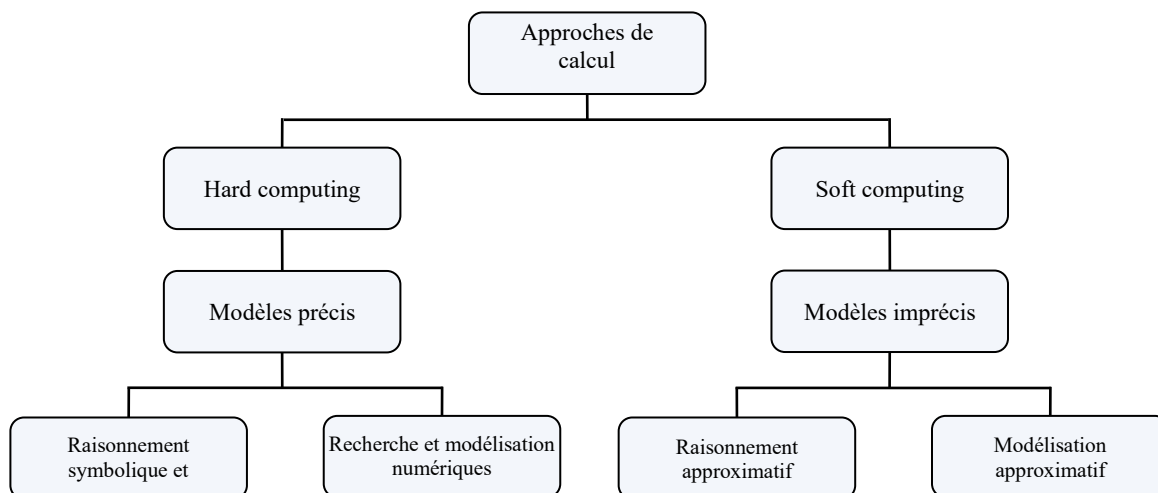


Figure 1.1. Approches de résolution des problèmes

Le tableau ci-dessous montre la différence entre les deux approches :

Hard Computing	Soft Computing
Utilise des modèles analytiques précis.	Tolère l'imprécision, l'incertitude, la vérité partielle et l'approximation.
Basé sur la logique binaire et des systèmes nets.	Repose sur une logique floue et un raisonnement probabiliste.
A des caractéristiques telles que la précision et la rigueur.	Présente des caractéristiques telles que le rapprochement et la disposition.
De nature déterministe.	Est de nature stochastique.
Peut fonctionner avec des données d'entrée exactes.	Peut fonctionner avec des données ambiguës et bruitées.
Effectue des calculs séquentiels.	Effectue des calculs parallèles.
Produit des résultats précis.	Produit un résultat approximatif.

Tableau 1.1. Soft computing vs Hard computing

1.4. Applications du Soft Computing

L'application du Soft Computing a démontré les avantages suivants :

- La résolution des applications qui ne peuvent pas être modélisés mathématiquement.
- Les problèmes non linéaires peuvent être résolus.
- Introduire les connaissances humaines telles que la cognition, la compréhension, la reconnaissance, l'apprentissage et d'autres dans le domaine du calcul et de recherche de solution.

Cette approche peut être utilisée dans la plupart des grands domaines où la mise au point de systèmes intelligents pose des problèmes.

- Apprentissage à la commande de processus
- Les bases de données ou le traitement d'images

Il existe des applications du soft computing à des problèmes réels dans des domaines aussi variés que la recherche d'information :

- La fouille de données (data mining).
- L'aide à la décision.
- Traitement des images et compression des données.
- La robotique.
- Le contrôle de systèmes complexes.
- Classification.
- Détection d'anomalies.
- La surveillance.

- Commande de tâches.
- Analyse de risque.
- Optimisation de revenu (applications financières).

Ainsi que d'autres applications dans diverse domaines.

1.5. Composants du Soft Computing

Les composants les plus populaires du soft computing dans la conception de la théorie du contrôle intelligent sont :

- ❖ La logique floue.
- ❖ Les réseaux de neurones.
- ❖ les algorithmes évolutionnaires (algorithmes génétiques, etc.).

Dans la suite de ce cours nous allons voir une par une toutes ces techniques

2.1. Introduction

La logique floue a été formulée par L.A. ZADEH dans le milieu des années 60. Elle sert à pouvoir programmer un ordinateur pour qu'il contrôle une machine un peu comme le ferait un être humain.

A première vue, la logique floue est une généralisation de la logique booléenne classique. Elle ajoute cependant une fonctionnalité déterminante : la possibilité de calculer un paramètre en disant simplement dans quelle mesure il doit se trouver dans telle ou telle zone de valeur.

Le principe du réglage par logique floue part du constat suivant : dans les problèmes de régulation auxquels il est confronté, l'homme ne suit pas, à l'image de ses inventions, un modèle mathématique fait de valeurs numériques et d'équations. Au contraire il utilise des termes tels que « un peu trop chaud, aller beaucoup plus vite, freiner à fond, etc... » ainsi que ses propres connaissances qu'il a dans le domaine. Ces connaissances sont, le plus souvent, acquises de façon empirique. Le principe du réglage par la logique floue s'approche de la démarche humaine dans le sens que les variables traitées ne sont pas des variables logiques (au sens de la logique binaire par exemple) mais des variables linguistiques, proches du langage humain de tous les jours. De plus, ces variables linguistiques sont traitées à l'aide de règles qui font référence à une certaine connaissance du comportement du système à régler. Sur la base de ce principe, différentes réalisations ont vu le jour et, actuellement, on trouve deux types d'approche pour le réglage par logique floue. Dans l'une de ces approches, les règles sont appliquées aux variables à l'aide d'une approche numérique par le biais d'un microprocesseur spécialisé ou non ou d'un ordinateur. Dans l'autre approche, les règles sont appliquées aux variables de façon analogique.

En effet, contrairement aux méthodes de réglage classiques qui reposent sur une modélisation adéquate et un traitement analytique du système à commander, le réglage par logique floue constitue une approche très pragmatique en ce sens qu'il fait appel à l'expérience et aux connaissances de l'opérateur humain. Elle se prête aussi à la commande et au réglage des processus très peu ou mal connus et donc mal maîtrisables par les méthodes classiques.

2.2. Qu'est-ce que la logique floue ?

Une des caractéristiques du raisonnement humain est qu'il est basé sur des données imprécises ou incomplètes.

Ainsi déterminer si une personne est de petite ou de grande taille est aisé pour n'importe lequel d'entre nous, et cela sans nécessairement connaître sa taille. Un ordinateur, lui, est basé sur des

données exactes. Il doit non seulement connaître la taille exacte de la personne mais également posséder un algorithme qui divise immanquablement une population en deux groupes bien distincts : les grands et les petits.

Supposons que la limite soit de 1m65. Je mesure 1m63, suis-je vraiment petit ? Tel qu'il est montré dans la figure 2.1.

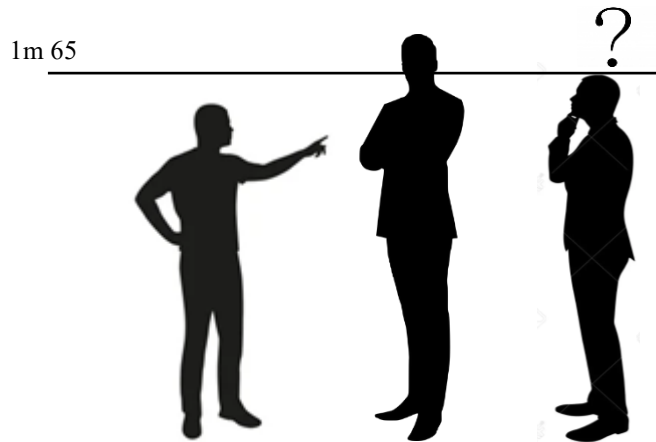


Figure 2.1. Exemple de raisonnement humain

L'idée de la logique floue est de transmettre cette richesse du raisonnement humain à un ordinateur.

Bien que dans l'esprit de tout le monde le mot « flou » soit de connotation négative, il n'en est rien en réalité. Venant à l'origine du mot « duvet » (en anglais « fuzz », c'est à dire le duvet qui couvre le corps des poussins), le terme « fuzzy » signifie 'indistinct, brouillé, mal défini ou mal focalisé'. L'adjectif se traduit par « flou » en français. Dans le monde universitaire et technologique, le mot « flou » est un terme technique représentant l'ambiguïté ou le caractère vague des intuitions humaines plutôt que la probabilité (ce point est traité plus en détails dans les prochains paragraphes).

2.3. Un peu d'histoire

Les quelques points de repères historiques suivants permettent de situer dans le temps le développement de la logique floue et ses applications au réglage :

- ✚ 1965 : Le Prof. L. A. Zadeh de l'Université de Berkeley (Californie) pose les bases théoriques de la logique floue. Et la naissance du concept flou.
- ✚ 1973 : L. A. Zadeh propose d'appliquer la logique floue aux problèmes de réglage.
- ✚ 1973 : Première application du réglage par la logique floue appliquée à une turbine à vapeur. Suivie en 1980 par une application sur un four à ciment et en 1983 sur un épurateur d'eau.
- ✚ 1985 : Premiers produits industriels (Japon) utilisant le principe de la logique floue appliqué à des problèmes de réglage et de commande.
- ✚ 1987 : explosion du flou au Japon (avec le contrôle du métro de Sendai) et qui atteint son apogée en 1990.

✚ 1990 à ce jour : Généralisation de l'utilisation de cette technique.

2.4. Quelques applications de la logique floue

Les domaines de recherche et d'application de la logique floue sont très nombreux. On la retrouve en :

- ✚ En automatique, pour faire de la commande et de la régulation floue, etc.
- ✚ En traitement du signal, pour faire de la fusion de données, de la classification, etc.
- ✚ En robotique, pour faire du control et de la planification de trajectoire, etc.
- ✚ En traitement d'image, pour atténuer le bruit d'une image, pour faire de l'interpolation, de la reconnaissance de forme ou de la recherche d'information, etc.
- ✚ etc.

La logique floue intervient donc dans de nombreux secteurs d'activités industrielles et des services :

- ✚ En médecine (diagnostic, guidage de systèmes chirurgicaux (laser chirurgie de l'œil par exemple), etc.)
- ✚ Contrôle aérien
- ✚ Gestion du Trafic urbain
- ✚ Finances et assurances (préventions des risques, aide à la décision)
- ✚ Environnement (météo, etc.)
- ✚ Robotique (assistance au gens endicapés, robotique mobile, appareils électroménagers, etc.)
- ✚ systèmes automobiles embarqués (BVA, ABS, suspension, climatisation,...etc.)
- ✚ etc.

2.5. Théorie des sous-ensembles flous

La logique booléenne classique ne permet que deux états : VRAI ou FAUX. La logique floue permet d'exprimer différents niveaux, plutôt que seulement 1 ou 0. Par exemple : la température est élevée, la température est très élevée. Quelle est la différence entre « élevée » et « très élevée » ? Ou encore, un homme est long s'il mesure 170cm. Un homme est très long s'il mesure 190cm. Où est la ligne de démarcation ?

Un homme de 180cm est-il haut ou très haut ? 180.5cm ? 179.5cm ?

Donc, La théorie des ensembles flous permet de manipuler des notions imprécises, des valeurs approximatives et d'utiliser un processus de raisonnement proche du langage naturel, comme le fait intuitivement un opérateur humain (par exemple un conducteur de véhicule).

2.5.1. Définitions

Soit une variable x (la taille) et un univers de référence ou de discours U (les individus).

Un sous-ensemble flou A est défini par une fonction d'appartenance $\mu_A(x)$ qui décrit le degré avec lequel l'élément x appartient à A .

$$A = \{(x, \mu_A(x)) / x \in U\}$$

Plus la valeur de $\mu_A(x)$ s'approche de 1, plus élevé est le taux d'appartenance de l'élément x à l'ensemble A . Si $\mu_A(x) = 1$ l'élément x appartient complètement à l'ensemble A . Si $\mu_A(x) = 0$ l'élément x n'appartient pas du tout à l'ensemble A .

Le tableau suivant (tableau. 2.1)) montre un exemple d'une comparaison entre la théorie classique et la théorie floue.

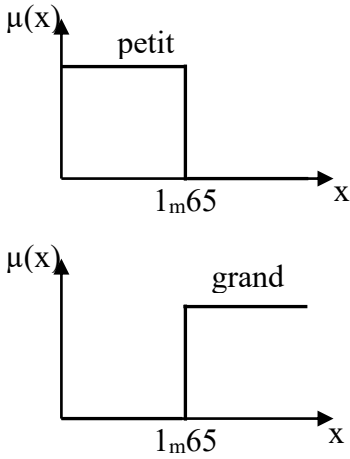
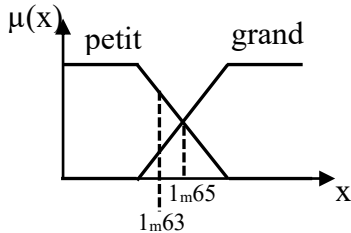
Théorie classique	Théorie floue
Mathématiquement : $\mu(x) = \begin{cases} 1 & \text{si } x \in A \\ 0 & \text{sinon} \end{cases}$	Mathématiquement : $\mu(x) \in [0, 1]$
Représentation graphique	Représentation graphique
 Le graphique de la théorie classique illustre deux sous-ensembles discrets. Le premier, 'petit', est représenté par un rectangle de hauteur 1 sur l'axe vertical $\mu(x)$ et de largeur jusqu'à la valeur 1m65 sur l'axe horizontal x. Le second, 'grand', est représenté par un rectangle de hauteur 1 commençant à la valeur 1m65 sur l'axe horizontal x.	 Le graphique de la théorie floue illustre deux sous-ensembles continus. Le premier, 'petit', est représenté par une fonction qui décroît linéairement de 1 à 0 entre les valeurs 1m63 et 1m65 sur l'axe horizontal x. Le second, 'grand', est représenté par une fonction qui croît linéairement de 0 à 1 entre les mêmes valeurs 1m63 et 1m65. Les deux fonctions se croisent à la valeur 1m63.
Ici un individu qui mesure 1m63 appartient nécessairement au sous-ensemble PETIT (avec un degré 1 c-à-d 100%)	Ici un individu qui mesure 1m63 appartient en même temps au sous-ensemble flou PETIT et au sous-ensemble flou GRAND respectivement avec un degré de 0,7 et de 0,3 .

Tableau 2.1. Comparaison entre la théorie classique et la théorie floue

Les sous-ensembles flous peuvent être décrits de nombreuses façons ; ils permettent tous d'exprimer une adhésion partielle. Soit l'univers $U = \{x_1, x_2, \dots, x_n\}$ alors un ensemble flou A de U peut se représenter par :

$$A = \mu_A(x_1)/x_1 + \mu_A(x_2)/x_2 + \dots + \mu_A(x_n)/x_n \quad (2.1)$$

Le symbole / ici ne dénote pas une division, ni le symbole + dénote une sommation. Le symbole de sommation est utilisé pour relier les termes et signifie donc une union de sous-ensembles à un seul terme.

Soit l'univers $U = \{x_1, x_2, x_3, x_4\}$. On peut définir un sous-ensemble flou comme :

$$A = 0.8/x_1 + 0.4/x_2 + 0.1/x_3 + 0.9/x_4$$

Remarque :

La logique floue permet de tenir compte de la nature imprécise du monde réel, grâce à des termes flous ou termes linguistiques (ex : "Petit", "Moyen", "Grand"). Chaque terme représente un sous-ensemble de valeurs numériques et caractérise ainsi la variable floue (ex : "Température"). Le domaine sur lequel ces termes et ces variables sont définies constitue l'univers de discours ou le domaine de définition (référentiel), ex : les réels positifs pour la variable "Taille". Une variable linguistique est présentée par un triplet {nom de la variable, domaine de définition, ensemble de valeurs linguistiques}.

Le découpage de cet univers de discours par les termes flous est appelé une partition floue.

Donc, dans le jargon de cette théorie, on parle de variable linguistique (taille, température, etc.) et de valeurs linguistiques (petit, grand, moyen, chaud, etc.).

Exemple :

Pour représenter la logique floue sur ordinateur, il faut déterminer la fonction d'appartenance. On utilise le plus souvent une représentation graphique. Les bornes de ces graphiques proviennent d'experts dans le domaine. La figure 2.2 montre deux exemples de représentation de la température, une en logique classique, et l'autre en logique floue.

Selon la figure 2.2, en logique classique, une température de 22.5 est considérée comme élevée. En logique floue, une température de 22.5 appartient au groupe "moyenne" avec un degré d'appartenance de 0.167, et appartient au groupe "élevée" avec un degré d'appartenance de 0.75.

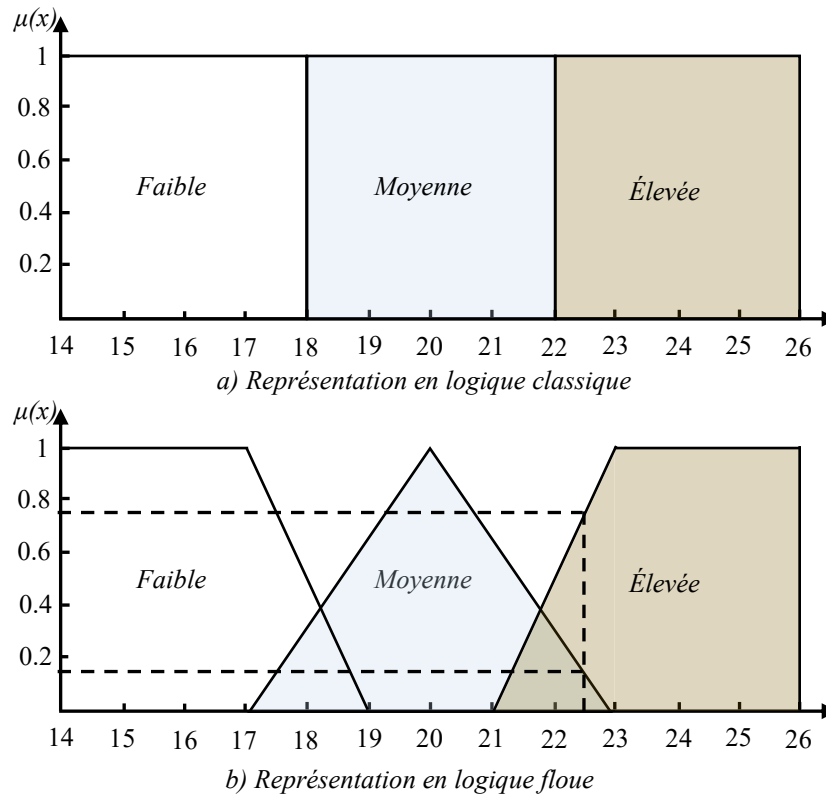


Figure 2.2. Exemple de Comparaison de l'appartenance de la température en logique classique vs la logique floue

Les variables floues faibles, moyennes et élevée sont représentées par des fonctions linéaires. D'autres fonctions auraient pu être utilisées, comme des trapézoïdes, des paraboles, etc. Cependant, les fonctions linéaires sont beaucoup plus faciles à implémenter de façon pratique, et donnent de bons résultats.

On utilise souvent une notation vectorielle pour représenter les fonctions. Pour les fonctions d'appartenance de la figure 2.2, on peut utiliser la notation suivante :

- Température faible : (1/17, 0/19)
- Température moyenne : (0/17, 1/20, 0/23)
- Température élevée : (0/21, 1/23)

2.5.2. Caractéristiques des fonctions d'appartenance

Un sous-ensemble flou est complètement défini par la donnée de sa fonction d'appartenance. A partir d'une telle fonction un certain nombre de caractéristiques du sous-ensemble flou peuvent être étudiées (voir figure 2.3).

➤ **Noyau**

Le noyau d'un sous-ensemble flou A de U , noté $Noy(A)$, est l'ensemble des éléments de U pour lesquels la fonction d'appartenance de A vaut 1 (lui appartiennent totalement) :

$$Noy(A) = \{x \in U / \mu_A(x) = 1\} \quad (2.2)$$

➤ **Support**

Le support d'un sous-ensemble flou A de U , noté $Supp(A)$, est l'ensemble qui contient tous les éléments de A pour lesquels la fonction d'appartenance est non nulle (les éléments qui lui appartiennent au moins un petit peu) :

$$Supp(A) = \{x \in U / \mu_A(x) > 0\} \quad (2.3)$$

Un sous-ensemble flou dont le support est un simple point sur U avec $\mu_A(x) \geq 1$ est nommé singleton.

➤ **Hauteur**

La hauteur d'un sous-ensemble flou A de U , notée $h(A)$, est la valeur maximale prise par sa fonction d'appartenance sur le support de A :

$$h(A) = \sup_{x \in U} (\mu_A(x)) \quad (2.4)$$

Un sous-ensemble flou est normalisé si sa hauteur $h(A)$, est égale à 1.

➤ **Cardinalité**

La cardinalité d'un sous-ensemble flou A de U , notée $|A|$, indique le degré global avec lequel les éléments de U appartiennent à A . Elle est définie par :

$$|A| = \sum_{x \in U} \mu_A(x) \quad (2.5)$$

➤ **α -coupe**

Il peut être utile de décrire un sous-ensemble flou en se référant à des sous-ensembles ordinaires. Une façon de réaliser une approximation d'un sous-ensemble flou consiste à fixer un seuil inférieur sur les degrés d'appartenance. Le sous-ensemble ordinaire A_α de U associé à A pour le seuil α est l'ensemble des éléments qui appartiennent à A avec un degré au moins égal à α . On dit que A_α est l' α -coupe de A :

$$A_\alpha = \{x \in U / \mu_A(x) \geq \alpha\} \quad (2.6)$$

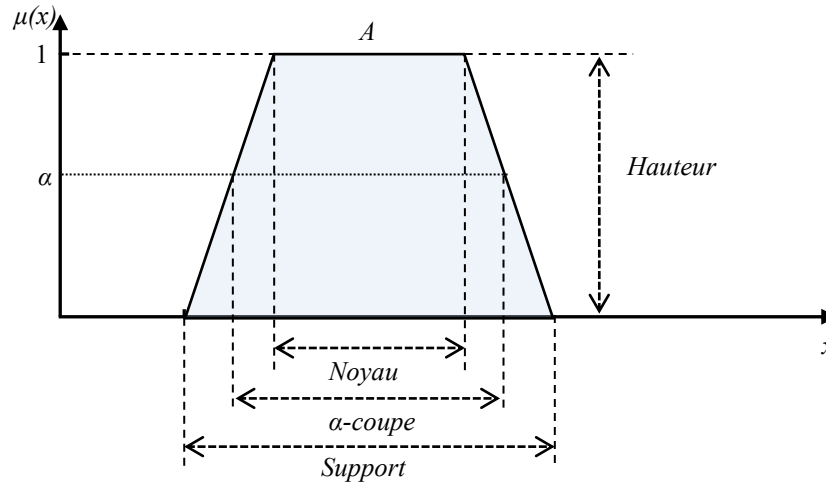


Figure 2.3. Caractéristiques d'un sous-ensemble flou

2.5.3. Probabilités et logique floue

La théorie des probabilités est différente de la théorie de la logique floue bien que toutes deux décrivent une notion de doute, d'incertain et ce à l'aide de nombre compris entre 0 et 1.

$$\mu_A(x) \neq P(x \in A)$$

Mathématiquement, il existe une différence essentielle :

- ❖ **Probabilités** : $P(A \cap \bar{A}) = P(\emptyset) = 0$ et $P(A \cup \bar{A}) = P(U) = 1$
- ❖ **Logique floue** : $P(A \cap \bar{A})$ pas nécessairement $= P(\emptyset)$ et : $P(A \cup \bar{A})$ pas nécessairement $= P(U)$

La logique floue décrit l'ambiguïté d'un évènement alors que les probabilités décrivent le doute quant à l'occurrence d'un évènement.

2.5.4. Opérateurs en logique floue

Les opérateurs flous décrivent comment des ensembles flous interagissent ensemble. Il s'agit de la généralisation de certaines opérations communes, comme le complément, l'intersection et l'union.

❖ **Egalité**

Deux sous-ensemble A et B de U sont dits égaux s'ils ont des fonctions d'appartenance égales en tout point de U . Formellement, $A=B$ ssi :

$$\forall x \in U, \mu_A(x) = \mu_B(x) \quad (2.7)$$

❖ **Inclusion**

Soient deux sous-ensemble A et B de U . Si pour n'importe quel élément x de U , x appartient toujours moins à A qu'à B , alors on dit que A est inclus dans B ($A \subseteq B$). Formellement, $A \subseteq B$ ssi :

$$\forall x \in U, \mu_A(x) \leq \mu_B(x) \quad (2.8)$$

❖ Complément

Le complément d'un sous-ensemble flou A de U est noté $\bar{A}$, et est défini mathématiquement par

$$\bar{A} = \{x/x \notin A\}$$

Sa fonction d'appartenance est représentée par

$$\forall x \in U, \mu_{\bar{A}}(x) = 1 - \mu_A(x) \quad (2.9)$$

Comme exemple, si on a l'ensemble des températures *faibles*, le complément est l'ensemble des températures qui ne sont pas faibles (voir figure 2.4).

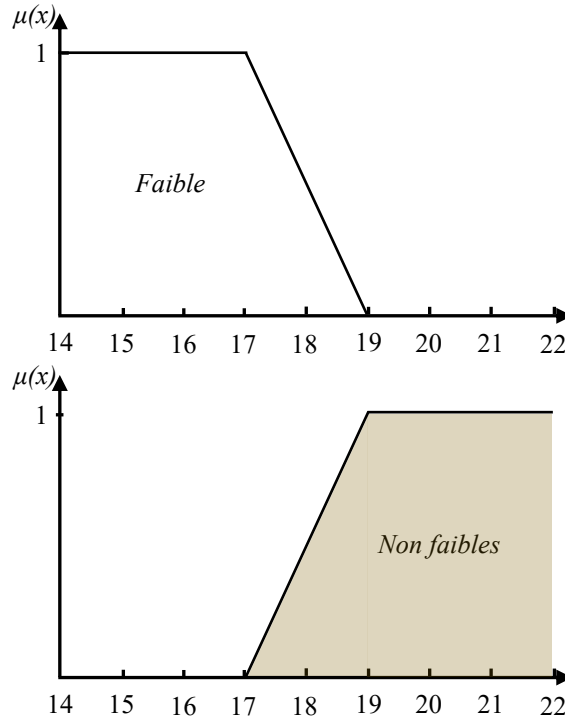


Figure 2.4. Complément d'un ensemble flou

❖ Intersection (L'opérateur ET)

L'intersection de deux sous-ensembles flous A et B de U est le sous-ensemble flou constitué des éléments de U affectés du plus petit des degrés avec lesquels ils appartiennent à A **ET** B . Formellement, $A \cap B = \{x / x \in A \wedge x \in B\}$ est donné par

$$\mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x)) \quad (2.10)$$

Sur la figure 2.5, on peut voir un exemple d'intersection entre les deux sous-ensembles flous des températures faibles et moyennes.

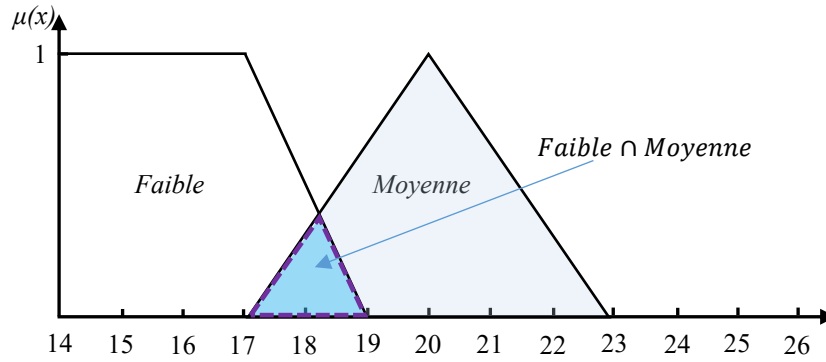


Figure 2.5. Intersection de deux sous-ensembles flous avec l'opérateur min

❖ Union (L'opérateur OU)

L'union de deux sous-ensembles flous A et B de U est le sous-ensemble flou constitué des éléments de U affectés du plus grand des degrés avec lesquels ils appartiennent à A **OU** B. Formellement, $A \cup B = \{x / x \in A \vee x \in B\}$ est donné par

$$\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x)) \quad (2.11)$$

Sur la figure 2.6, on peut voir un exemple d'union entre les deux sous-ensembles flous des températures faibles et moyennes.

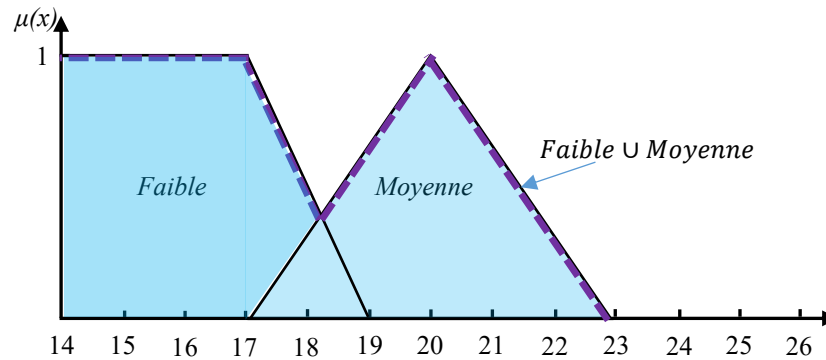


Figure 2.6. Union de deux sous-ensembles flous avec l'opérateur max

En fait, il existe de nombreuses possibilités pour représenter les opérateurs ET et OU. Les fonctions que l'on avait prises juste avant (MIN et MAX) sont dues à zadeh et sont encore aujourd'hui les plus utilisées. Le tableau ci-dessous récapitule plusieurs fonctions représentant les opérateurs ET et OU.

Appellation	ET	OU	NON
Zadeh	$\text{Min}(x,y)$	$\text{Max}(x,y)$	$1-x$
probalistique	xy	$x+y-xy$	$1-x$
lukasiewicz	$\text{Max}(x+y-1,0)$	$\text{Min}(x+y,1)$	$1-x$
Hamacher ($\beta > 0$)	$xy / (\beta + (1-\beta)(x+y-xy))$	$(x+y+xy-(1-\beta)xy) / (1-(1-\beta)xy)$	$1-x$
weber	$x \text{ si } y=1, y \text{ si } x=1$ Sinon 0	$x \text{ si } y=0, y \text{ si } x=0$ Sinon 1	$1-x$

Tableau 2.2. Les fonctions représentant les opérateurs ET et OU

2.6. La commande par logique floue

2.6.1. Pourquoi un contrôle flou ?

Une des principales applications de la logique floue est la conduite de processus. L'intérêt suscité par ce nouveau type de contrôle ne cesse de croître, et ce dans des domaines très variés. Les raisons en sont principalement une grande souplesse et une relative facilité de conception.

Un contrôleur standard (PID ou autres) demande toujours un modèle le plus précis possible du système à contrôler. Le modèle analytique doit être le plus précis possible afin de représenter au mieux la réalité physique.

Un contrôleur flou ne demande pas de modèle du système à régler. Les décisions sont prises sans avoir recours au modèle analytique. Les algorithmes de réglage se basent sur des règles linguistiques de la forme si.....Alors....

En fait, ces règles peuvent être exprimées en utilisant le langage de tous jours et la connaissance intuitive d'un opérateur humain. Ce qui conduit à deux avantages.

- le contrôle flou reste clair pour tous les opérateurs et utilisateurs de la machine.
- Les opérateurs peuvent aisément interpréter les effets ou conséquences de chaque règle.

2.6.2. Structures du régulateur flou

La figure ci-dessous présente la configuration de base d'un RLF (régulateur par logique floue) il a une structure identique à un système à réglage par feedback classique.

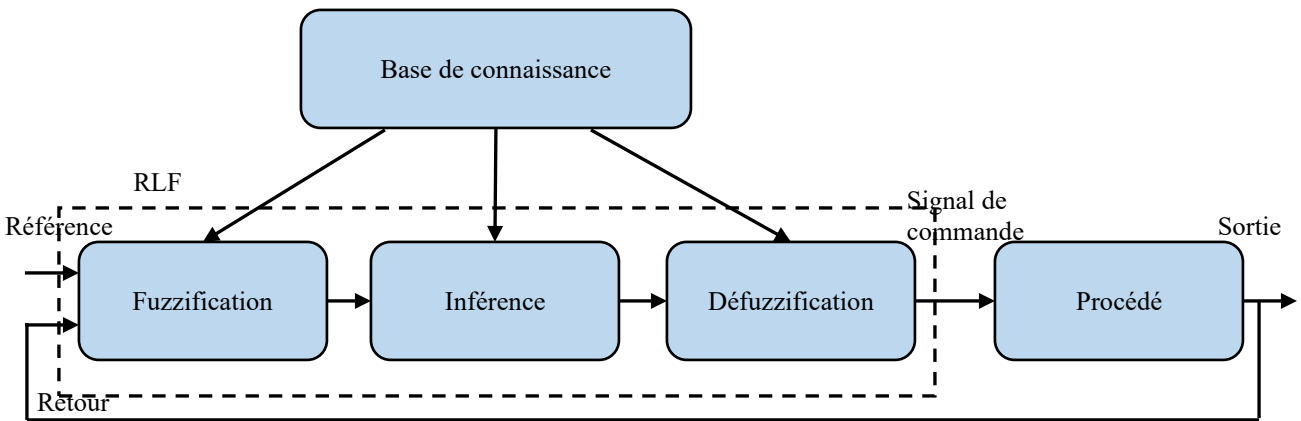


Figure 2.7. La configuration de base d'un RLF

La configuration de base d'un régulateur par logique floue comporte donc, quatre parties principales :

- Une interface de fuzzification,
- Une base de connaissance,
- Un mécanisme d'inférence,
- Et une interface de défuzzification.

2.6.3. Fuzzification

La fuzzification est l'opération de rendre une entrée classique en valeur linguistique. Des valeurs d'entrée sont traduites en concepts linguistiques représentés comme des ensembles flous. Les fonctions d'appartenance sont appliquées aux mesures et des degrés de vérité sont établis pour chaque proposition.

Le bloc fuzzification effectue les fonctions suivantes :

- Établit les plages de valeurs pour les fonctions d'appartenance à partir des valeurs des variables d'entrées ;
- Effectue une fonction de fuzzification qui convertit les données d'entrée en valeurs linguistiques convenables qui peuvent être considérées comme l'étiquette des ensembles flous.

❖ Les fonctions d'appartenance

L'ensemble des valeurs linguistiques que peut prendre la variable floue sont représentées dans la logique floue par des fonctions d'appartenance et chaque élément de cet ensemble à une valeur d'appartenance qui est le degré de compatibilité de cet élément avec le concept qui est représenté par l'ensemble flou. Puisque les données sont généralement numériques, l'univers de discours est le plus souvent un intervalle de nombres réels, ou en pratique, un ensemble fini de nombres réels. La forme d'une fonction d'appartenance dépend de la notion que l'ensemble est destiné à décrire et de l'application concernée. Les fonctions d'appartenance les plus couramment utilisées dans la

théorie du contrôle flou sont du type triangulaires, trapézoïdales, gaussiennes, sigmoïdales, Z- et S-fonctions, comme illustré dans les figures ci-dessous.

Les fonctions d'appartenance de type triangulaire et trapézoïdale, qui sont des fonctions linéaires par morceaux, sont souvent utilisées dans les applications. Les représentations graphiques et les opérations avec ces ensembles flous sont très simples. En outre, elles peuvent être construites facilement sur la base de peu d'informations.

- **La fonction Triangulaire :**

La fonction triangulaire est définie par trois points, (a, 0) et (b, 0) les points extrêmes et (c, 1) le point du sommet. Elle est exprimée par :

$$\mu_A(x) = \begin{cases} \frac{x-a}{c-a} & \text{si } a \leq x \leq c \\ \frac{x-b}{c-b} & \text{si } c \leq x \leq b \\ 0 & \text{ailleurs} \end{cases} \quad (2.12)$$

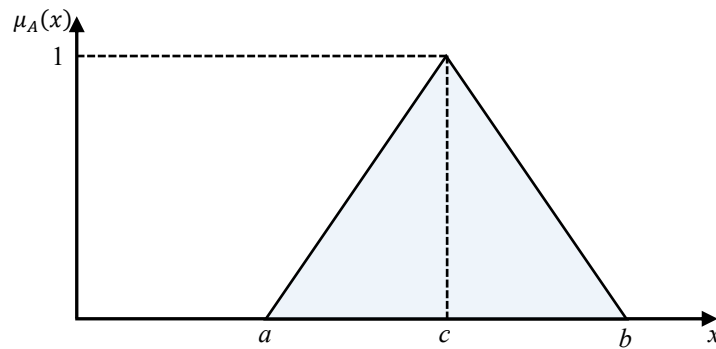


Figure 2.8. Fonction d'appartenance triangulaire

- **La fonction Trapézoïdale**

La fonction trapézoïdale est définie par quatre points, (a, 0) et (b, 0) les points extrêmes, (c, 1) et (d, 1) les point des sommets internes. Elle est exprimée par :

$$\mu_A(x) = \begin{cases} \frac{x-a}{c-a} & \text{si } a \leq x \leq c \\ 1 & \text{si } c \leq x \leq d \\ \frac{x-b}{d-b} & \text{si } d \leq x \leq b \\ 0 & \text{ailleurs} \end{cases} \quad (2.13)$$

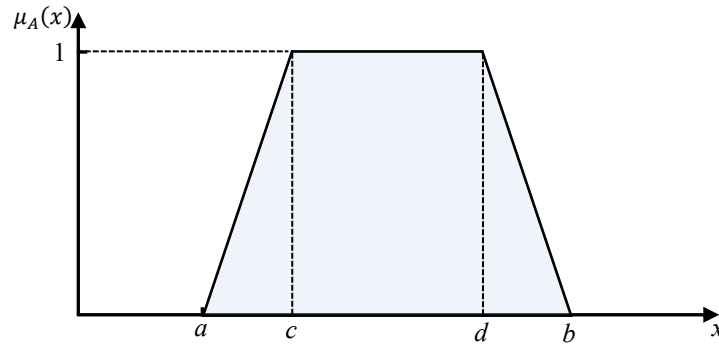


Figure 2.9. Fonction d'appartenance Trapézoïdale

- **La fonction Gaussienne :**

La fonction gaussienne, possède une courbe familière en forme de cloche. Elle a comme expression :

$$\mu_A(x) = e^{-\frac{1}{2}\left(\frac{x-c}{\sigma}\right)^2} \quad (2.14)$$

Cette forme est liée aux distributions de probabilités normales et a des propriétés mathématiques utiles.

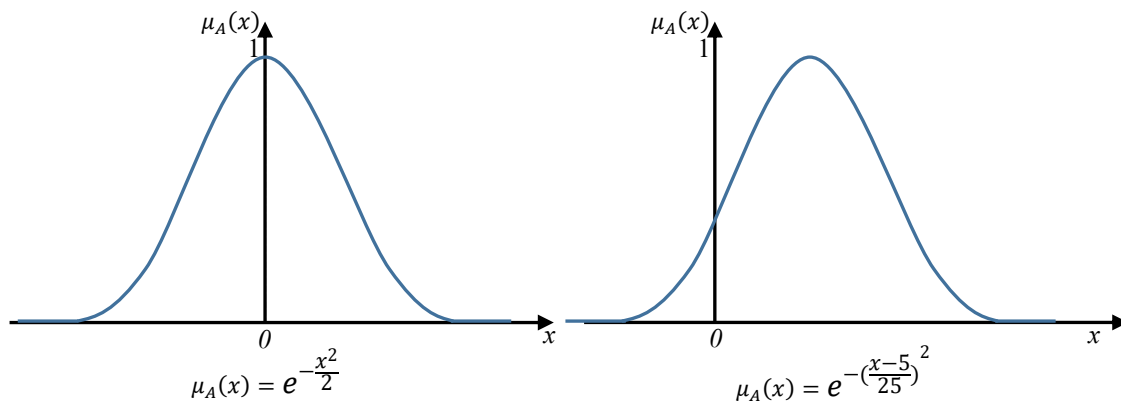


Figure 2.10. Fonction d'appartenance gaussienne

Les paramètres c et σ déterminent la position du centre et la largeur de la forme de la courbe gaussienne. Pour des valeurs $c = 0$ et $\sigma = 1$ nous obtenons la fonction d'appartenance gaussienne standard $e^{-\frac{x^2}{2}}$, centrée à $c = 0$, et avec une surface sous la courbe égale à $\sqrt{2\pi}$. C'est la courbe gaussienne représentée à gauche dans la figure 2.10.

- **La fonction sigmoïde (courbe en S et courbe en Z) :**

La fonction sigmoïde est exprimée par l'expression généralisée suivante :

$$\mu_A(x) = \frac{1}{1 + e^{-\sigma(x-m)}} \quad (2.15)$$

Une fonction sigmoïde peut avoir une courbe en S ou une courbe en Z selon la valeur de σ , ce qui est représenté sur la figure ci-dessous :

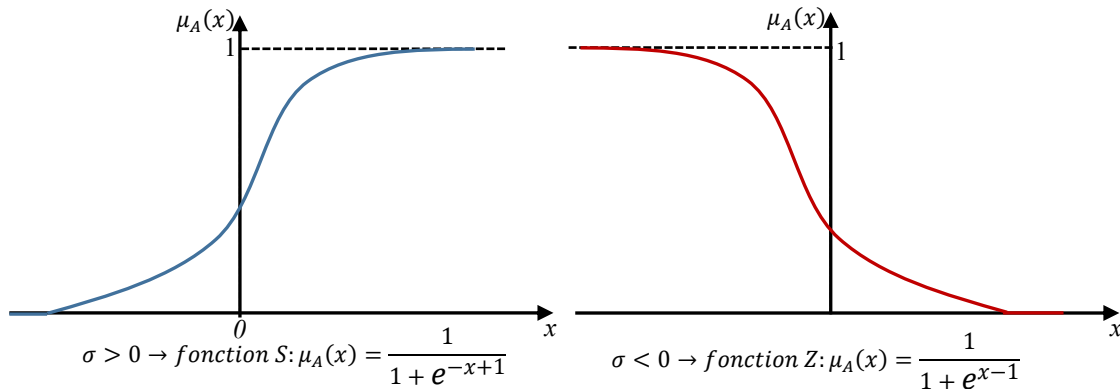


Figure 2.11. Fonction d'appartenance sigmoïde

Les valeurs de σ déterminent les fonctions croissantes ou décroissantes tandis que le paramètre m fait déplacer la fonction vers la droite ou la gauche. Ces mêmes courbes peuvent être obtenues à partir $\frac{1}{2}(1 + \tanh(x)) = \frac{1}{1 + e^{-2x}}$.

- **Singleton :**

Un ensemble flou particulier est l'ensemble singleton ou représentation flou d'un nombre défini par :

$$\mu_A(x) = \begin{cases} 1 & \text{si } x = x_0 \\ 0 & \text{ailleurs} \end{cases} \quad (2.16)$$

A titre d'exemple dans ce cas nous pouvons citer les fonctions d'appartenance de la classe « Le feu est rouge », « Le feu est vert », etc.

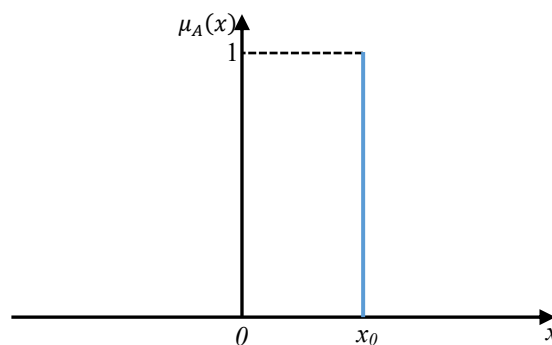


Figure 2.12. Fonction d'appartenance Singleton

❖ **Univers de discours**

La logique floue permet de tenir compte de la nature imprécise du monde réel, grâce à des termes flous ou termes linguistiques (ex : "Petit", "Moyen", "Grand"). Chaque terme représente un sous-

ensemble de valeurs numériques et caractérise ainsi la variable linguistique ("Température", "Taille", "Age", etc.). Le domaine sur lequel ces termes et ces variables sont définies constitue l'univers de discours ou le domaine de définition (référentiel). L'univers de discours d'une variable couvrira donc, l'ensemble des valeurs prises par cette variable.

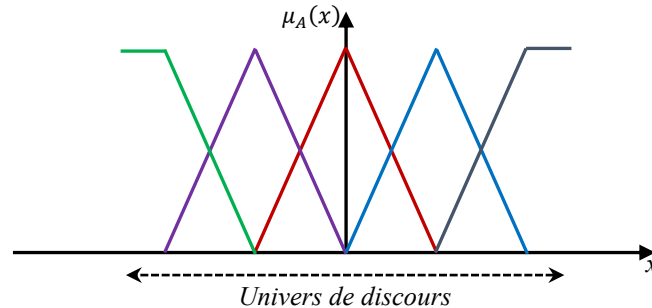


Figure 2.13. Fonction d'appartenance triangulaire

❖ Définition des fonctions d'appartenance pour les variables d'entrées

La fuzzification proprement dite consiste à définir les fonctions d'appartenance, en particulier pour les variables d'entrées.

Elle réalise alors le passage des grandeurs physiques en variables linguistiques (variables floues) qui peuvent être traitées par les inférences.

Les grandeurs physiques X (par exemple l'écart de réglage, dérivé approximative d'une grandeur, etc.) sont réduites à des grandeurs normalisées x qui varient normalement dans un intervalle $[-1, 1]$ (normalisation de l'univers de discours).

Dans un cas concret, cette normalisation n'est pas indispensable, on peut garder les grandeurs physiques (réelles) pour la définition des fonctions d'appartenance.

Le choix du nombre et de la forme des fonctions d'appartenance dépend de la résolution et de l'intervention du réglage désiré.

En général, on introduit pour une variable x trois, cinq ou sept ensembles, représentées par des formes triangulaires ou trapézoïdales (figure 2.14).

Une subdivision plus fine c'est-à-dire plus de sept ensembles, n'apporte en général aucune amélioration au comportement dynamique du réglage par logique floue, par contre elle compliquerait la formulation des règles d'inférences.

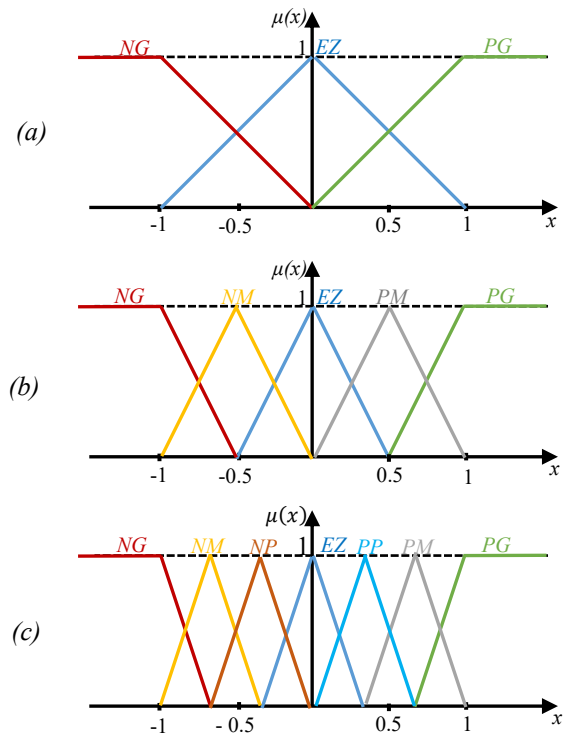


Figure 2.14. Fonction d'appartenance triangulaire

- (a) Fuzzification avec trois ensembles floue
- (b) Fuzzification avec cinq ensembles floue
- (c) Fuzzification avec sept ensembles floue

Les différents ensembles sont caractérisés par des désignations standard, la signification des symboles (abréviation) est indiquée par le tableau 2.3.

Symboles	Signification
NG	Négatif grand
NM	Négatif moyen
NP	Négatif petit
EZ	Environ zéro
PP	Positif petit
PM	Positif moyen
PG	Positif grand

Tableau 2.3. Désignation standard des ensembles flous

❖ Considération générale sur les fonctions d'appartenance

Le découpage d'un univers de discours par les termes flous est appelé une partition floue. Cette dernière est un découpage grossier d'une variable en sous-ensemble flous d'une manière générale, une partition (A_i, μ_{A_i}) est définie par :

Condition i : $(\forall x) (\exists i) (\mu_{A_i}(x) > 0)$

Condition ii : $(\exists i_0)(\exists x_0) (\mu_{A_{i_0}}(x_0) = 1) \Rightarrow (\forall j)(j \neq i_0) (\mu_{A_j}(x_0) < 1)$

La condition i traduit un recouvrement total du domaine, tandis que la condition ii indique qu'un élément ne peut appartenir à deux sous-ensembles flous avec le degré 1.

Une partition floue forte est définie par la condition : $(\forall x) (\sum_i \mu_{A_i}(x) = 1)$.

Une caractéristique supplémentaire intervenant dans la définition des sous-ensembles est le taux de recouvrement de ces sous-ensembles. Il est défini par : $T_r(x) = \sum_{i=1,n} \mu_{A_i}(x)$.

Ce taux de recouvrement influe d'une part sur la procédure de fuzzification mais surtout sur la sortie réelle du système flou. Bien qu'il soit souvent défini égal à 1 dans le cas de fonctions d'appartenance triangulaires, on peut vouloir renforcer l'influence de l'appartenance à un sous ensemble par rapport à un autre et donc avoir un taux de recouvrement différent de 1.

Il n'existe pas de règles précises pour la définition des fonctions d'appartenance, néanmoins on peut donner des directives générales, afin de faciliter un premier choix.

Il est préférable de choisir, pour première implémentation, des fonctions d'appartenance de forme symétrique et distribué de manière équidistante (figure 2.15(a)).

Pour obtenir un comportement optimal du réglage on peut adopter une distribution non équidistante (figure 2.15(b)), ou des formes non symétriques et non équidistantes (figure 2.15(c)).

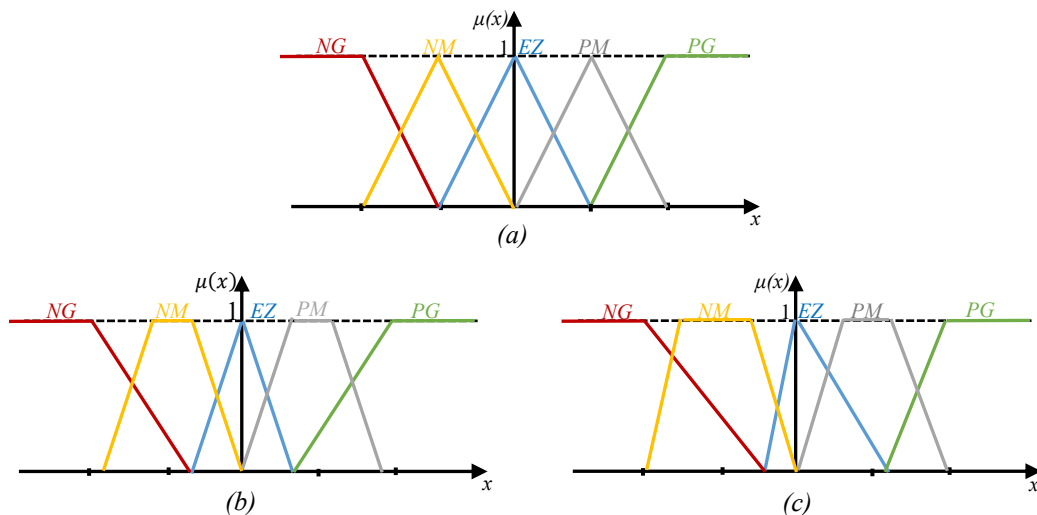


Figure 2.15. Différentes formes pour les fonctions d'appartenance

(a) Fonctions d'appartenance symétrique et équidistante

(b) Fonctions d'appartenance symétrique et non équidistante

(c) Fonctions d'appartenance non symétrique et non équidistante

Il faut éviter des lacunes dues à un chevauchement insuffisant entre les fonctions d'appartenance de deux ensembles voisins, (figure 2.16(a)), ce qui provoquerait des zones de non-intervention du régulateur (zones mortes) entraînant une instabilité du réglage, ou à un chevauchement trop important, surtout avec $\mu=1$, entre deux ensembles voisins (figure 2.16(b)). Ce qui conduit à un aplatissement de la caractéristique du régulateur.

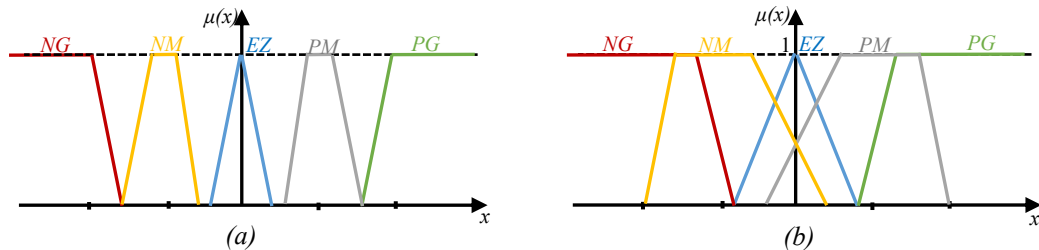


Figure 2.16. Différentes formes à éviter pour les fonctions d'appartenance

(a) Formes avec chevauchement insuffisant

(b) Formes avec chevauchement trop important

2.6.4. Base de connaissances

La partie base de connaissances comporte une connaissance dans le domaine d'application et le résultat de commande prévu, permettant ainsi une description du comportement du système à commander. Cette partie est composée d'une "*base de données*" et d'une "*base de règles linguistiques (floues) de commande*".

❖ La base de données

La base de données effectue des définitions qui sont nécessaires pour établir les règles de commande et manipuler les données floues dans un RLF. Ceci se traduit par :

- Une normalisation des univers de discours
- Une partition floue des espaces entrées-sorties.
- Un choix des fonctions d'appartenances
- Choix des opérateurs, etc.

❖ La base de règles

La base des règles floues, contient les règles floues décrivant le comportement du système ; elle est le cœur du système entier dans le sens où tous les autres composants sont utilisés pour interpréter et combiner ces règles pour former la stratégie de commande finale.

Une règle floue est une déclaration du type « Si...Alors... »:

Si x est A Alors y est B

Où x et y sont des variables linguistiques, et A et B sont des valeurs linguistiques, déterminées par les ensembles flous sur les ensembles X et Y .

Une base des règles floues R est une collection de règles floues de la forme "Si-Alors".

$R = [R_1, \dots, R_M]$ telle que :

$$R_l: \textbf{Si } x_1 \text{ est } A_{l1} \text{ et } \dots \text{ et } x_n \text{ est } A_{ln} \textbf{ Alors } y_l \text{ est } B_l$$

Où : $[x_1, \dots, x_n]$ représentant les entrées du système flou. Et y_l la sortie du système.

Exemple : le cas d'un freinage automatique des véhicules

Si Vitesse est petite et Distance de l'obstacle est faible **Alors** Freinage est faible

Ce type de règles dont la conclusion est un ensemble flou est dit de type « **Mamdani** ».

Une autre forme de règles dite de « **Sugeno** » existe. Dans cette forme la conclusion de la règle est nette. La sortie est donc calculée comme une fonction linéaire des entrées :

$$R_l: \textbf{Si } x_1 \text{ est } A_{l1} \text{ et } \dots \text{ et } x_n \text{ est } A_{ln} \textbf{ Alors } y_l = f_l(x)$$

Avec $f_l(x)$ est un polynôme.

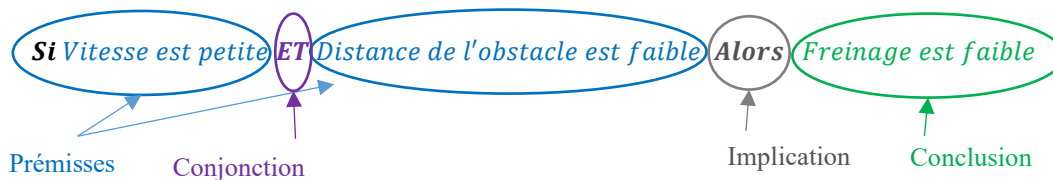
2.6.5. Mécanisme d'inférence

Le mécanisme d'inférence utilise la base des règles floues pour effectuer une transformation à partir des ensembles flous dans l'espace d'entrée vers les ensembles flous dans l'espace de sortie en se basant sur les opérations de la logique floue. En d'autres termes il exprime la relation qui lie les variables d'entrées (exprimées en variables linguistiques) à la variable de sortie (exprimée également en variable linguistique).

Il s'agit souvent d'inférence avec plusieurs règles. Ces règles doivent tenir compte du comportement du système à régler, ainsi que des buts du réglage envisagé. Elles s'expriment sous la forme :

Si conditions alors conclusion

La partie conditions (antécédent ou prémisse) est constituée d'une formule logique qui doit être vérifiée pour que la règle s'applique. La partie conclusion (conséquence) peut correspondre à l'ajout ou au retrait d'un fait ou d'une hypothèse, au déclenchement d'une action, etc.



Dans le jeu de règles du système flou interviennent les opérateurs flous "ET (AND)" et "OU (OR)". L'opérateur "ET" s'applique aux variables à l'intérieur d'une règle, tandis que l'opérateur "OU" lie les différentes règles. La solution finale ou globale est formée alors par l'agrégation des solutions (locales : inférence) de chaque règle

L'expérience et les connaissances professionnelles jouent un rôle important pour la détermination des règles.

❖ Description des inférences de réglage

Pour la présentation des différentes possibilités d'exprimer les inférences, on choisit un exemple de système à régler avec deux variables linguistiques x_1 et x_2 qui forment les variables d'entrées de l'inférence, et une variable de sortie x_r exprimée elle aussi comme variable linguistique.

Les règles d'inférence peuvent être exprimées de différentes manières :

- **Description linguistique**

On sait que pour le réglage par logique floue, il s'agit souvent d'inférence avec plusieurs règles. Chaque règle est de forme « **Si condition Alors action** (conclusion) ».

La description linguistique des inférences peut être écrite comme suit :

Si (x_1 est négatif grand) **ET** (x_2 est environ zéro) **Alors** (x_r est positif grand), **OU**

Si (x_1 est négatif moyen) **ET** (x_2 est positif moyen) **Alors** (x_r est environ zéro), **OU**

La condition d'une règle peut aussi contenir des opérateurs **OU** et **ET**, et les règles sont déterminées selon la stratégie de réglage adoptée.

- **Description symbolique**

C'est une simplification de la forme linguistique. Les ensembles flous peuvent être désignés de manière abrégée par leurs symboles, et la description symbolique des inférences s'écrira comme suit :

Si (x_1 est NG) **ET** (x_2 est EZ) **Alors** (x_r est PG), **OU**

Si (x_1 est NM) **ET** (x_2 est PM) **Alors** (x_r est EZ), **OU**

La description symbolique est plus claire et plus compact, surtout dans le cas où il y a plusieurs variables $x_1, x_2, \dots, x_n$.

- **Description par matrice d'inférence**

Une représentation plus simplifiée est réalisée par une matrice appelée matrice d'inférence ou table des règles (tableau 2.4).

Le tableau (tableau 2.4) représente une table d'inférence avec cinq sous-ensembles flous pour deux variables d'entrée x_1 et x_2 , et une variable de sortie x_r .

$x_1 \backslash x_2$	NG	NM	EZ	PM	PG
NG			PG	PM	
NM			PM	EZ	NM
EZ	PG	PM	EZ	NM	NG
PM	PM	EZ	NM		
PG		NM	NG		

Tableau 2.4. Matrice d'inférences

A l'intersection d'une colonne et d'une ligne se trouve l'ensemble correspondant à la variable de sortie x_r , défini par une règle d'inférence, dont les variables d'entrée sont liées par l'opérateur **ET**.

Cette représentation impose donc une restriction au cas le plus fréquent, ou la condition de toutes les règles ne contient que l'opérateur **ET**.

On peut toujours aboutir à cette forme, par une formulation adéquate des règles d'inférences. Les différentes variables de sortie sont combinées par l'opérateur OU.

Cette représentation facilite l'établissement des inférences, et permet d'éviter le cas d'inférences contradictoires.

Cependant, cette description devient complexe lorsqu'il y a plus de trois ou quatre variables, et qui sont subdivisées en un nombre élevé d'ensembles.

Si toutes les cases de la matrice sont remplies, on parle alors de règles d'inférences complètes. Dans le cas contraire on parle de règles d'inférence incomplètes.

- **Description par tableau d'inférence**

Cette description se prête bien et surtout à des systèmes multi-variables, avec plusieurs variables de sorties $x_{r1}, \dots, x_{rn}$.

Règle n°	x_1	x_2	x_3	x_{r1}	x_{r2}
1	NG	EZ	NG	PG	NG
2	NG	EZ	EZ	PG	NG
3	EZ	EZ	PG	EZ	NG
4	EZ	NG	NG	PG	NG
5	EZ	NG	EZ	PG	NG
6	EZ	NG	PG	EZ	NG
7	EZ	EZ	NG	NG	PG
8	PG	EZ	EZ	NG	PG
9	PG	EZ	PG	NG	PG

Tableau 2.5. Tableau d'inférences

❖ **Justification des règles de contrôle flou**

D'une manière générale, l'écriture des règles d'un contrôleur flou fait appel à l'expertise et l'expérience des opérateurs humains. Il existe plusieurs approches pour la détermination des règles d'un RLF. Une connaissance qualitative du comportement du processus conduit à l'utilisation d'une méthode heuristique où la détermination des règles se fait de telle sorte que l'écart entre la consigne et la sortie puisse être corrigée.

Une autre approche plus intéressante a été introduite pour la détermination des règles et cela en se référant à la trajectoire du système en boucle fermée. En asservissement et régulation, on utilise généralement l'erreur e et sa dérivée Δe pour décrire la dynamique du processus. A partir de ces mesures, traduites sous la forme de variables floues, il est possible de déterminer les règles, dans le domaine temporel.

En considérant la réponse à un échelon d'un processus à piloter en fonction des objectifs que l'on se sera fixé en boucle fermée, une analyse temporelle conduit à l'écriture du jeu de règle du RLF permettant de décrire chaque type de comportement du processus (figure 2.17).

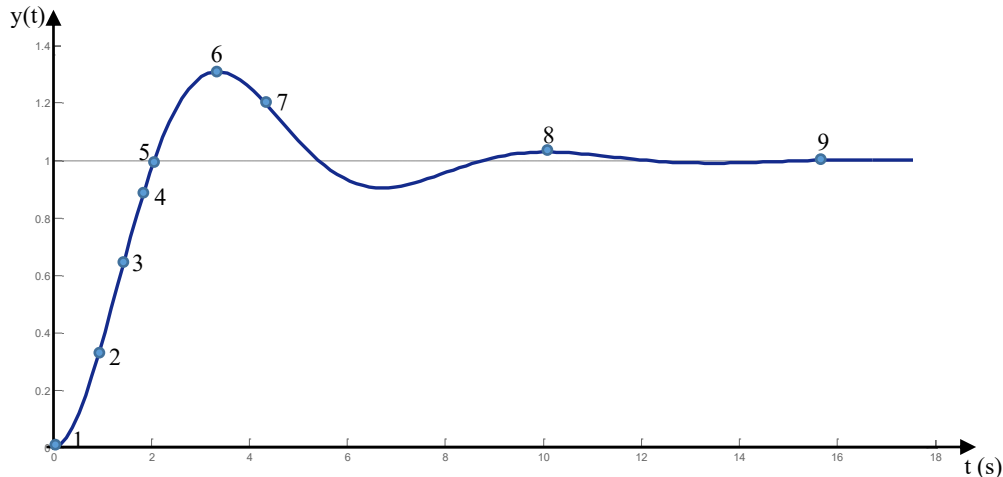


Figure 2.17. Ecriture du jeu de règles grâce à une analyse temporelle

Pour expliquer la procédure à suivre, on considère les neuf points indiqués sur la réponse à un échelon (figure 2.17) et, pour chacun de ces points, on explicite l'expertise sous la forme suivante :

En posant :

$$e = r - y$$

Avec : r est la référence, y la sortie, Δe est la dérivée de l'erreur et u signal de commande floue (avant défuzzification).

On a :

- 1- **Si** (e est PG) **ET** (Δe est EZ) **Alors** (u est PG) (départ)
- 2- **Si** (e est PG) **ET** (Δe est NP) **Alors** (u est PM) (augmentation de la commande pour gagner l'équilibre)
- 3- **Si** (e est PM) **ET** (Δe est NP) **Alors** (u est PP) (très faible augmentation pour éviter le dépassement)
- 4- **Si** (e est PP) **ET** (Δe est NP) **Alors** (u est EZ) (convergence vers l'équilibre correct)
- 5- **Si** (e est EZ) **ET** (Δe est NP) **Alors** (u est NP) (freinage du processus)
- 6- **Si** (e est NM) **ET** (Δe est EZ) **Alors** (u est NM) (rappel du processus vers l'équilibre)
- 7- **Si** (e est NP) **ET** (Δe est PP) **Alors** (u est EZ) (freinage et inversion de la variation de la commande)

8- **Si** (*e est NP*)**ET** (*Δe est EZ*) **Alors** (*u est NP*) (*convergence vers l'équilibre correcte*)

9- **Si** (*e est EZ*)**ET** (*Δe est EZ*) **Alors** (*u est EZ*) (*équilibre*)

En décrivant point par point le comportement du processus et l'action de variation de commande à appliquer, on en déduit la table suivante (table de contrôle flou de base) qui correspond à la table de règles très connues de Mac Vicar-Whelan.

$\Delta e \backslash e$		NG	NM	NP	EZ	PP	PM	PG
NG	NG	NG	NG	NG	NG	NM	NP	EZ
NM	NG	NG	NG	NM	NP	EZ	PP	
NP	NG	NG	NM	NP	EZ	PP	PM	
EZ	NG	NM	NP	EZ	PP	PM	PG	
PP	NM	NP	EZ	PP	PM	PG	PG	
PM	NP	EZ	PP	PM	PG	PG	PG	
PG	EZ	PP	PM	PG	PG	PG	PG	

Tableau 2.6. Table de contrôle flou de base de Mac Vicar-Whelan

❖ Traitement numérique des inférences

Dans les inférences par logique floue interviennent généralement les opérateurs ET et/ou OU qui s'appliquent aux variables à l'intérieur d'une règle et liant les différentes règles.

A cause de l'empiètement des fonctions d'appartenance, en général deux ou plusieurs règles sont activées en même temps, une règle d'inférence est activée lorsque le facteur d'appartenance lié à la condition de cette règle est non nul.

Il existe plusieurs possibilités pour réaliser ces opérateurs qui s'appliquent aux fonctions d'appartenance. On introduit alors la notion de méthode d'inférence. Elle détermine la réalisation de différents opérateurs dans une inférence, permettant un traitement numérique de cette dernière. Généralement, on utilise une des méthodes suivantes :

- Méthode d'inférence MAX-MIN (Mamdani).
- Méthode d'inférence MAX-PROD (Larsen).
- Méthode d'inférence SOM-PROD (Zadeh).

Afin de mettre en évidence le traitement numérique, on fera appel à deux variables d'entrée et une variable de sortie. Chacune est décomposée en trois ensembles NG, EZ et PG et définie par des fonctions d'appartenance (figure 2.18). Pour les variables d'entrée, on suppose que les valeurs numériques sont $x_1 = 0.44$ et $x_2 = -0.67$.

L'inférence est décomposée de deux règles :

Si (x_1 est PG) ET (x_2 est EZ) Alors (x_r est EZ), OU

Si (x_1 est EZ) OU (x_2 est NG) Alors (x_r est NG).

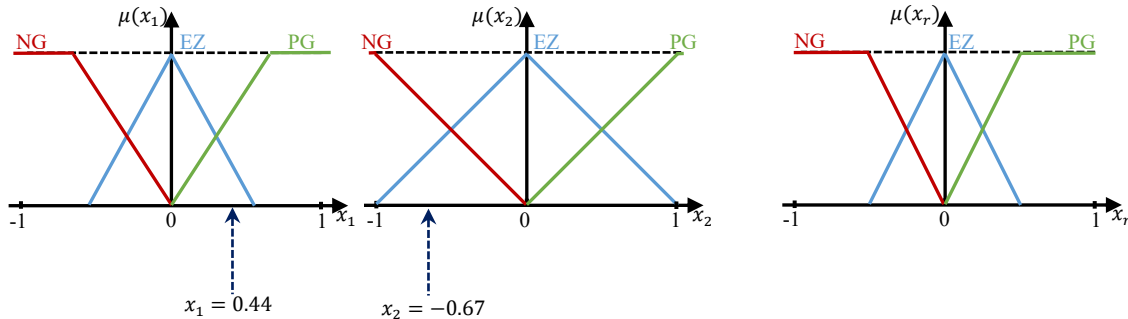


Figure 2.18. Fonctions d'appartenance pour l'exemple servant à la description des méthodes d'inférences

➤ Méthode d'inférence MAX-MIN

Au niveau de la condition dans la méthode d'inférence MAX-MIN l'opérateur « OU » est réalisé par la formation du maximum et l'opérateur « ET » par la formation du minimum. La conclusion dans chaque règle, introduite par « ALORS », qui lie le facteur d'appartenance de la condition avec la fonction d'appartenance de la variable de sortie x_r , est réalisée par la formation de minimum. Enfin, l'opérateur « OU » qui lie les différentes règles est réalisé par la formation du maximum.

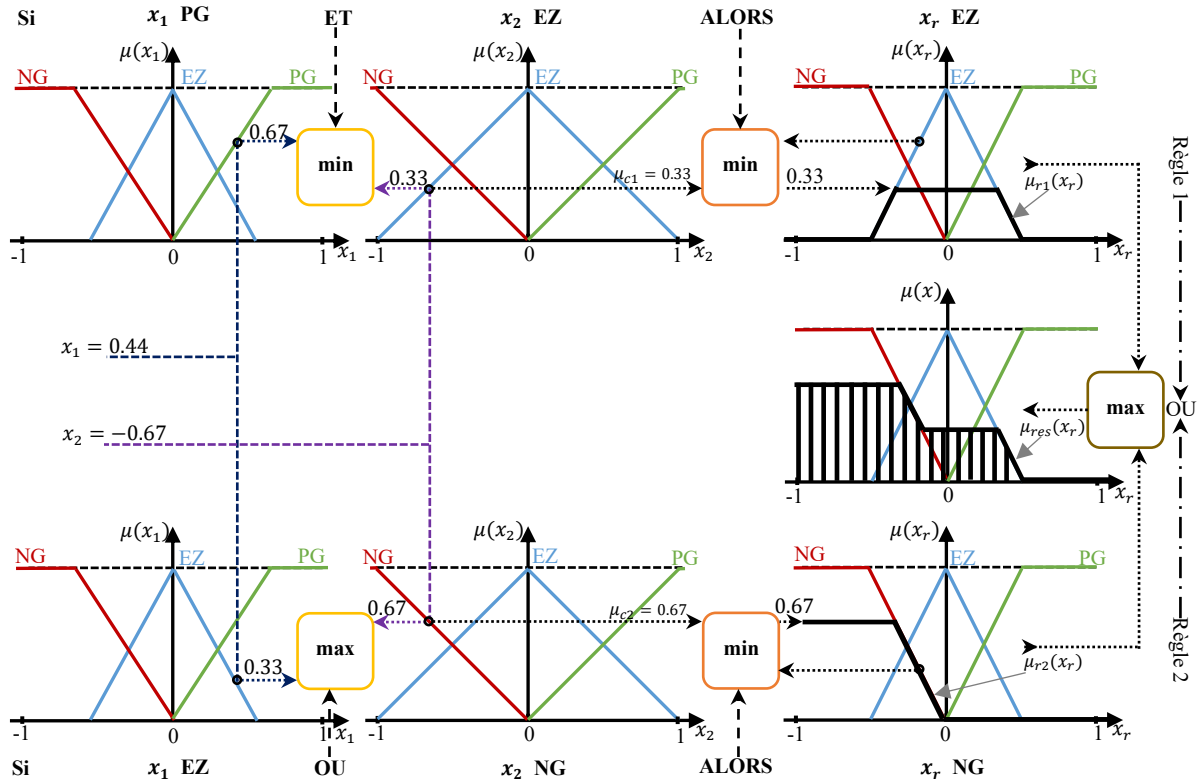


Figure 2.19. Méthode d'inférence MAX-MIN pour deux variables d'entrée et deux règles

La condition (x_1 PG ET x_2 EZ) de la première règle par exemple donne pour $x_1 = 0.44$ et $x_2 = -0.67$ les facteurs d'appartenance $\mu_{PG}(x_1 = 0.44) = 0.67$ et $\mu_{EZ}(x_2 = -0.67) = 0.33$, ce qui implique que la condition prend le facteur d'appartenance $\mu_{c1} = 0.33$ (minimum des deux valeurs à cause de l'opérateur ET).

La fonction d'appartenance $\mu_{EZ}(x_r)$ pour la variable de sortie est donc décrétée à 0.33 (à cause de la formation du minimum lié à ALORS). La fonction d'appartenance partielle $\mu_{r1}(x_r)$ ainsi obtenue pour x_r est mise en évidence par un trait renforcé (figure 2.19).

La deuxième règle implique par sa condition (x_1 EZ OU x_2 NG) les facteurs d'appartenance $\mu_{EZ}(x_1 = 0.44) = 0.33$ et $\mu_{NG}(x_2 = -0.67) = 0.67$. Ainsi, la condition possède le facteur d'appartenance $\mu_{c2} = 0.67$ (maximum des deux valeurs à cause de l'opérateur OU).

La fonction d'appartenance $\mu_{NG}(x_r)$ est donc décrétée à 0.67 (à cause de la formation du minimum lié à ALORS). La fonction d'appartenance partielle $\mu_{r2}(x_r)$ est également mise en évidence par un trait renforcé (figure 2.19).

Puisque chacune des règles exige une intervention, il faut encore déterminer la fonction d'appartenance résultante et $\mu_{res}(x_r)$. Elle s'obtient par la formation du maximum des deux fonctions d'appartenance partielles, étant donné que les deux règles liées par l'opérateur OU. Cette fonction d'appartenance résultante (hachurée) est obtenue en appliquant le maximum.

➤ Méthode MAX-PROD

Cette méthode réalise au niveau de la condition, l'opérateur OU par la formation du maximum et l'opérateur ET par la formation du minimum. En revanche, la conclusion dans chaque règle, introduite par ALORS, qui lie le facteur d'appartenance de la condition avec la fonction d'appartenance de la variable de sortie x_r , est réalisée par la formation du produit. L'opérateur OU qui lie les différentes règles est réalisée par la formation du maximum.

Donc l'opérateur OU liant les deux règles est réalisée par la formation du maximum et ALORS est réalisé par la formation du produit.

Ainsi, la première condition de la première règle prend le facteur d'appartenance $\mu_{c1} = 0.33$. La fonction d'appartenance $\mu_{EZ}(x_r)$ pour la variable de sortie est multipliée par ce facteur (à cause du produit lié à ALORS). Cette fonction d'appartenance est mise en évidence par un trait renforcé (figure 2.20.).

La deuxième condition de la deuxième règle donne le facteur d'appartenance $\mu_{c2} = 0.67$. La fonction d'appartenance partielle est alors obtenue en multipliant μ_{c2} par la fonction d'appartenance $\mu_{NG}(x_r)$ cette fonction d'appartenance est mise en évidence par un trait renforcé dans la figure (2.20.).

La fonction d'appartenance résultante $\mu_{res}(x_r)$ s'obtient par la formation du maximum des deux fonctions d'appartenances partielles réalisant l'opérateur OU.

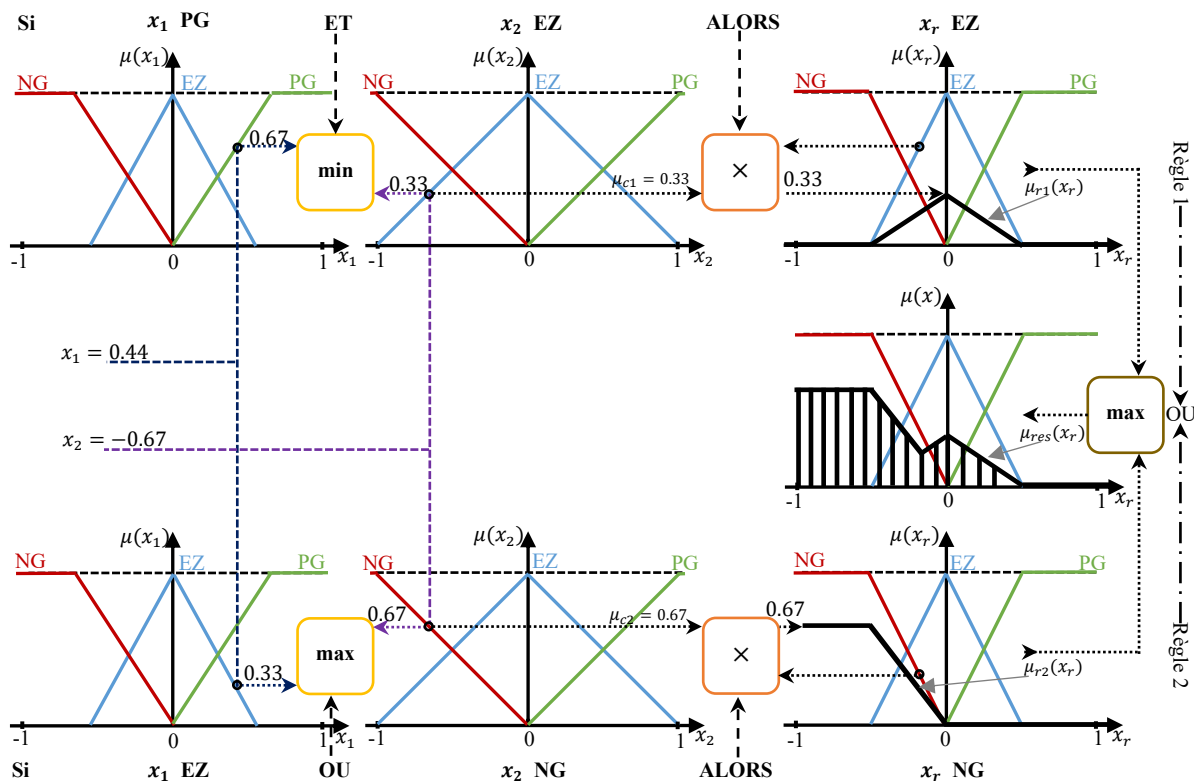


Figure 2.20. Méthode d'inférence MAX-PROD pour deux variables d'entrées et deux règles

➤ Méthode d'inférence SOM-PROD

Cette méthode réalise au niveau de la condition, l'opération OU par la formation de la somme (valeur moyenne), tandis que l'opérateur ET est réalisé par la formation du produit. La conclusion, précédée par ALORS, liant le facteur d'appartenance de la condition avec la fonction d'appartenance de la variable de sortie par l'opérateur ET, est réalisé par la formation du produit. L'opérateur OU qui lie les différentes règles est réalisé par la formation de la somme (moyenne). Avec les facteurs d'appartenance $\mu_{PG}(x_1 = 0.44) = 0.67$ et $\mu_{EZ}(x_2 = -0.67) = 0.33$, la condition de la première règle prend le facteur d'appartenance $\mu_{c1} = 0.22$ (produit des deux valeurs).

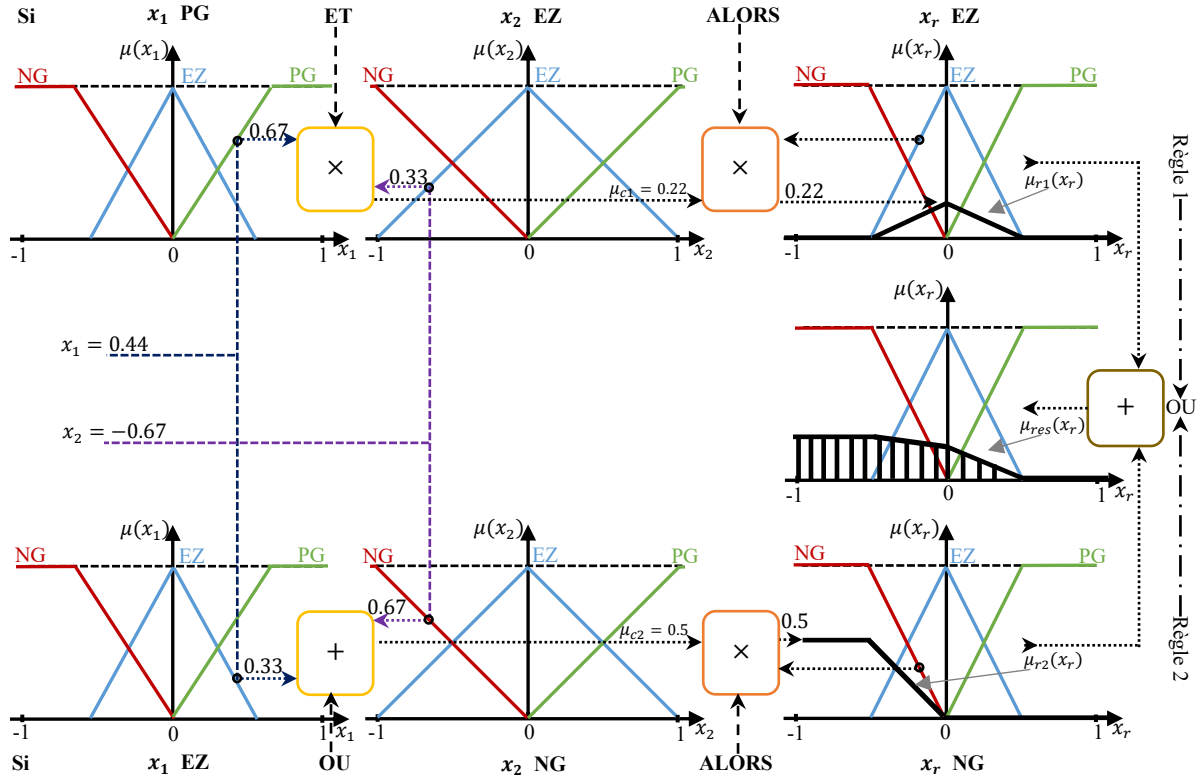


Figure 2.21. Méthode d'inférence MAX-PROD pour deux variables d'entrées et deux règles

La fonction d'appartenance $\mu_{EZ}(x_r)$ pour la variable de sortie est multipliée par ce facteur (car ALORS est réalisée par la formation du produit). On obtient ainsi la fonction d'appartenance partielle $\mu_{r1}(x_r)$.

La condition de la deuxième règle donne le facteur d'appartenance $\mu_{c2} = 0.5$ à cause de la formation de la somme des deux facteurs d'appartenance $\mu_{EZ}(x_1 = 0.44) = 0.33$ et $\mu_{NG}(x_2 = -0.67) = 0.67$ pour l'opérateur OU. Ainsi, la fonction d'appartenance $\mu_{NG}(x_r)$ est multipliée par le facteur $\mu_{c2} = 0.5$. Il en résulte ainsi la fonction d'appartenance partielle $\mu_{r2}(x_r)$.

La fonction d'appartenance résultante $\mu_{res}(x_r)$, s'obtient par la formation de la somme des deux fonctions d'appartenance partielles. Cette fonction d'appartenance résultante est hachurée (figure 2.21).

2.6.6. Défuzzification

Les méthodes d'inférence fournissent une fonction d'appartenance résultante $\mu_{res}(x_r)$ pour la variable de sortie x_r .

Il s'agit donc d'une information floue. Mais l'organe de commande doit être attaqué avec une valeur bien précise pour le signal de commande u_{cm} , il faut donc transformer la valeur floue en une valeur bien déterminée. Cette transformation est appelée défuzzification.

Il existe plusieurs méthodes de défuzzification ; les plus utilisées sont :

- Méthode par centre de gravité (centroïde)
- Méthode par valeur maximale
- Méthode par valeur moyenne des maxima.

➤ Défuzzification par la méthode du centre de gravité (COG)

La méthode de défuzzification la plus utilisée est celle de la détermination du centre de gravité de la fonction d'appartenance résultante $\mu_{res}(x_r)$.

L'abscisse du centre de gravité de la fonction d'appartenance résultante $\mu_{res}(x_r)$ est donnée par la relation générale suivante :

$$x_r^* = \frac{\int_U x_r \mu_{res}(x_r) dx_r}{\int_U \mu_{res}(x_r) dx_r} \quad (2.17)$$

Avec U est l'Univers du discours pour que toutes les valeurs de sorties soient considérées.

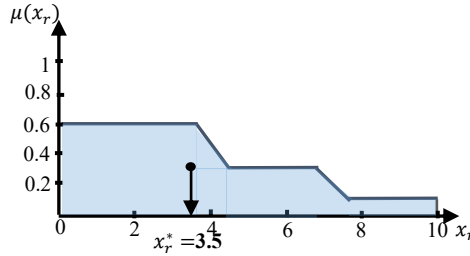


Figure 2.22. Exemple de défuzzification par la méthode du centre de gravité

Dans le cas où la fonction $\mu(x_r)$ est discrétisée, le centre de gravité est donné par :

$$x_r^* = \frac{\sum_i^k x_{ri} \mu_i(x_{ri})}{\sum_i^k \mu_i(x_{ri})} \quad (2.18)$$

Où k est le nombre des niveaux de quantisation, x_{ri} la valeur de sortie pour le niveau i et μ_i sa valeur d'appartenance.

Cette méthode de défuzzification est la plus couteuse en temps de calcul, surtout pour l'exécution en temps réel.

➤ **Défuzzification par la méthode de la valeur maximale**

Cette méthode de défuzzification est plus simple, elle consiste à prendre comme valeur de sortie x_r^* , l'abscisse de la valeur maximale de la fonction d'appartenance résultante $\mu_{res}(x_r)$ (figure 2.23).

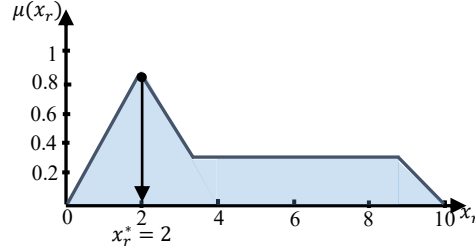


Figure 2.23. Exemple de défuzzification par la méthode de la valeur maximale

Néanmoins cette méthode n'est pas intéressante pour le réglage lorsqu'il y a plusieurs maximums ou si $\mu_{res}(x_r)$ est écrêtée (figure 2.24).

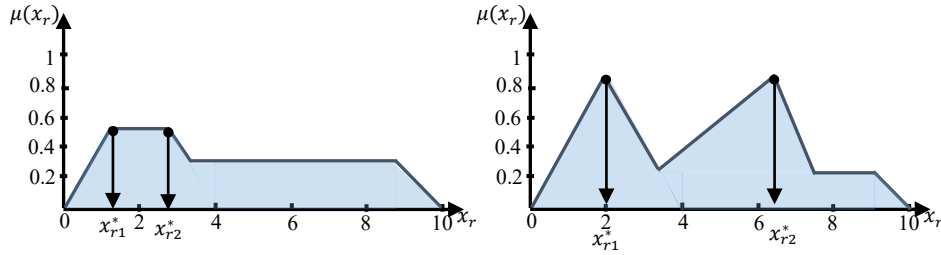


Figure 2.24. Exemple de cas avec plusieurs maximums

➤ **Défuzzification par la méthode moyenne des maxima (MM)**

Pour éviter l'indétermination présentée lors de la méthode par valeur maximale, on fait appel la méthode de défuzzification par valeur moyenne des maxima. La défuzzification par la méthode moyenne des maxima définit la sortie x_r^* comme étant la moyenne des abscisses des maxima de l'ensemble flou issu de l'agrégation des conclusions.

$$x_r^* = \frac{\int_S x_r dx_r}{\int_S dx_r} \quad (2.19)$$

Avec $S = \{x_{r0} \in U / \mu(x_{r0}) = \sup_{x_r \in U} (\mu(x_r))\}$

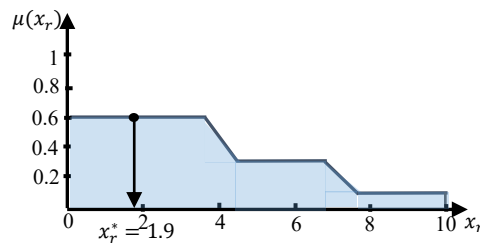


Figure 2.25. Exemple de défuzzification par la méthode moyenne des maxima

Cependant, cette méthode présente également un grand inconvénient qui réside dans le fait le signal de sortie saute si la dominance change d'une fonction d'appartenance partielle à une autre. Ce comportement discontinu du signal de commande conduit à un mauvais comportement de circuit de réglage, mais cette méthode de défuzzification reste valable.

N.B : Afin d'obtenir le signal de commande u_{cm} , le signal de sortie x_r^* fournit par la défuzzification est soumis à un traitement.

Dans la plus part des cas il y a proportionnalité entre x_r^* et u_{cm} lorsque l'organe de commande fonctionne comme amplificateur en puissance si l'on exprime le signal de commande en grandeurs relatives, rapportées à u_{cm} on a :

$$u_{cm} = x_r^*$$

2.6.7. Différents types de contrôleurs flous

Dans la littérature, il existe plusieurs types de contrôleur flou qui peuvent être appliqués dans les applications de la commande. Les deux types qui sont largement utilisées en pratique sont celui de Mamdani et celui de Takagi-Sugeno. Ces deux types de contrôleur varient quelque peu dans la façon dont les résultats sont déterminés.

❖ Contrôleur flou de type Mamdani

La technique de commande par logique floue avait été présentée pour la première fois en 1975 par Mamdani sur la base des travaux de Zadeh. Cette technique consiste à déterminer un ensemble de règles permettant de maîtriser le comportement dynamique du système à contrôler. L'obtention de ces règles est facile auprès des experts qui connaissent bien le système. Le contrôleur de type Mamdani est caractérisé par des règles à prémisses et conclusions symboliques, l'inférence (Max-Min), et la défuzzification par centre de gravité.

Ce type de contrôleur est basé sur un ensemble fini de règles "Si-Alors" de la forme :

Si x_1 est A_{l1} et ... et x_n est A_{ln} **Alors** y est B_l

Avec $x_1, \dots, x_n$ sont les variables d'entrée du contrôleur et y est la variable de sortie

La conséquence de ce type de système est une valeur floue. Une structure de ce type de contrôleur est représentée sur la figure 2.26.

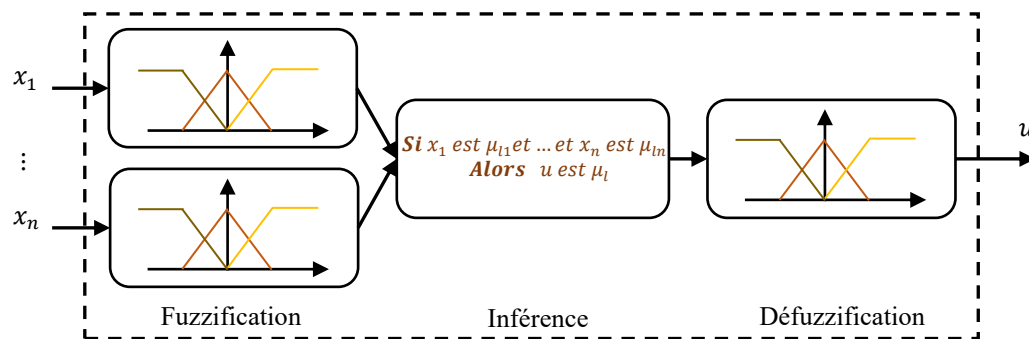


Figure 2.26. Contrôleur flou de type Mamdani

❖ Contrôleur flou de type Takagi-Sugeno

Appelé aussi contrôleur de Takagi-Sugeno-Kang (TS ou TSK). Ce contrôleur diffère de celui de Mamdani au niveau de la partie défuzzification (figure 2.27).

Ce type de contrôleur est, comme celui de Mamdani, construit à partir d'une base de règles "Si...Alors...", les valeurs d'entrées dans les règles sont décrites par des ensembles flous. Cependant, en utilisant un contrôleur TS, la partie conclusion d'une règle ne consiste plus en un ensemble flou, mais détermine une fonction linéaire avec les variables d'entrée comme arguments. L'idée de base est que la fonction correspondante est une bonne fonction de contrôle local pour la région floue qui est décrite par la partie prémisse de la règle. La sortie non floue de ce raisonnement flou est donnée par l'agrégation des valeurs floues résultantes des règles utilisées par les entrées.

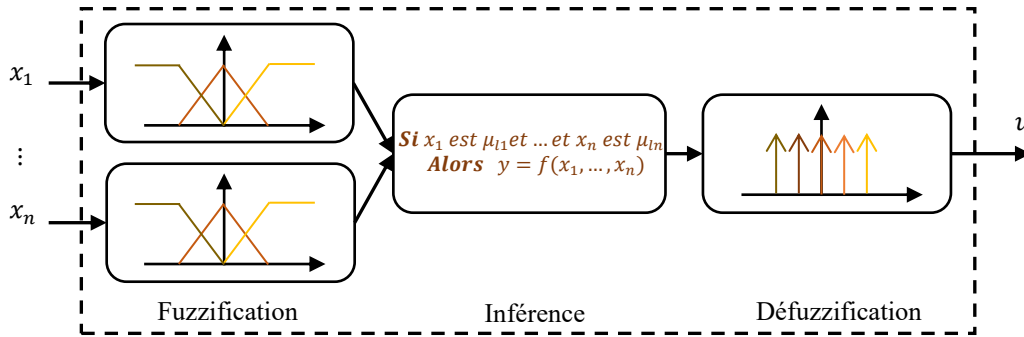


Figure 2.27. Contrôleur flou de type Takagi-Sugeno

Soit $\mu_A(x)$ la fonction d'appartenance à un sous-ensemble flou A avec $x \in U$ univers du discours. La règle floue R est de la forme :

$$R: \text{Si } x_1 \text{ est } A_1 \text{ et } \dots \text{ et } x_n \text{ est } A_n \text{ Alors } y = f(x_1, \dots, x_n)$$

Où $x_1, \dots, x_n$ sont les variables d'entrée, y la variable de sortie et f une fonction polynomiale en fonction des variables d'entrées

Dans le cas du contrôleur de Takagi-Sugeno la règle R_i s'écrit :

$$R_i: \text{Si } x_1 \text{ est } A_{i1} \text{ et } \dots \text{ et } x_n \text{ est } A_{in} \text{ Alors } y_i = a_0 + a_1 x_1 + \dots + a_n x_n$$

Ensuite la valeur réelle finale de la sortie y inférée depuis k règles est calculée comme la moyenne de toutes les sorties de chaque règle y_i par la relation suivante :

$$y = \frac{\sum_{i=1}^k W_i y_i}{\sum_{i=1}^k W_i} \quad (2.20)$$

$$\text{Avec : } W_i = \prod_{j=1}^n \mu_{ij}(x_j)$$

Où $\mu_{ij}(x)$ est la fonction d'appartenance du sous-ensemble flou A_{ij} .

L'approximation linéaire faite par les règles prises séparément, avec le raisonnement flou, ces équations linéaires liant entrées et sorties permettent de passer de façon continue de l'une des équations à l'autre. Ceci est aussi un moyen de réduire le nombre de règles.

2.6.8. Construction des différentes classes des contrôleurs flous

Les contrôleurs flous peuvent être construits de différentes façons, mais il est possible d'établir une classification entre les contrôleurs conçus en se basant sur un modèle et ceux sans modèle. Les contrôleurs basés sur des modèles exigent habituellement une description « complète » de la dynamique du système à commandé. Les stratégies sans modèle sont appelées ainsi parce qu'elles ne sont pas basées sur des modèles mathématiques complets ; cependant, elles ne sont pas complètement sans modèle. Ils sont basés sur des informations extraites d'expériences simples ou d'un modèle heuristique présent dans l'esprit du concepteur.

Cette partie présente un aperçu des méthodes « classiques » pour construire des contrôleurs flous.

Selon les informations utilisées pour construire des contrôleurs flous, ils peuvent être classés comme des systèmes de contrôle sans modèle et basés sur un modèle.

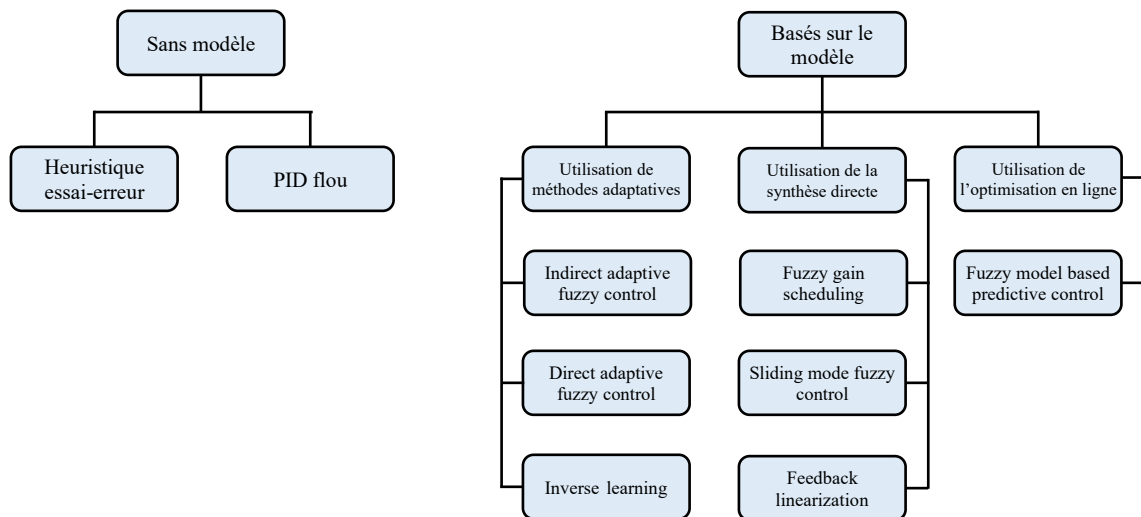


Figure 2.28. Classification des contrôleurs flous

❖ Conception heuristique par essais-erreurs

Cette méthodologie est probablement la première technique utilisée pour concevoir des contrôleurs flous. La technique utilise l'expérience accumulée pendant des années de contrôle manuel. L'approche typique pour construire ces contrôleurs a été la formulation de bases de règles en utilisant les informations fournies par le manuel de l'opérateur. La plupart du temps, ces contrôleurs sont utilisés à un niveau élevé comme une sorte de contrôle de supervision lorsque des questions comme la stabilité ne sont pas critiques. La stabilisation du système est une tâche accomplie par des contrôleurs basique. Dans de nombreux cas, le contrôleur flou n'est pas utilisé directement dans le système en mode automatique, mais il est utilisé comme système de support pour l'opérateur. Des applications réussies de cette technique ont été signalées dans les domaines du contrôle des fours dans les cimenteries, des séquences de démarrage des chaudières, de l'incinération des déchets et du traitement des eaux usées.

❖ Conception de contrôleurs flous de type PID

Ces contrôleurs flous de type PID (proportionnel intégral et dérivé) ont été inclus dans cette classe sans modèle parce qu'un contrôleur flou peut être conçu en utilisant les mêmes expériences conçues pour régler les contrôleurs linéaires de type PID sur une base sans modèle ou en utilisant des modèles simples (la réponse indicielle). L'idée principale derrière cette approche est que n'importe quel PID avec entrée et sortie limitée peut être reproduit exactement par un système flou. La méthode de conception se déroule comme suit :

- Régler un contrôleur PID en utilisant l'une des méthodes traditionnelles (Ziegler Nichols ou Kappa-Tau, etc.).
- Construire un contrôleur flou équivalent au PID trouvé.
- Ajuster le contrôleur flou d'une manière heuristique.

La présentation la plus populaire du contrôleur flou basique est le contrôleur $e, \Delta e$. Il s'agit d'un contrôleur flou avec deux entrées, e = erreur et Δe = variation de l'erreur, et une sortie, qui est l'action de contrôle u ou Δu selon que le contrôleur agit comme PD (proportionnel dérivé) ou PI (proportionnel intégrale). Une caractéristique intéressante de cette description du contrôleur est le fait qu'une analogie directe peut être établie avec les contrôleurs PD et PI classiques. Le contrôleur flou avec action directe sera analogique à un contrôleur PD, et le contrôleur flou avec action incrémentale sera analogique à un contrôleur PI.

A partir de la structure générale d'un système flou, différents types de contrôleurs flous vont alors pouvoir être définis.

✚ Contrôleur PD flou

Dans ce contrôleur deux entrées sont traitées, l'erreur e et la dérivée de l'erreur de pour une unique commande u_{PD} . Les deux entrées sont normalisées au moyen de gains de normalisation, G_e pour l'erreur et G_{de} pour la dérivée de l'erreur. Un gain de dénormalisation, G_u , est affecté sur la sortie. L'univers du discours pour le moteur flou est ainsi ramené sur l'intervalle $[-1, +1]$. Les facteurs de normalisation permettent ainsi de définir le domaine de variation normalisé des entrées et le gain de dénormalisation définit le gain en sortie du correcteur flou de type PD. Ces éléments permettent d'agir de façon globale sur la surface de commande en élargissant ou réduisant l'univers du discours des grandeurs de commande.

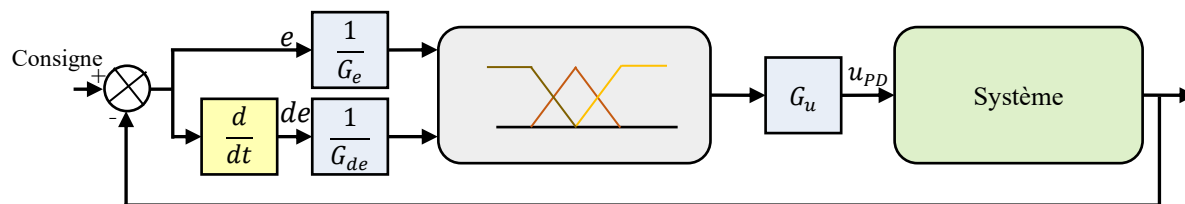


Figure 2.29. Structure du correcteur PD flou

Un contrôleur flou parfaitement équivalent à un régulateur PD peut être construit, et l'ajustage des paramètres est initialement effectué par les facteurs d'échelle à l'entrée et la sortie du contrôleur. Une méthode très sûre pour ajuster le contrôleur flou est tout d'abord commencer par un régulateur PD stabilisant puis le remplacer par son équivalent version floue, ensuite on ajuste les paramètres des fonctions d'appartenance afin d'améliorer les performances du contrôleur.

La procédure peut se résumer comme suit :

Pour remplacer un contrôleur PD décrit par la fonction de transfert :

$$C(p) = K_p(1 + \tau_d p) \quad (2.21)$$

Les facteurs d'échelle doivent satisfaire les conditions sur les gains suivantes :

$$\begin{cases} \frac{G_u}{G_e} = K_p \\ \frac{G_u}{G_{de}} = K_p \tau_d \\ \frac{\max|e|}{G_e} \leq 1 \\ \frac{\max|de|}{G_{de}} \leq 1 \end{cases} \quad (2.22)$$

Les deux premières conditions garantissent l'application des mêmes gains du contrôleur. Cependant que, les deux dernières permettent d'éviter la saturation des efforts du contrôleur.

✚ Contrôleur PI flou

Pour réaliser un contrôleur de type PI, il suffit d'intégrer la sortie du système flou comme il est indiqué sur la figure 2.30.

Deux entrées sont traitées, l'erreur e et la dérivée de l'erreur de pour une unique commande u_{PI} .

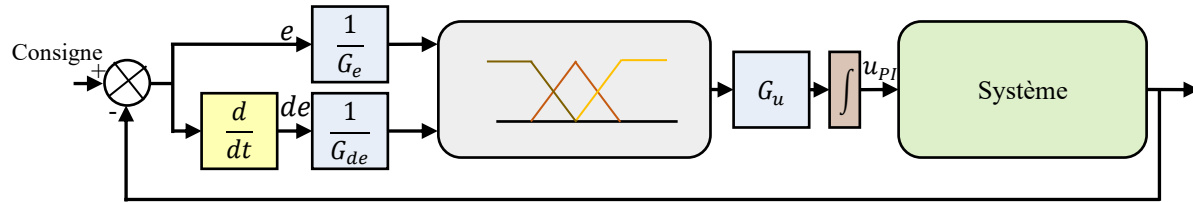


Figure 2.30. Structure du correcteur PI flou

La procédure est identique à celle du PD flou et se résume comme suit :

Pour remplacer un contrôleur PI décrit par la fonction de transfert :

$$C(p) = K_p(1 + \frac{1}{\tau_i} p) \quad (2.23)$$

Les facteurs d'échelle doivent satisfaire les conditions sur les gains suivantes :

$$\begin{cases} \frac{G_u}{G_{de}} = K_p \\ \frac{G_u}{G_e} = \frac{K_p}{\tau_i} \\ \frac{\max|e|}{G_e} \leq 1 \\ \frac{\max|de|}{G_{de}} \leq 1 \end{cases} \quad (2.24)$$

✚ Contrôleur PID flou

La conception d'un contrôleur PID flou peut être réalisée de différentes manières. L'une d'entre elles est de construire un contrôleur flou avec trois entrées : l'erreur (action proportionnelle), la

dérivée de l'erreur (action dérivée) et la somme de l'erreur (action intégrale). L'inconvénient pour un tel contrôleur est que le nombre de règles va augmenter et au lieu de 25 règles (pour les systèmes avec cinq fonctions d'appartenance sur chaque entrée), ce contrôleur aura 125 règles, ce qui rend la tâche de réglage très difficile. Une solution plus efficace est de diviser le contrôleur en deux contrôleurs, un qui est l'équivalent PD et un autre qui fournit l'action intégrale. Le nombre de règles sera donc réduit (voir la figure 2.31).

Les deux entrées sont l'erreur e et la dérivée de l'erreur de et la commande est notée u_{PID} .

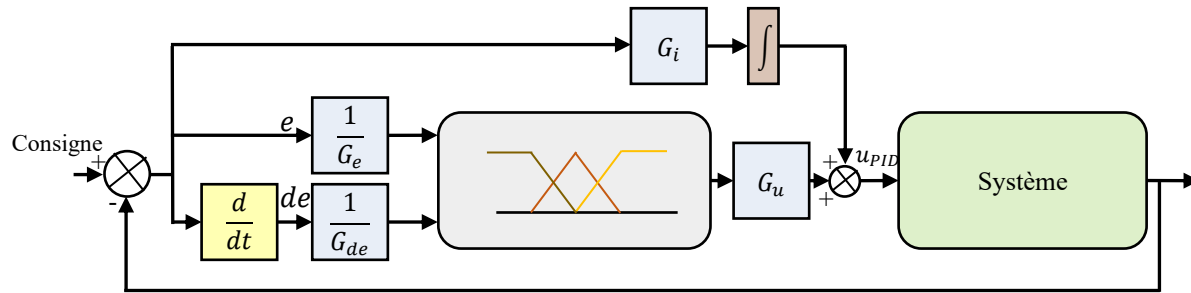


Figure 2.31. Structure du correcteur PID flou

La procédure d'ajustement du contrôleur se résume comme suit :

Pour remplacer un contrôleur PID décrit par la fonction de transfert :

$$C(p) = K_p(1 + \tau_d p + \frac{1}{\tau_i} p) \quad (2.25)$$

Les facteurs d'échelle doivent satisfaire les conditions sur les gains suivantes :

$$\begin{cases} \frac{G_u}{G_e} = K_p \\ G_i = \frac{K_p}{\tau_i} \\ \frac{G_u}{G_{de}} = K_p \tau_d \\ \frac{\max|e|}{G_e} \leq 1 \\ \frac{\max|de|}{G_{de}} \leq 1 \end{cases} \quad (2.26)$$

2.6.9. Organigramme de la conception d'un contrôleur flou

Les principales étapes à suivre pour concevoir un système de réglage par logique floue sont indiquées dans la figure 2.32.

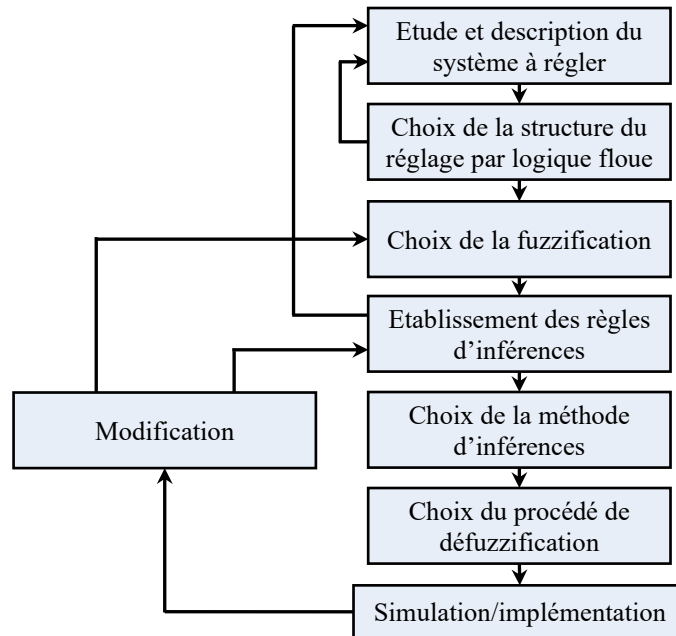


Figure 2.32. Organigramme de conception d'un contrôleur flou

2.7. Avantages et inconvénients de la commande par logique floue

La commande par logique floue réunit un certain nombre d'avantages et de désavantages. Les avantages essentiels sont :

- La non-nécessité d'une modélisation (cependant, il peut être utile de disposer d'un modèle convenable) ;
- La possibilité d'implanter des connaissances (linguistiques) de l'opérateur de processus ;
- La maîtrise du procédé avec un comportement complexe (fortement non-Linéaire et difficile à modéliser) ;
- L'obtention fréquente de meilleures prestations dynamiques (régulateur non-linéaire) ;
- Deux solutions sont possibles : solution par logiciels (par microprocesseur, DSP et PC) ou solution matérielle (par fuzzy processeurs).

Les inconvénients de la commande par logique floue sont :

- Le manque de directives précises pour la conception d'un réglage (choix des grandeurs à mesurer, détermination de la fuzzification, des inférences et de la défuzzification) ;
- L'approche artisanale et non systématique (implantation des connaissances des opérateurs souvent difficile) ;
- L'impossibilité de la démonstration de la stabilité du circuit de réglage en toute généralité (en l'absence d'un modèle valable) ;
- La possibilité d'apparition de cycles limites à cause de fonctionnement non-linéaire ;

- La cohérence des inférences non garantie a priori (apparition de règles d'inférence contradictoires possible).

En tout cas, on peut confirmer que le réglage par logique floue présente une solution valable par rapport aux réglages conventionnels. Cela est confirmé non seulement par un fort développement dans beaucoup de domaines d'application, mais aussi par des travaux de recherche sur le plan théorique. Ainsi, il est possible de combler quelques lacunes actuelles, comme le manque de directives pour la conception et l'impossibilité de la démonstration de la stabilité en l'absence d'un modèle valable.

2.8. Conclusion

En conclusion, on peut dire que la logique floue est très utile lorsque l'on se trouve confronté à des systèmes qui ne sont pas modélisables, ou ceux qui le sont, mais difficilement.

De même, cette méthode est très avantageuse si l'on possède un bon niveau d'expertise humaine. En effet, il faut fournir au système flou toute une base de règle exprimée en langage naturel pour permettre de raisonner et de tirer des conclusions. Plus l'expertise humaine d'un système est importante et plus on est capable d'ajouter des règles d'inférences au système.

En logique floue, la qualité de la méthode dépend du développeur.

3.1. Introduction

Le Machine Learning connu spécialement dans le jargon de l'informatique, est un domaine de l'intelligence artificielle qui consiste à programmer une machine pour que celle-ci apprenne à réaliser des tâches en étudiant des exemples de ces dernières. D'un point de vue mathématique, ces exemples sont représentés par des données que la machine utilise pour développer un modèle.

A titre d'exemple pour une fonction du type $f(x) = ax + b$, le but du jeu en Machine Learning est de trouver les paramètres a et b qui donnent le meilleur modèle possible. C'est-à-dire, le modèle qui s'ajuste le mieux à nos données.

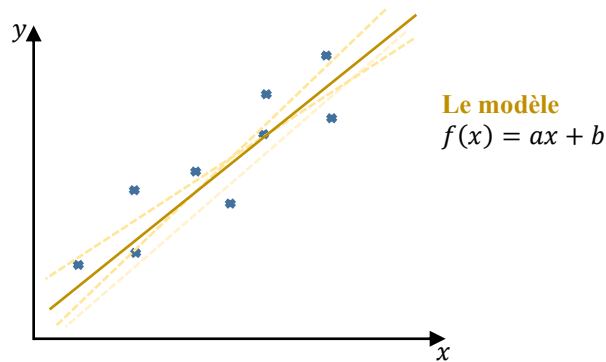


Figure 3.1. Ajustement du meilleur modèle pour des données

Pour cela, on programme dans la machine un algorithme d'optimisation qui va venir tester différentes valeurs de a et b jusqu'à obtenir la combinaison qui minimise la distance entre le modèle et les points.

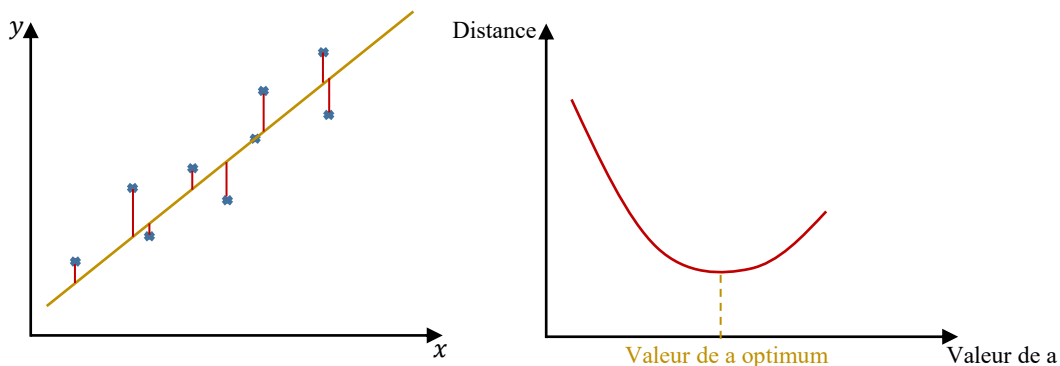


Figure 3.2. Distance entre les points et le modèle

Ça consiste donc à développer un modèle en se servant d'un algorithme d'optimisation pour minimiser les erreurs entre le modèle et les données tout simplement. Et des modèles on en compte beaucoup, tels que les modèles linéaires, les arbres de décision, etc. Chaque modèle vient avec son algorithme d'optimisation (l'algorithme de la descente de gradient pour les modèles linéaires par exemple).

Connu aussi dans le jargon de l'informatique, le Deep Learning est un domaine du Machine Learning dans lequel, au lieu de développer un des modèles que nous avons cité dans le paragraphe précédent, on développe à la place ce qu'on appelle **des réseaux de neurones artificiels (RNA)**. Alors le principe reste exactement le même : c'est à dire que l'on fournit à la machine des données, et elle utilise un algorithme d'optimisation pour ajuster le modèle à ces données, mais cette fois ci, notre modèle n'est pas une simple fonction du type $f(x) = ax + b$, mais plutôt un réseau de fonctions connectées les unes aux autres « un réseau de neurones ».

Mais ce qu'il faut savoir, c'est que plus ces réseaux sont profonds, c'est-à-dire plus ils contiennent de fonctions à l'intérieur, plus la machine est capable d'apprendre à réaliser des tâches complexes, comme reconnaître des objets, identifier une personne sur une photo, conduire une voiture, etc. Voilà donc pourquoi on parle d'apprentissage profond, c'est-à-dire de Deep Learning, lorsque l'on développe des réseaux de neurones artificiels.

Un réseau de neurones artificiels est l'une des approches d'intelligence artificielle les plus sophistiquées au monde, dont le développement se fait à travers des méthodes par lesquelles l'être humain essaye toujours d'imiter la nature et de reproduire des modes de raisonnement et de comportement qui lui sont propre. Comme leur nom l'indique, ce sont des réseaux informatiques qui tentent de simuler, de manière fidèle, le processus de décision dans les réseaux de cellules nerveuses (neurones) du système nerveux biologique (humain ou animal).

Ils sont employés dans divers domaines et dans toutes sortes d'applications :

- Contrôle : commande des processus, modélisation et identification des systèmes physiques, asservissement de robots, aide à la décision, diagnostic, etc.
- Traitement d'images : reconnaissance faciale, reconnaissance de caractères, reconnaissance de forme, classification et clustering, compression d'image, cryptage et décryptage, etc.
- Optimisation : Planification, ordonnancement et allocation de ressources, gestion et finances, etc.
- Simulation : prévision météorologique, simulation, évaluation des écosystèmes, etc.
- Traitement du signal : traitement de la parole, filtrage, classification, identification de source, etc.

L'excitation à un RNA ne devrait pas se limiter à sa ressemblance avec les processus de décision dans le cerveau humain. Même son degré de capacité d'auto-organisation peut être intégré dans les ordinateurs numériques conventionnels en utilisant des algorithmes d'intelligence artificielle compliqués. La principale contribution des RNA est que, dans leur imitation du réseau neuronal biologique, ils permettent une programmation de très bas niveau pour permettre de résoudre des problèmes complexes, en particulier ceux qui sont non analytiques et/ou non linéaires et/ou non stationnaires et/ou stochastiques, et de le faire d'une manière auto-organisée qui s'applique à un

large éventail de problèmes sans reprogrammation ou autre interférence dans le programme lui-même.

L'objectif de cette partie du cours est de donner les définitions de base relatives aux réseaux de neurones, nous verrons par la suite les différentes architectures des RNA et les modes d'apprentissage les plus utilisées.

Tout au long de cette partie, nous utiliserons le terme « réseaux neuronaux » pour désigner les réseaux de neurones artificiels plutôt que les réseaux biologiques.

3.2. Aspect historique

Pour bien comprendre le fonctionnement des réseaux de neurones artificiels, nous allons revenir à l'origine de leur histoire. Nous verrons comment ils ont été inventés et quelle furent leur évolution à travers le temps pour en arriver à la technologie que nous connaissons aujourd'hui.

1890 : Cette année a connu l'apparition des premières notions des RNA, lorsqu'un célèbre psychologue américain W. James introduit le concept de mémoire associative, et propose une loi de fonctionnement qui servira par la suite pour l'apprentissage sur les réseaux de neurones. Cette loi est connue de nos jours sous le nom de la règle de Hebb.

1943 : Invention des premiers neurones artificiels. Les premiers réseaux de neurones ont donc été inventés en 1943 par deux mathématiciens et neuroscientifiques du nom de Warren McCulloch et Walter Pitts. Dans leur article scientifique intitulé « A LOGICAL CALCULUS OF THE IDEAS IMMANENT IN NERVOUS ACTIVITY » ils expliquent comment ils ont pu programmer des neurones artificiels en s'inspirant du fonctionnement des neurones biologiques. Ils ont pu démontrer qu'avec ce modèle, il était possible de reproduire certaines fonctions logiques, telles que la porte AND et la porte OR.

1949 : D. Hebb, physiologiste américain explique le conditionnement chez l'animal par les propriétés des neurones eux-mêmes. Il a pu démontrer en partie ses résultats expérimentaux en utilisant la loi de modification des propriétés des connexions entre neurones. Le terme connexionnisme fut donc introduit pour parler de modèles massivement parallèles et connectés.

1957 : Bien que le modèle proposé par McCulloch et Pitts pose les bases de ce qu'est encore aujourd'hui le RNA, il contient un certain nombre de failles, notamment le fait qu'il ne dispose pas d'algorithme d'apprentissage. Il a fallu attendre une quinzaine d'année plus tard, pour qu'un psychologue américain trouva comment améliorer ce modèle, en proposant le premier algorithme d'apprentissage de l'histoire des RNA en s'inspirant de la théorie de Hebb. Il s'agit de Franck Rosenblatt, l'inventeur du perceptron. Ce dernier était capable d'apprendre à différencier des formes simples et à calculer certaines fonctions logiques.

1960 : B. Widrow, chercheur Américain à Stanford, développe un modèle ressemblant au perceptron appelé ADALINE (ADaptive LInear NEuron). Ce modèle sera par la suite le modèle de base des réseaux multicouches.

1969 : M.L. Minsky et S. Papert publièrent un livre intitulé « perceptrons » dans lequel on trouve des arguments mathématiques démontrant les limitations du perceptron, en particulier son incapacité à résoudre les problèmes qui sont non linéairement séparables, tel que la fonction XOR.

Cela va avoir une grande incidence sur la recherche dans ce domaine. Elle va fortement diminuer jusqu'en 1972, où T. Kohonen présente ses travaux sur les mémoires associatives. Et propose des applications à la reconnaissance de formes. On connut alors le premier hiver de l'intelligence artificielle, du début des années 70 au début des années 80, période durant laquelle il n'y eut quasiment plus d'investisseurs pour financer les recherches en I.A.

1982 : Ce n'est qu'au début des années 80 que l'intérêt des chercheurs pour les réseaux de neurones renaît grâce à Hopfield qui proposa les réseaux de neurones associatifs.

1986 : Tout changea avec le développement du perceptron multicouche (réseaux de neurones multicouches) en 1986 par David Rumelhart, Ronald J. Williams et Geoffrey Hinton. En se basant sur l'algorithme de rétropropagation du gradient établi par Paul Werbos, permettant l'entraînement pratique des réseaux multicouches. Au fil du temps le modèle perceptron multicouche continua d'évoluer, notamment avec l'apparition de nouvelles fonctions d'activation.

1990 : dans les années 90, on commença à développer les premières variantes du perceptron multicouches. Le célèbre Yann Lecun inventa les premiers réseaux de neurones Convolutifs (ConvNet), réseaux qui sont capables de reconnaître et de traiter des images.

1997 : Apparition des premiers réseaux de neurones récurrents, qui sont encore une fois une variante du perceptron multicouches et qui permettent de traiter efficacement des problèmes de série temporelles comme la lecture de textes ou encore la reconnaissance vocale.

Malgré l'existence de tout ça sur les RNA dans les années 90, il a fallu attendre longtemps avant de voir émerger les technologies que l'on a aujourd'hui pour deux grandes raisons : la première, c'est que pour bien fonctionner, un RNA doit être entraîné sur une très grosse quantité de données dépassant parfois les millions voire les dizaines de millions de données. Or dans les années 90, on ne disposait pas d'autant de données (on n'avait pas des millions voir des dizaines de millions de photos de chats, de chiens, de voiture, de piétons, etc. toutes bien classées et bien répertoriées). Il a fallu attendre l'arrivée d'internet, des smartphones, et des objets connectés pour commencer à collecter de grosses quantités de données, d'images et de son pouvant être exploitées pour les RNA. La deuxième raison de cette attente est liée à la puissance des ordinateurs des années 80 à 90. Entraîner un RNA demande pas mal de temps et pas mal de puissance, et il a fallu attendre qu'on dispose d'excellents CPU et GPU pour enfin obtenir de bons résultats.

2012 : en fait, les RNA n'ont pris leur envol qu'en 2012, lors d'une compétition de vision par ordinateur nommée ImageNet, où une équipe de chercheurs menée par Geoffrey Hinton, développa un réseau de neurones capable de reconnaître n'importe quelle image avec une meilleure performance que tous les autres algorithmes de l'époque.

Depuis ce jour, l'utilisation des RNA a suscité intérêt de tout le monde et connaît un développement sans précédent.

3.3. Le neurone biologique

La base de l'intelligence des êtres humains est le fonctionnement harmonieux du cerveau qui est constitué d'environ 100 milliards de neurones interconnectés. Chaque neurone est connecté en

moyenne avec plus de 10 000 autres. Ces neurones nous permettent d'exécuter des tâches extrêmement complexes du point de vue machine.

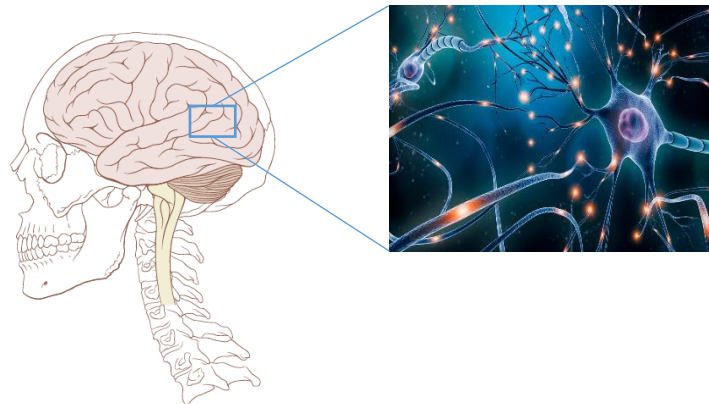


Figure 3.3. Système nerveux

En termes de biologie, les neurones sont des cellules excitables connectées les unes aux autres et ayant pour rôle de transmettre des informations dans notre système nerveux. Chaque neurone est composé de plusieurs dendrites, d'un corps cellulaire contenant le noyau, et d'un axone (voir figure 3.4).

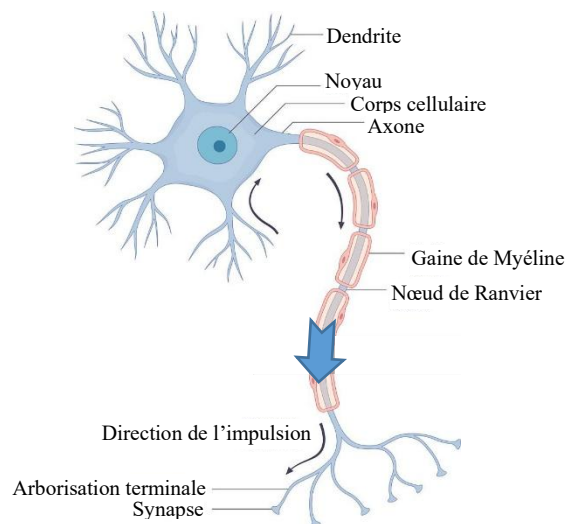


Figure 3.4. Neurone biologique

Les dendrites sont en quelque sorte les portes d'entrée d'un neurone. C'est à cet endroit, au niveau de la synapse que le neurone reçoit des signaux lui provenant des neurones qui le précèdent. Ces signaux peuvent être de type excitateur ou à l'inverse inhibiteur (un peu comme si nous avions des signaux qui valent 1 et d'autres qui valent 0). Lorsque la somme de ces signaux dépasse un certain seuil, le neurone s'active et produit alors un signal électrique. Ce signal circule le long de l'axone en direction des terminaisons pour être envoyé à son tour vers d'autres neurones (qui fonctionnent exactement de la même manière) de notre système nerveux.

Ces neurones peuvent apprendre en changeant l'intensité de leurs connexions avec d'autres neurones ou la détruire ou même créer de nouvelles connexions. Elles peuvent aussi changer leur(s)

règle(s) de traitement de l'information. Ce processus de changement est appelé apprentissage et joue un rôle fondamental dans le comportement du neurone.

3.4. Le neurone formel

Un neurone formel inspiré à la base des observations relatives au fonctionnement des neurones biologiques est un modèle théorique de traitement de l'information, ayant comme objectif la reproduction du raisonnement intelligent d'une manière artificielle. Son premier modèle a été présenté pour la première fois par Warren McCulloch et Walter Pitts au début des années quarante.

Ce que Mc Culloch et Pitts ont essayé de faire, c'est de modéliser ce fonctionnement en considérant qu'un neurone pouvait être représenté par une fonction de transfert qui prend en entrée des signaux x et qui retourne une sortie y (voir figure 3.5).

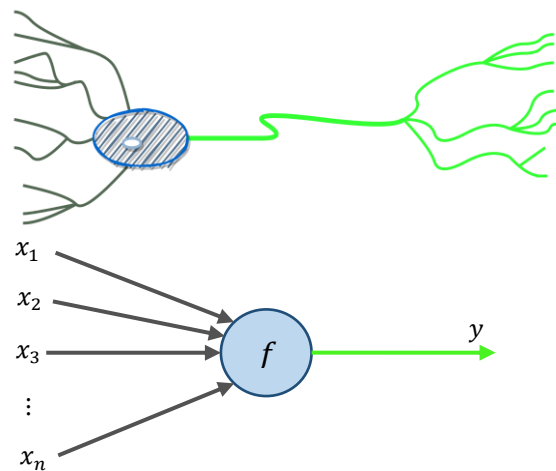


Figure 3.5. Principe du neurone artificiel

A l'intérieur de cette fonction, on trouve deux grandes étapes, la première, c'est une étape d'agrégation. On fait la somme de toutes les entrées du neurone, en multipliant au passage chaque entrée par un coefficient w appelé poids synaptique tel qu'il est montré sur la figure 3.6.

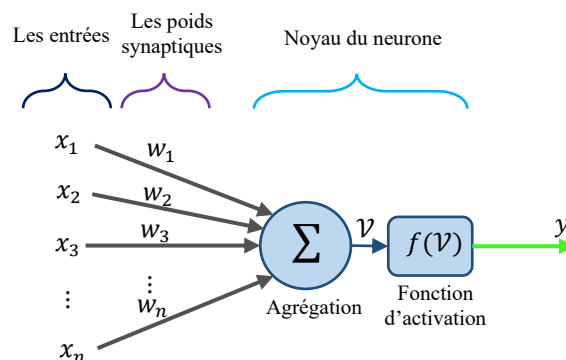


Figure 3.6. Composition d'un neurone artificiel

Avec x_i (pour $i = 1$ à n) qui représentent le vecteur d'entrée, elles proviennent soit des sorties d'autres neurones, soit de stimuli sensoriels (capteur visuel, sonore...). Et les w_i représentants les poids synaptiques du neurone. Ils correspondent à l'efficacité synaptique dans les neurones biologiques (un poids négatif vient inhiber une entrée c.-à-d. « synapse inhibitrice » ; alors qu'un poids positif vient l'accroître « synapse excitatrice »). Ces poids pondèrent les entrées et peuvent être modifiés par apprentissage.

Dans la phase d'agrégation, on obtient donc une expression de la forme :

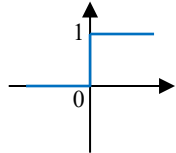
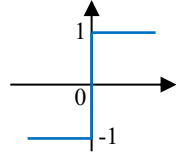
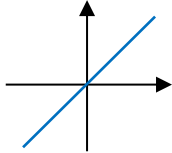
$$\text{Agrégation} \quad \mathcal{V} = w_1x_1 + w_2x_2 + w_3x_3 + \dots w_nx_n \quad (3.1)$$

Une fois cette étape réalisée, on passe à la phase d'activation. On regarde le résultat du calcul effectué précédemment, si celui-ci dépasse un certain seuil (dans le neurone de Mac Cullochs et Pitts, la fonction d'activation est du type à seuil, le seuil de déclenchement est généralement provoqué par une entrée inhibitrice x_0 , parfois appelée biais), en général 0, alors le neurone s'active et retourne une sortie $y = 1$. Sinon, il reste à 0.

$$\text{Activation} \quad \begin{cases} y = 1 & \text{si } \mathcal{V} \geq 0 \\ y = 0 & \text{sinon} \end{cases} \quad (3.2)$$

Il existe de nombreuses formes possibles pour la fonction d'activation f . Ces fonctions définissent le rendement du neurone à partir d'une entrée ou un ensemble d'entrées $y = f(\mathcal{V})$.

Comme nous venons de le voir, la première version du neurone formel était implémentée avec une fonction d'activation à seuil, par la suite de nombreuses versions ont été proposées. Le fameux neurone de McCulloch et Pitts a été généralisé avec plusieurs variantes, en fonction de différentes fonctions d'activations. Les fonctions les plus utilisées sont les fonctions à seuil, linéaires et sigmoïdes. Quelques fonctions d'activation usuelles sont données dans le tableau ci-dessous.

Nom de la fonction	Relation entrée/sortie	Forme de la courbe
Seuil	$\begin{cases} y = 0 & \text{si } \mathcal{V} < 0 \\ y = 1 & \text{si } \mathcal{V} \geq 0 \end{cases}$	
Seuil symétrique	$\begin{cases} y = -1 & \text{si } \mathcal{V} < 0 \\ y = 1 & \text{si } \mathcal{V} \geq 0 \end{cases}$	
Linéaire	$y = \mathcal{V}$	

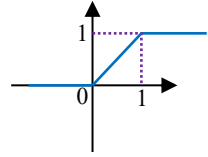
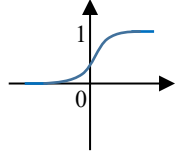
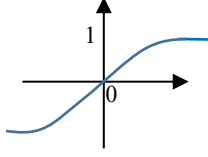
Linéaire saturée	$\begin{cases} y = 0 & \text{si } \mathcal{V} < 0 \\ y = f(x) & \text{si } 0 \leq \mathcal{V} \leq 1 \\ y = 1 & \text{si } \mathcal{V} > 1 \end{cases}$	
Sigmoïde	$y = \frac{1}{1 + e^{-\mathcal{V}}}$	
Tangente hyperbolique	$y = \frac{e^{\mathcal{V}} - e^{-\mathcal{V}}}{e^{\mathcal{V}} + e^{-\mathcal{V}}}$	

Tableau 3.1. Exemples de fonctions d'activation utilisées dans les RNA.

Pour bien comprendre la transition entre le neurone biologique et le neurone artificiel une simple analogie permettra de faire la correspondance pour chaque élément. Cette analogie est résumée par le tableau ci-dessous :

Neurone biologique	Neurone formel
Dendrites	Signal d'entrée
Synapses	Poids des connexions
Noyau ou Somma	Fonction d'activation
Axones	Signal de sortie

Tableau 3.2. Analogie entre le neurone biologique et le neurone formel.

3.5. Description mathématique

En considérant le cas général d'un neurone formel j ayant n entrées, auquel on doit donc soumettre les n grandeurs numériques ou signaux, notées x_1 à x_n . Un modèle de neurone formel est une règle de calcul qui permet d'associer aux n entrées une sortie : c'est donc une fonction à n variables et à valeurs réelles. Un poids synaptique est associé à chaque entrée n , c'est-à-dire une valeur numérique notée w_{j1} pour l'entrée 1 jusqu'à w_{jn} pour l'entrée n . La première opération réalisée par le neurone formel consiste en une somme des grandeurs reçues en entrées, pondérées par les coefficients synaptiques, c'est-à-dire la somme :

$$\begin{aligned} \mathcal{V} &= w_{j1}x_1 + w_{j2}x_2 + w_{j3}x_3 + \cdots w_{jn}x_n \pm b_j \\ &= \sum_{i=1}^n w_{ji}x_i \pm b_j \end{aligned} \quad (3.3)$$

Cette sortie correspond à une somme pondérée des poids et des entrées plus un seuil d'activation b qu'on nomme aussi le biais du neurone.

Un biais peut être inclus en ajoutant une composante $x_0 = 1$ au vecteur x des entrées. Le biais est donc traité exactement comme tout autre poids, c.-à-d., $w_{j0} = b_j$.

$$v = \sum_{i=1}^n w_{ji}x_i + w_{j0}x_0 \quad (3.4)$$

La sortie associée aux entrées x_1 à x_n en ajoutant la fonction d'activation f est donnée par :

$$y = f(v) = f(\sum_{i=1}^n w_{ji}x_i + w_{j0}x_0) \quad (3.5)$$

Ou encore :

$$y = f(v) = f(w_j^T x + w_{j0}x_0)$$

Avec : $w_j^T = [w_{j1}, w_{j2}, \dots, w_{jn}]$

Et : $x^T = [x_1, x_2, \dots, x_n]$

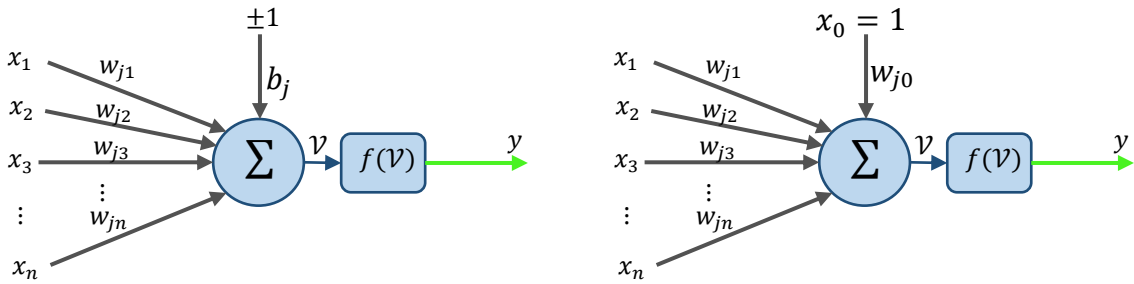


Figure 3.7. Neurone artificiel avec biais

Il est possible de décrire un neurone sous une forme matricielle, présentée dans la figure 3.8 :

$$y = f(Wx + w_0x_0) \quad (3.6)$$

Avec :

$$W = w^T = [w_1, w_2, \dots, w_n]$$

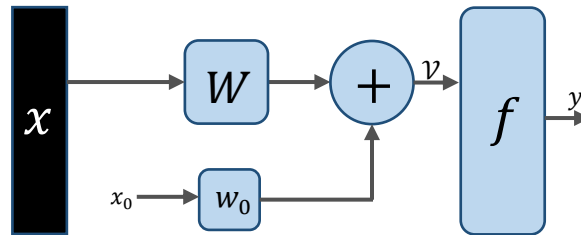


Figure 3.8. Représentation matricielle d'un neurone artificielle

Le neurone formel est l'unité élémentaire des réseaux de neurones artificiels (RNA) dans lesquels il est associé à d'autres neurones formels pour calculer des fonctions arbitrairement complexes, utilisées pour diverses applications.

3.6. Les réseaux de neurones artificiels

En réalité, un neurone élémentaire reste très limité dans les applications qu'il peut réaliser. Un neurone permet en effet, en fonction de ses variables d'entrées, de réaliser une fonction non linéaire simple. L'intérêt dans l'utilisation des neurones réside dans les propriétés qui résultent en les associant dans une structure qu'on appelle « **Réseau de Neurones** », tout en respectant une certaine logique d'interconnexion. Le comportement collectif à travers cette structure permet donc la réalisation de fonctions plus complexes par rapport aux fonctions élémentaires que peut réaliser un simple neurone. Dans un réseau de neurones, un neurone admet comme entrées soient les entrées du réseau, soient les sorties issues des autres neurones, selon son emplacement et l'architecture du réseau.

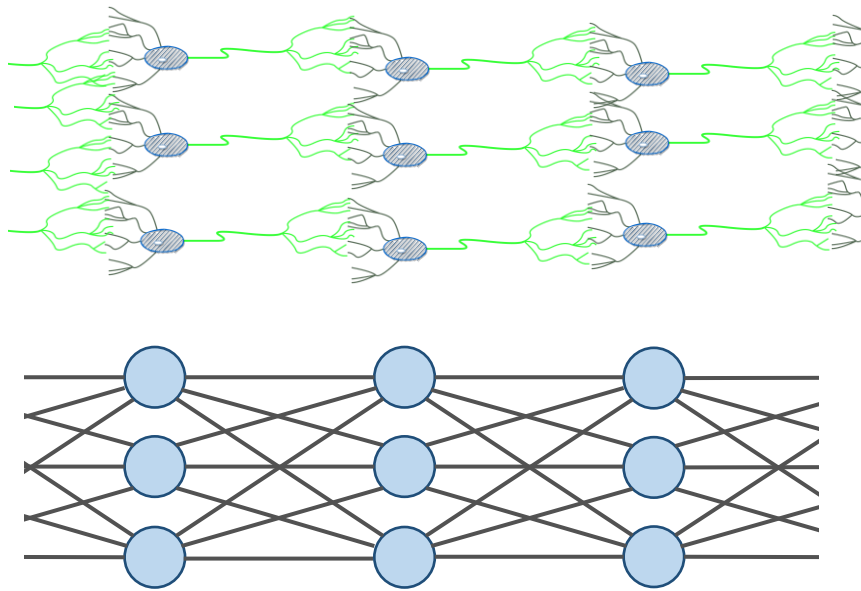


Figure 3.9. Structure d'un réseau de Neurones

Les réseaux de neurones artificiels ainsi inspirés du système nerveux, permettent de reproduire certaines caractéristiques des mémoires biologiques par le fait qu'ils sont :

- massivement parallèle ;
- capables d'apprentissage ;
- capables de mémoriser l'information dans les connexions interneurones ;
- capables de traiter des informations incomplètes.

3.6.1. Différentes architectures des réseaux de neurones artificiels

La façon avec laquelle les neurones sont organisés au sein d'un réseau de neurones définit son architecture. Ceci est traduit par la manière dont ils sont disposés et connectés. Le recours à ces différentes structures de réseaux de neurones artificiels dépend de l'objectif recherché et aussi de

la méthode d'apprentissage utilisée. Dans la plupart des cas on trouve le même type de neurones dans un même réseau.

Un réseau de neurones est constitué généralement de plusieurs couches de neurones en allant des entrées jusqu'aux sorties. On peut classer les réseaux de neurones selon leurs architectures en deux types : les réseaux de neurones à propagation avant (Feedforward ou non bouclés) et les réseaux de neurones récurrents (Feedback ou bouclés).

3.6.1.1. Les réseaux de neurones à propagation avant (Feedforward ou non bouclés)

Un réseau de neurones à propagation avant peut être imaginé comme un ensemble de neurones connectés entre eux dont l'information circulant des entrées vers les sorties sans retour en arrière. On peut alors représenter le réseau par un graphe acyclique dont les nœuds sont les neurones et les arêtes sont les connexions entre les nœuds. Si l'on se déplace dans le réseau à partir d'un neurone quelconque, en suivant les connexions et en respectant leurs sens, on ne peut pas revenir au neurone de départ (voir figure 3.10).

Ces réseaux sont des objets de type statique, c.-à-d. que si les entrées sont indépendantes du temps, les sorties le sont également. Ils sont utilisés principalement pour effectuer des tâches d'approximation de fonction non linéaire, de classification ou de modélisation de processus statiques non linéaire.

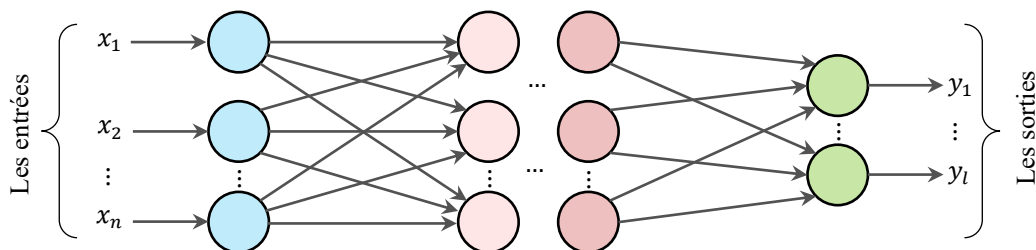


Figure 3.10. Exemple d'un réseau de neurones à propagation avant

Ce type de réseau ne peut transmettre l'information que dans un unique sens de traitement. Ils peuvent être monocouches, c'est-à-dire constitués uniquement de couches d'entrée et de sortie, ou multicouches, c'est-à-dire disposant d'un certain nombre de couches cachées.

A. Réseaux de neurones monocouches

Ce type de réseau est simple à concevoir, sa structure se compose principalement d'une couche d'entrée et d'une couche de sortie. Chacun des neurones de la couche de sortie est connecté aux neurones de la couche d'entrée avec un poids modifiable, tels qu'il est montré sur la figure 3.11.

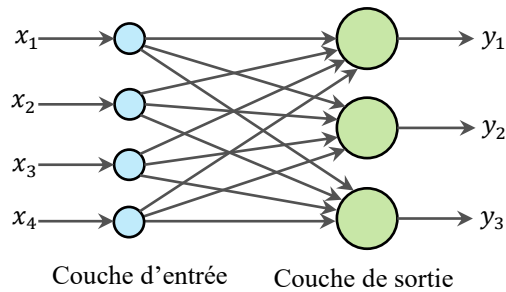


Figure 3.11. Exemple d'un réseau de neurones monocouche

B. Réseaux de neurones multicouches

Dans ce type de réseau, les neurones sont arrangés par couches. Un réseau de neurones multicouche est donc, organisé en plusieurs couches : une couche d'entrée représentant l'ensemble des neurones d'entrée (c'est une couche virtuelle associée aux entrées du système), une ou plusieurs couches cachées n'ayant aucun contact avec l'extérieur et une couche de sortie représentant l'ensemble des neurones de sortie.

Dans cette architecture, la sortie de chaque neurone dans une couche est connectée à toutes les entrées des neurones de la couche qui suit, sachant qu'il n'y a pas de connexion entre les neurones qui appartiennent à la même couche.

La figure 3.12 représente un exemple d'un réseau de neurones multicouche contenant une couche d'entrée, deux couches cachées et une couche de sortie. Dans cet exemple, il y a trois neurones sur la couche d'entrée, quatre neurones sur la première couche cachée, trois neurones sur la deuxième couche cachée et trois neurones sur la couche de sortie.

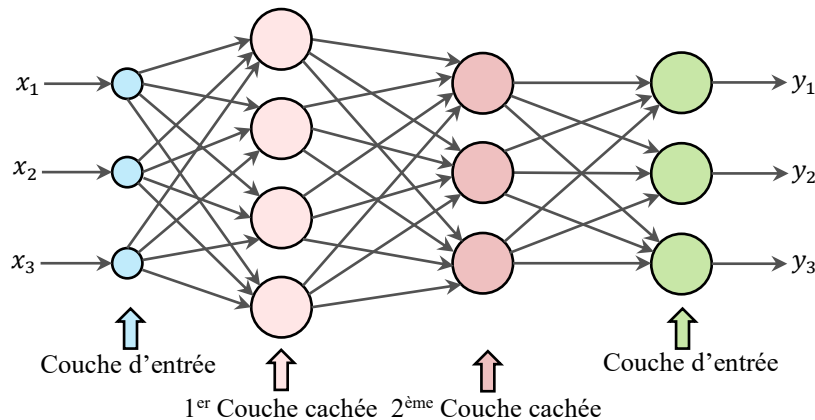


Figure 3.12. Exemple d'un réseau de neurones multicouche

Il faut noter que les réseaux de neurones multicouches sont plus efficaces que les réseaux de neurones monocouches. On a la possibilité d'approximer la majorité des fonctions sous certaines conditions en utilisant ce genre de réseau. Un théorème dit d'approximation universelle montre qu'une structure élémentaire à une seule couche cachée est suffisante pour prendre en compte les problèmes classiques de modélisation ou apprentissage statistique. En effet, toute fonction régulière peut être approchée uniformément avec une précision arbitraire et dans un domaine fini

de l'espace de ses variables, par un réseau de neurones comportant une couche de neurones cachés en nombre fini possédant tous la même fonction d'activation et un neurone de sortie linéaire.

3.6.1.2. Réseaux de neurones à connexions locales

C'est une structure qui fait partie toujours des structures multicouches en conservant une certaine topologie. Chaque neurone dans une couche possède un nombre réduit et localisé de connexion avec les neurones de la couche avale (voir figure 3.13). Les connexions sont donc moins nombreuses que dans le cas d'un réseau de neurones multicouche classique.

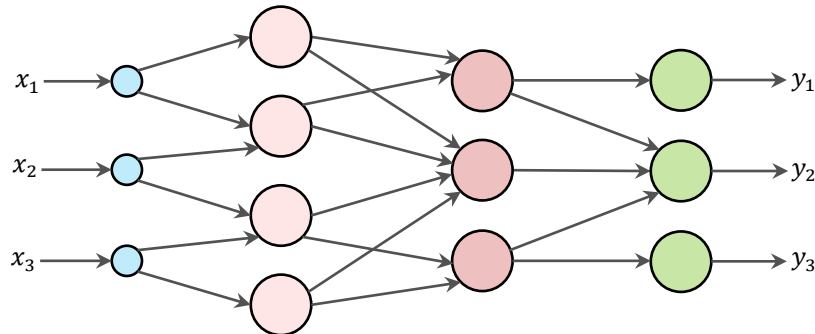


Figure 3.13. Exemple d'un réseau à connexions locales

3.6.1.3. Les réseaux de neurones récurrents (feedback network ou bouclés)

Dans les réseaux de neurones récurrents le graphe des connexions est cyclique, contrairement aux réseaux de neurones à propagation avant. Il est possible dans ce type de réseaux de faire circuler l'information dans des boucles de rétroaction, et ainsi de la faire revenir vers une couche précédente (voir figure 3.14). Ces rétroactions permettent au système de se constituer une mémoire.

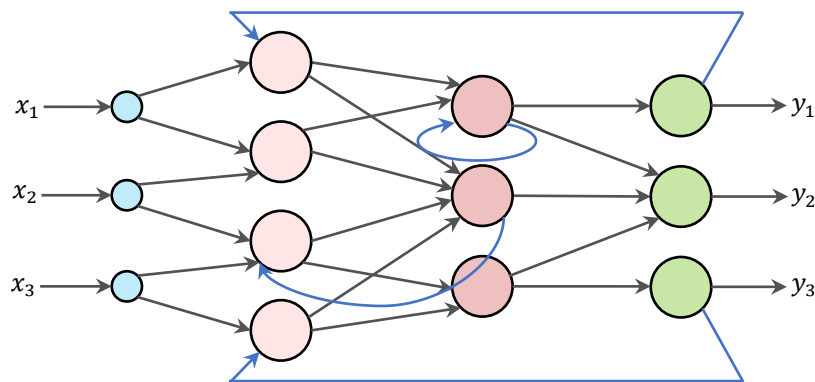


Figure 3.14. Exemple d'un réseau récurrent

La sortie d'un neurone du réseau récurrent peut donc être fonction d'elle-même ; ceci n'est évidemment concevable que si la notion de temps est explicitement prise en considération.

Les réseaux de neurones récurrents sont utilisés généralement en matière de reconnaissance vocale, de traduction et de reconnaissance d'écriture manuscrite.

N.B. Il existe de nombreuses autres architectures possibles, mais elles n'ont pas eu à ce jour la notoriété des quelques-unes que nous avons décrites dans ce cours.

3.7. Quelques modèles des RNA

Il existe dans la littérature plusieurs modèles de réseaux de neurones artificiels (figure 3.15), dans ce document nous allons citer quelques-uns les plus célèbres.

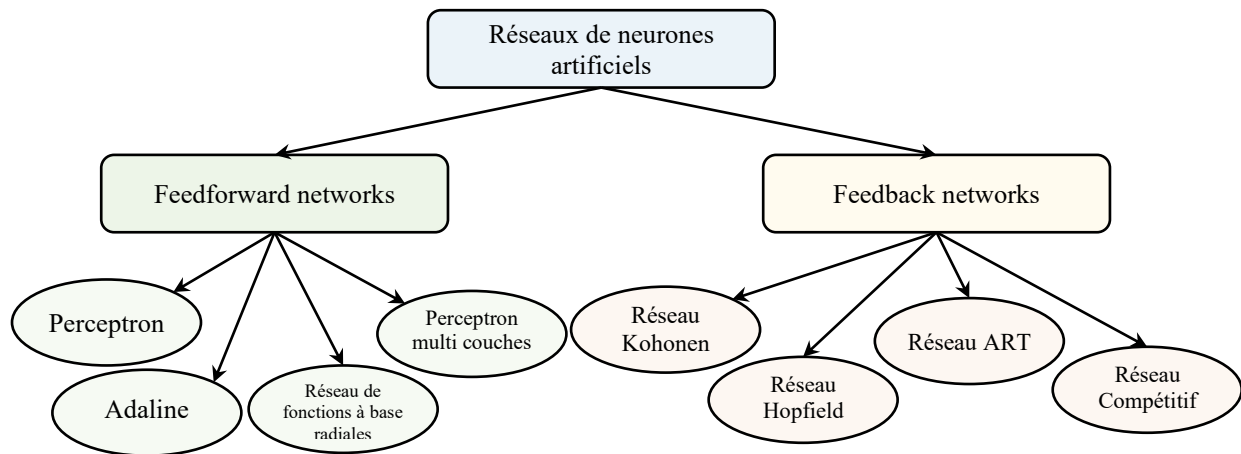


Figure 3.15. Différents modèles des réseaux de neurones artificiels

3.7.1. Le réseau de Kohonen

Appelé aussi cartes auto organisatrices de Kohonen, Leur principe est des plus simples. À partir d'un corpus d'informations, les réseaux de Kohonen classifient les données et génèrent une carte où elles sont regroupées par zones, en fonction de leurs similarités.

Ce modèle a été présenté par un chercheur finlandais du nom de Teuvo Kohonen en 1982, et malgré son intérêt, il est resté assez longtemps ignoré. Dans les années 90, a proposé diverses variantes pour la classification.

Il s'agit d'un réseau non supervisé (l'apprentissage non supervisé sera traité dans une partie ultérieure de ce cours) avec un apprentissage compétitif c'est-à-dire qu'aucune intervention humaine n'est requise et que peu d'informations sont nécessaires relativement aux caractéristiques des données d'entrée, ou l'on apprend non seulement à modéliser l'espace des entrées avec des prototypes, mais également à construire une carte à une ou deux dimensions permettant de structurer cet espace.

L'architecture des réseaux de Kohonen est constituée de deux couches de neurones, une en entrée et une en sortie. Notons que les neurones de la couche d'entrée sont entièrement connectés à la couche de sortie (voir figure 3.16).

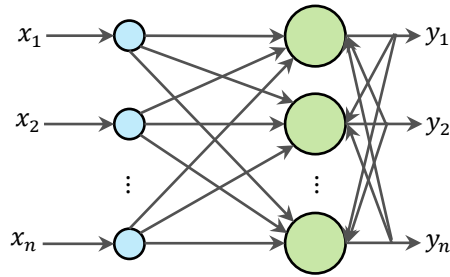


Figure 3.16. Le réseau de Kohonen

Les neurones de la couche de sortie sont placés dans un espace d'une ou de deux dimensions en général, chaque neurone possède donc des voisins dans cet espace. Et enfin chaque neurone de la couche de sortie possède des connexions latérales récurrentes dans sa couche. Le neurone inhibe les neurones éloignés et laisse agir les neurones voisins.

3.7.2. Le réseau de Hopfield

Les réseaux de Hopfield sont des réseaux récurrents et entièrement connectés qui est présenté en 1982 par le physicien John Hopfield. Dans ce type de réseau, chaque neurone est connecté à chaque autre neurone et tous les neurones de cette architecture sont à la fois neurone d'entrée et neurone de sortie du réseau. Ce modèle de Hopfield utilise l'architecture des réseaux entièrement connectés et récurrents, les sorties sont en fonction des entrées et du dernier état pris par le réseau. Ils fonctionnent comme une mémoire associative non linéaire et sont capables de trouver un objet stocké en fonction de représentations partielles ou bruitées. L'application principale des réseaux de Hopfield est l'entrepôt de connaissances mais aussi la résolution de problèmes d'optimisation. Le mode d'apprentissage utilisé ici est le mode non supervisé.

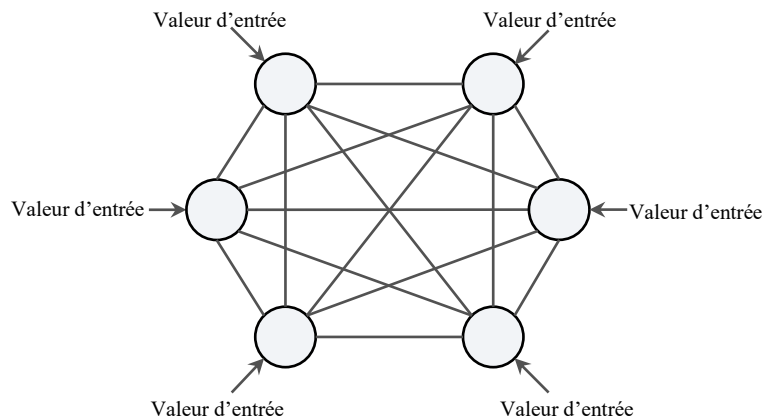


Figure 3.17. Exemple d'un réseau de Hopfield

3.7.3. Le réseau ART

Carpenter et Grossberg développent, en 1987, l'ART (Adaptive Resonance Theory /Théorie de Résonance Adaptative). C'est un modèle de réseau de neurones à architecture évolutive.

Ce sont des réseaux à apprentissage par compétition. Le problème majeur qui se pose dans ce type de réseaux est le dilemme « stabilité/plasticité ». En effet, dans un apprentissage par compétition, rien ne garantit que les catégories formées vont rester stables. La seule possibilité, pour assurer la stabilité, serait que le coefficient d'apprentissage tende vers zéro, mais le réseau perdrait alors sa plasticité. Les ART ont été conçus spécifiquement pour contourner ce problème. Dans ce genre de réseau, les vecteurs de poids ne seront adaptés que si l'entrée fournie est suffisamment proche, d'un prototype déjà connu par le réseau. On parlera alors de résonance. A l'inverse, si l'entrée s'éloigne trop des prototypes existants, une nouvelle catégorie va alors se créer, avec pour prototype, l'entrée qui a engendré sa création. Il est à noter qu'il existe deux principaux types de réseaux ART : les ART-1 pour des entrées binaires et les ART-2 pour des entrées continues. Le mode d'apprentissage des ART peut être supervisé ou non.

3.7.4. Le Perceptron

C'est historiquement le premier réseau de neurones, Il a été proposé en 1958 par Frank Rosenblatt. C'est un réseau simple, puisqu'il ne se compose que d'une couche d'entrée et d'une couche de sortie. Il est calqué, à la base, sur le système visuel et de ce fait, il a été conçu dans un but premier à la reconnaissance de formes.

Le perceptron est un algorithme d'apprentissage supervisé de classifieurs binaires (c'est-à-dire séparant deux classes). Il s'agit d'un neurone formel muni d'une règle d'apprentissage qui permet de déterminer automatiquement les poids synaptiques de manière à séparer un problème d'apprentissage supervisé. Si le problème est linéairement séparable, un théorème assure que la règle du perceptron permet de trouver une séparatrice entre les deux classes.

Le perceptron peut être vu comme le type de réseau de neurones le plus simple. C'est un classifieur linéaire. Ce type de réseau neuronal ne contient aucun cycle (il s'agit d'un réseau de neurones à propagation avant). Dans sa version simplifiée, le perceptron est mono-couche et n'a qu'une seule sortie (booléenne) à laquelle toutes les entrées (booléennes) sont connectées.

Un perceptron à n entrées $[x_1, \dots, x_n]$ et à une seule sortie y est défini par la donnée de n poids (ou coefficients synaptiques) $[w_1, \dots, w_n]$ et un biais (ou seuil) b par :

$$y = f(v) = \begin{cases} 1 & \text{si } \sum_{i=1}^n w_i x_i > b \\ 0 & \text{sinon} \end{cases} \quad (3.7)$$

La sortie y résulte alors de l'application de la fonction de Heaviside (seuil) au potentiel post-synaptique.

$$v = f(\sum_{i=1}^n w_i x_i - b) \quad (3.8)$$

Avec :

$$\forall x \in \mathbb{R}, H(x) = \begin{cases} 1 & \text{si } x \geq 0 \\ 0 & \text{si } x < 0 \end{cases}$$

Cette fonction non linéaire est appelée fonction d'activation. Une alternative couramment employée est la fonction tangente hyperbolique.

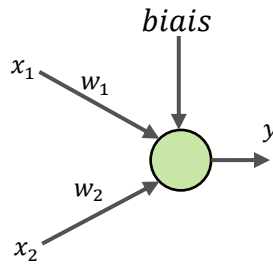


Figure 3.18. Exemple d'un Perceptron simple (monocouche)

Un Perceptron simple ne peut traiter que les problèmes linéairement séparables où les états peuvent être séparés par une droite, comme les fonctions logiques **AND** et **OR** par exemple. Pour les problèmes qui ne sont pas linéairement séparables comme la fonction logique **XOR** par exemple, on doit utiliser un Perceptron multicouche.

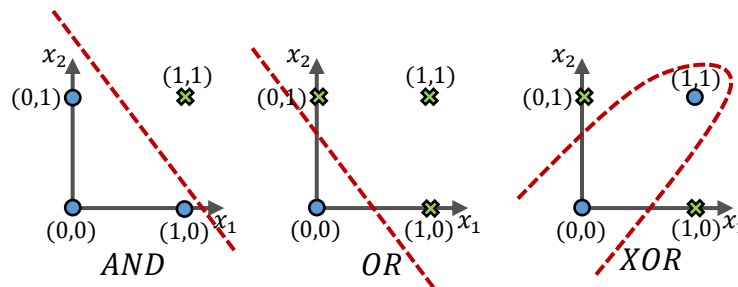


Figure 3.19. Les problèmes linéairement séparables et non linéairement séparables

3.7.5. Le Perceptron multicouche

Le perceptron multicouche « PMC » (ou Multi Layer Perceptron « MLP » en anglais) a été développé par M.Minsky et S.Papert en 1969. C'est un type de réseau de neurones artificiels organisé en plusieurs couches au sein desquelles une information circule de la couche d'entrée vers la couche de sortie uniquement ; il s'agit donc d'un réseau à propagation directe (feedforward). Ce sont une extension du perceptron monocouche, avec une ou plusieurs couches cachées entre l'entrée et la sortie. Chaque neurone dans une couche est connecté à tous les neurones de la couche précédente et de la couche suivante (sauf pour les couches d'entrée et de sortie) et il n'y a pas de connexions entre les cellules d'une même couche. Les fonctions d'activation utilisées dans ce type de réseaux sont principalement les fonctions à seuil ou sigmoïdes. Il peut résoudre des problèmes non linéairement séparables et des problèmes logiques plus compliqués, et notamment le fameux problème du XOR.

3.7.6. Réseau ADALINE

Au début des années 60 B. Widrow et M.E. Hoff ont proposé un système adaptatif qu'ils ont appelé ADALINE (*ADaptive LInear NEuron* ou plus tard *ADaptive LInear Element*).

L'ADALINE est un réseau à trois couches : une d'entrée, une couche cachée et une couche de sortie. Ce modèle est similaire au modèle de perceptron, seule la fonction de transfert change, mais

reste toujours linéaire. Les modèles des neurones utilisés dans le perceptron et l'ADALINE sont des modèles linéaires.

La structure de l'Adaline diffère du perceptron par l'utilisation d'une seule cellule d'association et l'utilisation d'une fonction de seuil différent de celle de Heaviside (-1 et t+1). De plus, il utilise un algorithme adaptatif pour mesurer l'écart entre la sortie réelle et la sortie du processeur élémentaire.

3.8. L'apprentissage

Une des propriétés les plus intéressantes des réseaux de neurones est leurs capacités à apprendre, en d'autres termes l'apprentissage (à reconnaître des images, des lettres, des chiffres, etc. à titre d'exemple). Cependant, ces connaissances ne sont pas acquises dès le départ. La plupart des réseaux de neurones apprennent à travers des exemples selon un algorithme d'apprentissage. Lors de l'apprentissage, la dynamique du réseau est modifiée par des pondérations en fonction du jeu de données présenté en entrée jusqu'à ce que le comportement souhaité soit atteint. L'apprentissage peut ou non être supervisé selon la présence ou l'absence de la réponse souhaitée. Bien que, il n'existe pas une définition générale de l'apprentissage, car ce concept touche à beaucoup de notions distinctes qui dépendent du point de vue adopté. Nous pouvons proposer dans le contexte de ce cours les définitions suivantes :

Définition 01 : *L'apprentissage d'un réseau de neurones artificiels est induit par une procédure itérative d'ajustement de ses paramètres internes (poids synaptiques et nombres de neurones). Cette procédure d'ajustement est décrite par un algorithme d'apprentissage. Celui-ci détermine alors le comportement du réseau. L'apprentissage neuronal fait appel à des exemples de comportement.*

Définition 2 : *le mécanisme pour lequel les paramètres libres d'un réseau de neurones (poids) sont adaptés à travers un processus de stimulation par l'environnement dans lequel le réseau est intégré. Le type d'apprentissage est déterminé par la façon dont les changements de paramètres sont mis en œuvre.*

Pour veiller à ce que les connexions au sein d'un réseau de neurones artificiels soient correctement établies, il faut au préalable procéder à son « entraînement » en modifiant ses paramètres. Les poids d'interconnexion seront ajustés donc suivant une règle d'apprentissage prédéfinie, de façon à ce que les sorties du réseau soient aussi proches que possible des sorties désirées.

Pour commencer l'apprentissage, les poids d'interconnexion doivent être initialisés arbitrairement. L'apprentissage sera considéré comme accompli lorsque le réseau atteint un niveau de performances défini par l'utilisateur, ce niveau signifie que le réseau a achevé une précision statistique désirée.

Une fois l'apprentissage achevé, le réseau peut être utilisé pour prédire le nouvel échantillon de sortie, en se basant seulement sur la connaissance de l'échantillon d'entrée.

On peut distinguer trois procédures de base pour l'apprentissage :

L'apprentissage se divise grossièrement en trois catégories

➤ L'apprentissage supervisé.

- L'apprentissage non supervisé.
- L'apprentissage par renforcement.

Pour ces trois types d'apprentissage, il y a également un choix traditionnel entre :

- L'apprentissage Off-Line : Toutes les données sont dans une base d'exemples d'apprentissage qui sont traités simultanément.
- L'apprentissage On-Line : Les exemples sont présentés les uns après les autres au fur et à mesure de leur disponibilité.

3.8.1. Apprentissage supervisé

L'apprentissage supervisé est utilisé pour définir un modèle prédictif, c.-à-d. des modèles qui font de la prédiction. Et pour obtenir un modèle prédictif on doit au préalable l'entraîner en lui fournissant des résultats antérieurs c.-à-d. des exemples dont on connaît déjà la bonne réponse, et tout l'enjeu consiste à affiner, à ajuster un modèle de telle sorte qu'au fur et à mesure des entraînements, le modèle s'améliore et ses prédictions se rapprochent de plus en plus des bonnes réponses consignés dans les exemples d'entraînement, alors cela implique que pour chaque exemple on doit nécessairement fournir la bonne réponse associée (la donnée de sortie).

Il s'agit donc de trouver une relation entre une entrée x et une sortie y . Un superviseur donne à l'algorithme durant son entraînement un jeu de données d'entraînement qui va être des couples d'entrée $(x^{(i)}, y^{(i)})$ avec $1 \leq i \leq m$, donc le $i^{\text{ème}}$ exemple du jeu de données, sachant que m est le nombre d'exemples total qu'on donne à l'algorithme. Il est à noter que x et y peuvent représenter n'importe quoi, ça peut être des nombres, une image, un texte, etc. et donc pendant l'entraînement l'algorithme va optimiser ses paramètres internes (les poids synaptiques généralement) afin de prédire au mieux la sortie $y^{(i)}$ d'une entrée $x^{(i)}$ qui est présente dans le jeu de données (voir figure 3.20). Pour que à la fin il puisse performer sur des données qu'il n'a jamais vues.

Cette méthode permet de modifier les pondérations au sein du réseau de neurones artificiels et d'optimiser l'algorithme.

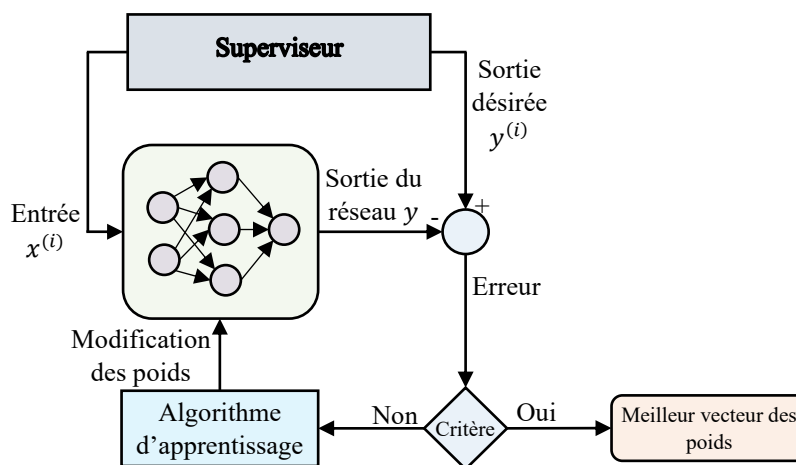


Figure 3.20. Apprentissage supervisé

Par exemple pour mettre en place un système qui prédise le prix de vente d'un véhicule, on doit obligatoirement l'entraîner avec des exemples de véhicule dont on connaît déjà le prix de vente.

De même pour apprendre à reconnaître un chiffre manuscrit dans une image, on fournit des exemples d'images de chiffre qui ont déjà été identifiés, et ensuite on peut présenter au système une image qu'il n'a jamais vue, et il sera en mesure de reconnaître le chiffre qu'elle représente, tels qu'il est montré sur la figure 3.21. Alors l'entraînement est supervisé par les bonnes réponses au même titre qu'un enseignant connaissant les bonnes réponses supervise l'apprentissage de ses étudiants.

Entrée $x^{(i)}$	Sortie $y^{(i)}$
	3
	6
	0
	9

$x =$ 

 $y = 5$

Figure 3.21. Exemple de reconnaissance de chiffres manuscrits

Avec l'apprentissage supervisé on peut faire deux tâches différentes :

3.8.1.1. La classification

Dans la classification on cherche à quelle classe appartient la donnée. Par exemple si on veut prédire si une image est une image de chien ou de chat. Justement, dans cet exemple classique de la reconnaissance d'une image (chien/chat), l'entrée x ça va être donc une image et la sortie peut valoir soit chien soit chat (voir figure 3.22).

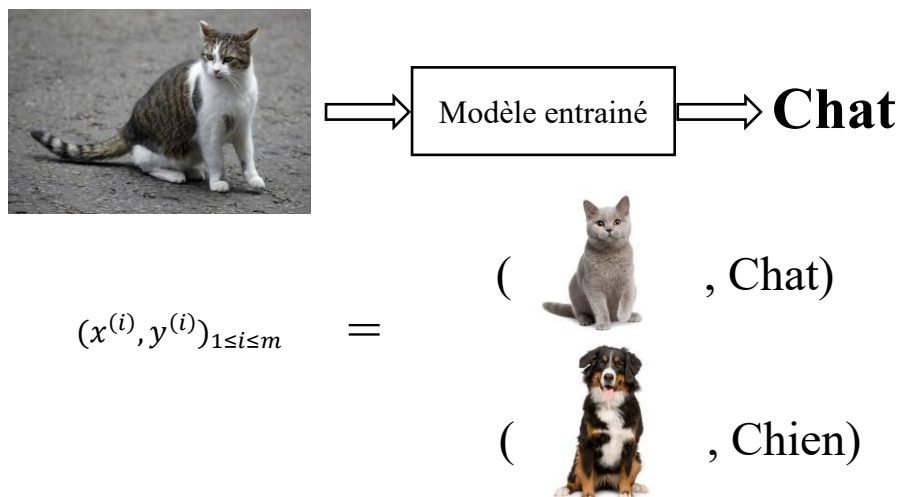


Figure 3.22. Exemple de reconnaissance de chiffres manuscrits

Dans la classification on a actuellement des applications absolument spectaculaires tels que :

- Diagnostics médicaux : détection de maladies (le cancer par exemple).
- Secteur bancaire : fidélisation de clients, analyse de risques, etc.
- En finance : détection de fraudes.
- En sécurité : détection d'intrusions malveillantes.

3.8.1.2. La régression

Avec la régression la valeur prédite n'indique pas une catégorie parmi plusieurs comme dans la classification, la valeur prédite ici est une valeur numérique (un nombre réel).

Donc, x et y dans ce cas sont des nombres, reliés par une loi linéaire. La figure 3.23 présente un jeu de données, où chaque point correspond à un couple entrée/sortie qui est présent dans le jeu de données d'entraînement. Pendant l'entraînement l'algorithme va optimiser ses paramètres internes pour que son modèle (représenté sur la figure 3.23 par une ligne verte) colle au mieux au jeu de données, c.-à-d. en fait que ses prédictions sur le jeu de données soit les meilleurs possibles. Et c'est une fois cet entraînement fait, on peut réaliser l'inférence, on demande quelle est la sortie associée à une valeur choisie (9 par exemple) et on se sert du modèle pour avoir la réponse (qui est 7.5 sur la figure 3.23).

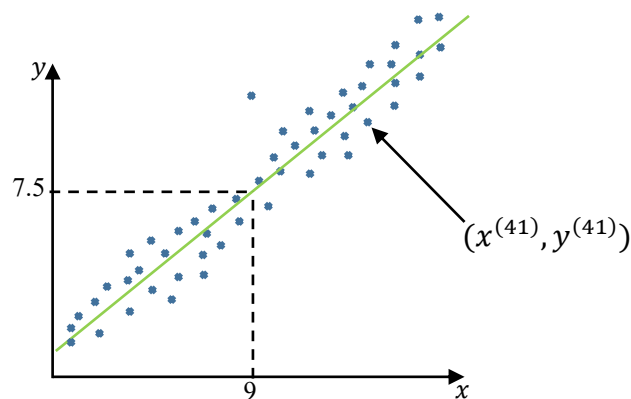


Figure 3.23. Exemple de régression linéaire.

Parmi les applications de la régression on peut citer :

- Le prix d'un produit.
- Prévion de marché ou une valeur boursière.
- Prévion du taux de croissance ou d'une dépense énergétique.
- Prédiction de popularité (pubs).

3.8.2. Apprentissage non supervisé

Dans l'apprentissage non supervisé on ne va pas prédire le prix d'un produit ou si une image correspond à un chat ou non, on va essayer de trouver les structures sous-jacentes d'un jeu de données à partir de données non labélisées. Aucune information sur la sortie désirée n'est fournie au réseau. On présente une entrée au réseau et on le laisse évoluer librement jusqu'à ce qu'il se stabilise, ce comportement est connu sous le nom « auto organisation ». Le réseau de neurones dans ce cas-là détecte automatiquement les régularités qui figurent dans les exemples présentés et

il modifie les poids des connexions pour que les exemples ayant les mêmes caractéristiques de régularité provoquent la même sortie.

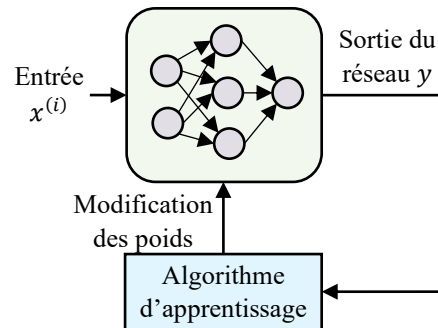


Figure 3.24. Apprentissage non supervisé

Une des applications les utilisées en apprentissage non supervisé est le **clustering** qui est une méthode d'analyse statistique utilisée pour organiser des données brutes en groupes homogènes. Bien que dans la classification et le clustering on parle de classe, les algorithmes sont fondamentalement différents et ne servent pas à faire la même chose. Dans l'apprentissage supervisé on prend les données x et on essaye de prédire des données y . Si on prend comme exemple de personnes qui ont un bilan sanguin et on va essayer de définir si ces personnes sont malades ou non. On a des personnes qui sont malades en bleu et on a des personnes qui sont saines en vert tels qu'il est représenté sur la figure 3.25, le but donc sera de trouver une fonction qui arrive à prédire la classe. Si on a de nouveaux points qui arrivent suivant de quels côtés de la fonction ils se trouvent ils seront classifiés en tant que malade ou en tant que personnes saines.

Imaginons maintenant que l'on ait toutes les données du bilan, mais on ne sait pas si ces personnes sont malades ou pas. Là on va essayer de trouver une fonction qui arrive à regrouper les personnes les plus proches, par exemple on va utiliser un algorithme de clustering pour apprendre la structure cachée de ces données. Par exemple on voit que les points noirs sur la gauche sont plus proches, donc on va essayer de faire groupe avec eux, les points noirs à droite sont plus proches donc faire groupe avec eux (voir figure 3.25). Et en fait on va essayer d'analyser les données de ces deux groupes pour comprendre en quoi ils sont différents et en quoi ces groupes forment des groupes homogènes.

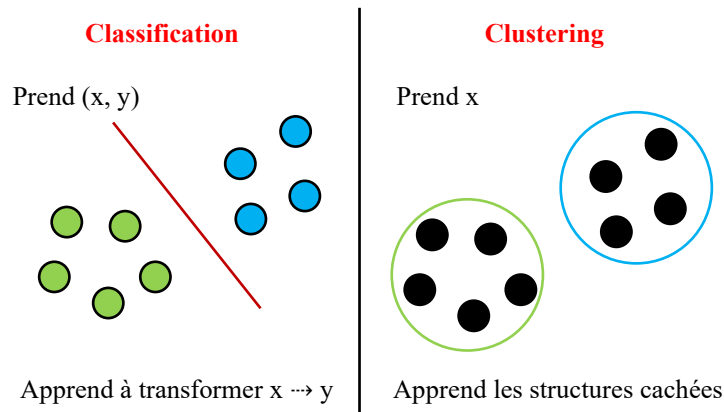


Figure 3.25. Classification vs Clustering

3.8.3. Apprentissage par renforcement

L'apprentissage par renforcement ou Reinforcement Learning est une méthode qui permet de trouver, par un processus d'essais et d'erreurs, l'action optimale à effectuer pour chacune des situations que le système va percevoir afin de maximiser une récompense. En revanche, aucune instruction, aucun indice n'est donné au système pour lui suggérer comment accomplir la tâche demandée. C'est à lui de découvrir comment maximiser sa récompense, en commençant par des tentatives totalement aléatoires pour terminer par des tactiques extrêmement sophistiquées.

Ce type d'apprentissage ressemble au principe de PAVLOV pour apprendre à faire des tours aux animaux. L'animal de compagnie par exemple ne comprend pas le langage humain, et il est donc impossible de lui apprendre des tours en lui expliquant verbalement ce qu'il doit faire. A la place, il est possible de provoquer une réaction de l'animal en créant une situation.

Si la réaction de l'animal est la bonne, on peut le récompenser. Par exemple, un chien qui donne la patte ou fait le beau peut recevoir un biscuit en récompense.

Grâce aux récompenses, l'animal de compagnie sera progressivement conditionné. Dès lors qu'il se retrouvera confronté à la même situation, il pourra exécuter la même action lui permettant de recevoir un biscuit.

Une solution utilisée couramment pour mettre en œuvre un apprentissage par renforcement est de considérer un agent au sein d'un environnement et qui doit prendre des décisions en fonction de sa situation actuelle (état) dans l'environnement.

L'agent peut interagir avec l'environnement par le biais d'actions et l'environnement lui rend une récompense (positive, négative ou nulle) en fonction de celles-ci. Plus formellement, l'apprentissage par renforcement fait référence à une classe de problèmes d'apprentissage automatique, dont le but est d'apprendre, à partir d'expériences, ce qu'il convient de faire en différentes situations, de façon à optimiser une récompense numérique au cours du temps.

L'agent va donc chercher à maximiser les récompenses obtenues en effectuant différentes expériences et types d'expériences (exploration, suivi d'une trajectoire, etc.) au sein de l'environnement. La nature et le choix des expériences à effectuer à un moment précis de l'apprentissage varient en fonction de l'algorithme d'apprentissage choisi.

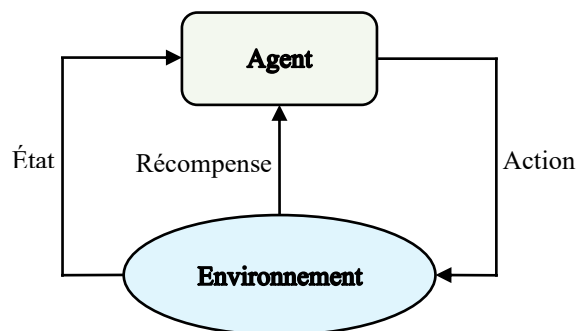


Figure 3.26. Apprentissage par renforcement

En guise d'exemple d'utilisation du l'apprentissage par renforcement, on peut citer :

- Les véhicules autonomes.
- L'automatisation industrielle : les robots par exemple peuvent apprendre par renforcement.
- La finance et le trading : L'apprentissage par renforcement permet à un agent de prendre la décision de vendre, d'acheter ou de garder les actions en fonction du prix.
- Le traitement naturel du langage.
- La médecine : les systèmes entraînés peuvent apprendre à dispenser les traitements optimaux, ou pour les maladies chroniques permet d'avoir une meilleure anticipation des effets secondaires sur le long terme.
- Les jeux, etc.

3.9. Algorithmes d'apprentissage

Dans la plupart des architectures des réseaux de neurones artificiels, l'apprentissage se traduit par une modification de l'efficacité synaptique, c.-à-d. par un changement dans la valeur des poids reliant les neurones d'une couche à une autre. Un ensemble de règles bien définies permettant de réaliser un tel processus d'adaptation des poids constitue ce qu'on appelle l'algorithme d'apprentissage du réseau. Par ailleurs, le choix d'un algorithme d'apprentissage au détriment d'un autre dépend en grande partie de la topologie du réseau de neurones utilisé. Dans ce qui suit nous allons passer en revue les principales méthodes d'apprentissages les plus couramment utilisées pour l'adaptation des poids.

3.9.1. La règle de Hebb

Cette règle s'inspire des travaux du neurophysiologiste Donald Hebb et qui a été proposée en 1949. Hebb cherchait dans un contexte neurobiologique à établir une forme d'apprentissage associatif au niveau cellulaire. Dans le contexte des réseaux artificiels, on peut reformuler l'énoncé de Hebb sous la forme d'une règle d'apprentissage en deux parties :

1. Si deux neurones de part et d'autre d'une synapse (connexion) sont activés simultanément (d'une manière synchrone), alors la force de cette synapse doit être augmentée ;
2. Si les mêmes deux neurones sont activés d'une manière asynchrone, alors la synapse correspondante doit être affaiblie ou carrément éliminée.

La règle de Hebb peut être exprimée mathématiquement comme suit :

$$w_{ij}(t + 1) = w_{ij}(t) + \eta x_i(t) y_j(t) \quad (3.9)$$

Où

- $w_{ij}(t)$ est le poids synaptique reliant l'entrée $x_i(t)$ au neurone j à l'étape t , sachant que $y_j(t)$ la sortie du neurone j .

- $w_{ij}(t + 1)$ est le nouveau poids à l'étape $t+1$.

- η est le coefficient d'apprentissage (compris entre 0 et 1).

Les poids sont modifiés d'une façon itérative par l'algorithme d'apprentissage afin d'adapter la réponse obtenue à la réponse désirée. La modification des poids n'intervient que s'il y a erreur.

Algorithme 3.1 : Algorithme d'apprentissage par la règle de Hebb

Début

- Initialisation des poids et du seuil d'une façon aléatoire (de préférence on choisit de petites valeurs)

Répéter

- présentation d'une entrée $x = (x_1, \dots, x_n)$ de la base d'apprentissage

- calcul de la sortie obtenue y pour cette entrée présentée.

$a = \sum(w_i x_i) - s$ (s est la valeur du seuil)

$y = \text{sign}(a)$

- Si la sortie y est différente de la sortie désirée d pour cet exemple d'entrée x alors modification des poids :

$w_{ij}(t + 1) = w_{ij}(t) + \eta x_i(t) y_j(t)$

Jusqu'à tous les exemples de la base d'apprentissage sont traités correctement

Fin

3.9.2. La règle du perceptron

La règle de Hebb ne s'applique pas dans certain cas, bien qu'une solution existe. Un autre algorithme d'apprentissage a donc été proposé, qui tient compte de l'erreur observée en sortie. Dans le neurone du perceptron on utilise la fonction d'activation à seuil, ce qui permet de classer les vecteurs d'entrée dans un hyperplan. La règle d'adaptation permet de modifier la position de l'hyperplan séparateur dont l'équation dans l'espace d'entrée est définie par les poids du neurone, afin de réduire l'erreur ($d(t) - y(t)$). Ainsi, à chaque présentation d'un couple (x, y) mal-classé (c'est-à-dire dont la sortie proposée $y(t)$ par le neurone ne correspond pas à la sortie désirée $d(t)$), on modifie le vecteur poids de manière à ramener le point mal classé du bon côté de l'hyperplan (voir Figure 3.27). Si le point est bien classé, on ne fait rien.

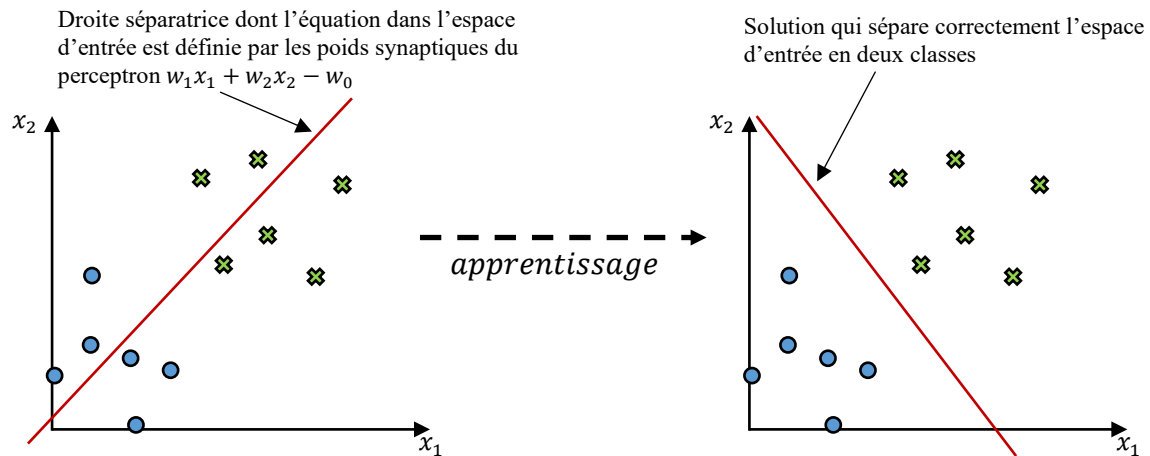


Figure 3.27. Principe d'apprentissage

L'algorithme d'apprentissage du Perceptron est semblable à celui utilisé pour la loi de Hebb. Les différences se situent au niveau de la modification des poids. La règle d'apprentissage du Perceptron est la suivante :

$$w_{ij}(t + 1) = w_{ij}(t) + \eta (d_j(t) - y_j(t))x_i(t) \quad (3.10)$$

Avec :

- $w_{ij}(t)$: poids de la connexion entre les neurones i et j à l'instant t .
- η : pas d'apprentissage. Situé entre 0 et 1, on fait souvent varier ce coefficient au cours de l'apprentissage.
- d_j : sortie désirée pour l'entrée courante.
- y_j : sortie obtenue pour l'entrée courante.
- x_i : signal transmis par le neurone i .

L'inconvénient majeur du Perceptron est qu'il ne s'applique que si les classes sont linéairement séparables. La structure de l'algorithme du perceptron est donnée comme suit :

Algorithme 3.2 : Algorithme d'apprentissage du perceptron

Début

- Initialiser aléatoirement les poids w_{ij}

Répéter

- Présentation d'une entrée $x = (x_1, \dots, x_n)$ de la base d'apprentissage.

Calcul de la sortie obtenue $y = (y_1, \dots, y_m)$ pour cette entrée :

Pour chaque y_j Faire

$$a_j = \sum(w_{ij}x_i) - s$$

$$y_j = f(a_j) \text{ (ici } f \text{ est la fonction Heaviside)}$$

Fin pour

- Si la sortie y du Perceptron est différente de la sortie désirée d pour cet exemple

d'entrée x alors modification des poids :

$$w_{ij}(t+1) = w_{ij}(t) + \eta (d_j(t) - y_j(t))x_i(t)$$

Jusqu'à tous les exemples de la base d'apprentissage sont traités correctement

Fin

3.9.2.1. Exemple d'apprentissage par l'algorithme du Perceptron

Voyons maintenant un exemple de fonctionnement de l'algorithme d'apprentissage du Perceptron.

Pour mieux comprendre le déroulement du processus de l'apprentissage, nous allons voir un exemple d'apprentissage de la fonction « **ET** ». La base de l'exemple d'apprentissage du perceptron qui est représenté dans la figure 3.28, est donc donnée par le tableau ci-dessous :

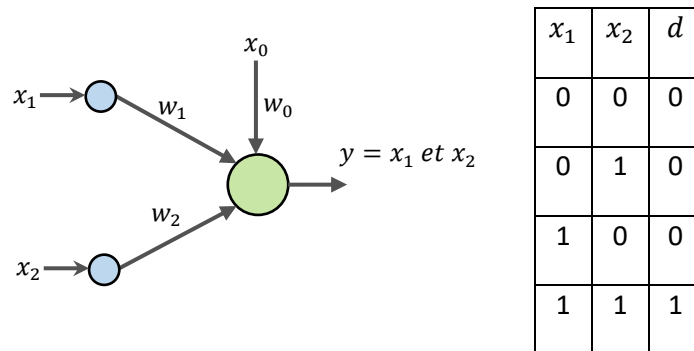


Figure 3.28. Apprentissage de la fonction ET

Nous commençons par initialiser les poids du perceptron avec des valeurs aléatoires. Nous avons donc deux poids w_1 et w_2 , un pour chaque entrée, et un poids supplémentaire w_0 qui relie le biais x_0 (vaut toujours 1) au neurone : $w_1 = 0.2$, $w_2 = 0.3$ et $w_0 = -0.1$.

Supposons un pas d'apprentissage de 0.1.

La fonction d'activation utilisée dans ce cas est la fonction Heaviside (avec un seuil fixé à 0) représentée sur la figure 3.29.

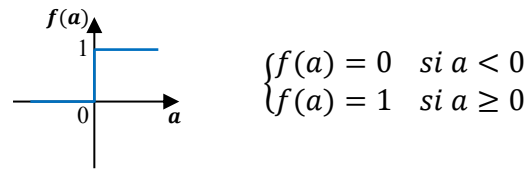


Figure 3.29. La fonction d'activation

Au cours de l'apprentissage on présente successivement l'ensemble des entrées possible, et on applique la règle d'apprentissage (c'est ce que l'on appelle une « itération » ou une « époque ») ceci jusqu'à ce que les résultats soient considérés comme acceptables. Il existe dans ce cas $2^2 = 4$ configurations d'entrée possibles que l'on va présenter une première fois au perceptron (tableau 3.3). Lorsque la sortie obtenue y est égale à la sortie désirée d , les poids ne seront pas modifiés.

Époque	Entrée $x_1 \ x_2 \ x_0$	$a = \sum (w_i x_i)$	$y = f(a)$	d	w_1	w_2	w_0
1.1	0 0 1	$0.2 \times 0 + 0.3 \times 0 + (-0.1) \times 1$	0	0	0.2	0.3	-0.1
1.2	0 1 1	$0.2 \times 0 + 0.3 \times 1 + (-0.1) \times 1$	1	0	0.2	0.2	-0.2
1.3	1 0 1	$0.2 \times 1 + 0.2 \times 0 + (-0.2) \times 1$	1	0	0.1	0.2	-0.3
1.4	1 1 1	$0.1 \times 1 + 0.2 \times 1 + (-0.3) \times 1$	1	1	0.1	0.2	-0.3
2.1	0 0 1	$0.1 \times 0 + 0.2 \times 0 + (-0.3) \times 1$	0	0	0.1	0.2	-0.3
2.2	0 1 1	$0.1 \times 0 + 0.2 \times 1 + (-0.3) \times 1$	0	0	0.1	0.2	-0.3

Tableau 3.3. Les phases d'apprentissage

A l'exemple 1.2 (0,1,1), le neurone reçoit le signal suivant :

$$a = \sum_{i=1}^2 (w_i x_i) + w_0 x_0 = 0.2 \times 0 + 0.3 \times 0 + (-0.1) \times 1 = 0.2$$

Le seuil étant fixé à 0, on a $y = 1$, ce qui est contraire à la sortie désirée d qui est égale à 0. On applique alors la règle d'apprentissage :

$$w_1 \leftarrow w_1 + \eta (d - y) x_1 = 0.2 + 0.1 \times (0 - 1) \times 0 = 0.2$$

$$w_2 \leftarrow w_2 + \eta (d - y)x_2 = 0.3 + 0.1 \times (0 - 1) \times 1 = 0.2$$

$$w_0 \leftarrow w_0 + \eta (d - y)x_0 = -0.1 + 0.1 \times (0 - 1) \times 1 = -0.2$$

A l'exemple 1.3 avec les nouveaux poids, la sortie y ne correspond pas à la sortie désirée d , on applique donc encore une fois la règle d'apprentissage :

$$w_1 \leftarrow w_1 + \eta (d - y)x_1 = 0.2 + 0.1 \times (0 - 1) \times 1 = 0.1$$

$$w_2 \leftarrow w_2 + \eta (d - y)x_2 = 0.2 + 0.1 \times (0 - 1) \times 0 = 0.2$$

$$w_0 \leftarrow w_0 + \eta (d - y)x_0 = -0.2 + 0.1 \times (0 - 1) \times 1 = -0.3$$

On remarque que tous les exemples de la base ont été traités correctement, l'apprentissage est achevé.

Donc : $w_1 = 0.1$, $w_2 = 0.2$ et $w_0 = -0.3$.

Ce perceptron calcule le ET logique pour tout couple (x_1, x_2) .

3.9.3. La règle de Widrow-Hoff (la règle delta)

Le perceptron étant limité à des sorties de type binaire comme nous l'avons vu dans la partie antérieure, Widrow et Hoff ont pris en considération des fonctions d'activation continues et différentielles en étudiant l'algorithme d'apprentissage du perceptron.

Cette loi est aussi une version modifiée de la loi de Hebb. Les poids des liens entre les neurones sont continuellement modifiés de façon à réduire la différence (le delta) entre la sortie désirée et la valeur calculée de la sortie du neurone. Les poids sont modifiés de façon à minimiser l'erreur quadratique à la sortie du RNA. L'erreur est alors propagée des neurones de sortie vers les neurones des couches inférieures, dans la règle de Widrow-Hoff appelé aussi règle de Delta, l'apprentissage est réalisé par itération. Le principe est le suivant :

➤ Calculer l'erreur quadratique selon la formule :

$$e = \sum_{j=1}^n (d_j - y_j)^2 \quad (3.11)$$

➤ Minimiser cette erreur en modifiant les poids de chaque neurone suivant la règle :

$$w_{ij}(t+1) = w_{ij}(t) + \eta \delta_j(t) x_i(t) \quad (3.12)$$

Où :

- d_j : est la sortie désirée.

- y_j : est la sortie calculée.

- η : est le pas d'apprentissage.

$\delta_j(t) = (d_j(t) - y_j(t))$: est l'erreur faite par le neurone.

- x_i : est l'entrée i du neurone j .

Cette règle de correction permet donc aux neurones d'adapter leurs poids pour se rapprocher à une valeur désirée correspondante à chaque exemple présenté. Cette règle a été utilisée pour l'apprentissage de l'ADALINE.

3.9.4. Apprentissage compétitif

C'est un apprentissage non supervisé qui consiste à faire une compétition entre les neurones d'un réseau pour déterminer lequel sera actif à un instant donné. Contrairement aux autres types d'apprentissage, où la mise à jour des poids de tous les neurones s'effectue simultanément, ce mode d'apprentissage considère à chaque fois un neurone vainqueur, et parfois un ensemble de voisins du vainqueur, et seuls les poids de ces neurones seront adaptés.

Les poids d'un neurone vainqueur seront modifiés de telle sorte qu'ils se rapprochent de l'exemple X présenté en entrée et pour lequel ce neurone a gagné la compétition contre tous les autres neurones. En revanche, les poids d'un neurone qui ne gagne aucune compétition ne seront pas modifiés. La règle d'apprentissage est donnée par :

$$\Delta w_i = \begin{cases} \eta_1 (X - w_i) & \text{si le neurone est vainqueur} \\ \eta_2 (X - w_i) & \text{si le neurone est voisin du vainqueur} \\ 0 & \text{Autrement} \end{cases}$$

Avec η_1 et η_2 représentent les taux d'apprentissage et $\eta_1 < \eta_2$. Cette incrémentation reflète la distance entre la valeur courante du poids et la valeur d'entrée.

Ce type d'apprentissage caractérise une classe de réseaux de neurones tels que le réseau de Kohonen, où la décision est prise en déterminant quel neurone représente le mieux l'exemple d'entrée.

3.9.4. Algorithme de la retro-propagation du gradient d'erreur (Back-propagation)

La règle d'apprentissage de Rétro-propagation du gradient ou encore appelée La règle delta généralisée est une généralisation de la règle de Widrow-Hoff qui s'applique à un **réseau multicouche** utilisant des fonctions d'activation différentiables et un apprentissage supervisé. cet algorithme a été proposé indépendamment par Rumelhart, Le Cun et Hilton en 1984.

Les différentes étapes à suivre lors de l'apprentissage d'un réseau de neurones à propagation avant avec l'algorithme « Back propagation » sont données par l'algorithme suivant :

Algorithme 3.3 : Algorithme d'apprentissage de Rétro-propagation du gradient (Back-propagation)

Début

- Initialiser les poids des liens entre les neurones. Souvent une valeur entre 0 et 1, déterminée aléatoirement, est assignée à chacun des poids

Répéter

- Application d'un vecteur entrées-sorties à apprendre
- Calculer par **propagation** les valeurs de sortie de tous les neurones de la couche d'entrée vers la couche de sortie.

$y_j(t) = \sigma(\sum_{i=0}^n (w_{ij}(t)y_i(t)))$ (Où $x_0 = -1$ et w_{0j} est le seuil du neurone j , et y_i correspond à l'entrée x_i du réseau connectée au neurone j ou bien à la sortie du neurone i de la couche précédente)

- Calculer par **rétro-propagation** l'erreur des neurones de la couche de sortie vers la couche d'entrée :

$\delta_j(t) = (d_j(t) - y_j(t)) \sigma'(\sum_{i=0}^n (w_{ij}(t)y_i(t)))$ (Pour tous les neurones j de la couche de sortie reliés à n neurones i de la couche cachée précédente)

$\delta_j(t) = \sum_{k=1}^m (w_{jk}(t)\delta_k(t)) \sigma'(\sum_{i=0}^n (w_{ij}(t)y_i(t)))$ (Pour tous les neurones j d'une couche cachée reliés à m neurones k de la couche suivante, et n neurones i de la couche précédente où y_i correspond à l'entrée x_i du réseau connectée au neurone j ou bien à la sortie du neurone i de la couche précédente)

- Modifier tous les poids du réseau en appliquant la règle du Delta généralisé :

$w_{ij}(t+1) = w_{ij}(t) + \eta \delta_j(t) y_i(t)$ (Où y_i correspond à l'entrée x_i du réseau connectée au neurone j ou bien à la sortie du neurone i de la couche précédente)

Jusqu'à ce que toutes les performances du RNA (erreur sur les sorties) sont satisfaisantes

Fin

Pour mieux comprendre le fonctionnement de cet algorithme nous allons prendre un exemple d'un réseau à deux entrées, trois neurones dans une couche cachée, et deux neurones dans la couche de sortie (figure 3.30) :

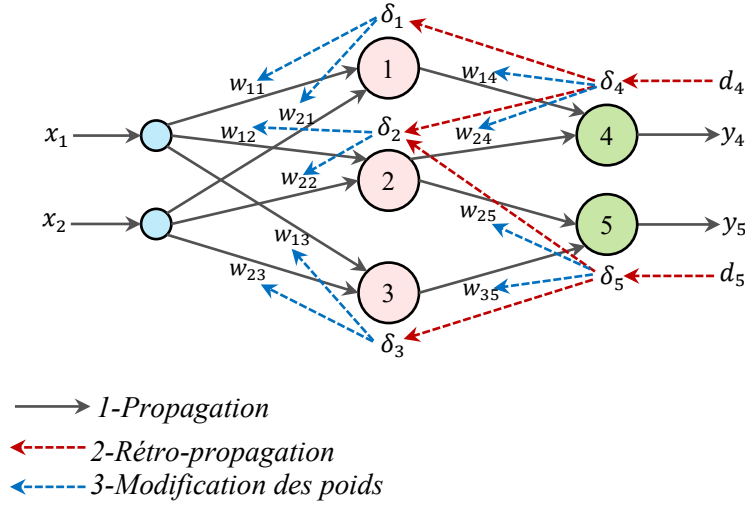


Figure 3.30. Algorithme de Rétro propagation

Pour pouvoir modifier les poids synaptiques reliant la couche d'entrée à la couche cachée (w_{11} , w_{12} , w_{13} et w_{21} , w_{22} , w_{23}), il faut connaître les sorties désirées d_1 , d_2 , d_3 qui permettent d'appliquer la règle du delta généralisé, i.e. connaître les erreurs δ_1 , δ_2 , δ_3 que font les neurones 1, 2 et 3.

L'idée consiste alors à propager les erreurs δ_4 et δ_5 vers les neurones 1, 2 et 3, au travers des poids w_{14} , w_{24} , w_{25} et w_{35} , d'où le nom de rétro-propagation du gradient d'erreur de l'algorithme.

1^{ère} étape : on calcule y_1 , y_2 , y_3 (on n'oublie pas le seuil w_0 qui n'est jamais représenté):

$$\begin{cases} y_1 = \sigma(w_{11}x_1 + w_{21}x_2 - w_{01}) \\ y_2 = \sigma(w_{12}x_1 + w_{22}x_2 - w_{02}) \\ y_3 = \sigma(w_{13}x_1 + w_{23}x_2 - w_{03}) \end{cases} \quad (3.13)$$

Puis y_4 et y_5 :

$$\begin{cases} y_4 = \sigma(w_{14}y_1 + w_{24}y_2 - w_{04}) \\ y_5 = \sigma(w_{25}y_2 + w_{35}y_3 - w_{05}) \end{cases} \quad (3.14)$$

2^{ème} étape : on calcule les erreurs de la couche de sortie :

$$\begin{cases} \delta_4 = (d_4 - y_4) \sigma'(w_{14}y_1 + w_{24}y_2 - w_{04}) \\ \delta_5 = (d_5 - y_5) \sigma'(w_{25}y_2 + w_{35}y_3 - w_{05}) \end{cases} \quad (3.15)$$

Et les erreurs de la couche cachée :

$$\begin{cases} \delta_1 = w_{14} \delta_4 \cdot \sigma'(w_{11}x_1 + w_{21}x_2 - w_{01}) \\ \delta_2 = (w_{24} \delta_4 + w_{25} \delta_5) \cdot \sigma'(w_{12}x_1 + w_{22}x_2 - w_{02}) \\ \delta_3 = w_{35} \delta_5 \cdot \sigma'(w_{13}x_1 + w_{23}x_2 - w_{03}) \end{cases} \quad (3.16)$$

3^{ème} étape : on calcule la nouvelle valeur de chaque poids entre la couche d'entrée et la couche cachée :

$$\begin{cases} w_{11}(t+1) = w_{11}(t) + \eta \delta_1(t) x_1 \\ w_{21}(t+1) = w_{21}(t) + \eta \delta_1(t) x_2 \\ w_{12}(t+1) = w_{12}(t) + \eta \delta_2(t) x_1 \\ w_{22}(t+1) = w_{22}(t) + \eta \delta_2(t) x_2 \\ w_{13}(t+1) = w_{13}(t) + \eta \delta_3(t) x_1 \\ w_{23}(t+1) = w_{23}(t) + \eta \delta_3(t) x_2 \end{cases} \quad (3.17)$$

Et entre la couche cachée et la couche de sortie :

$$\begin{cases} w_{14}(t+1) = w_{14}(t) + \eta \delta_4(t) y_1 \\ w_{24}(t+1) = w_{24}(t) + \eta \delta_4(t) y_2 \\ w_{25}(t+1) = w_{25}(t) + \eta \delta_5(t) y_2 \\ w_{35}(t+1) = w_{35}(t) + \eta \delta_5(t) y_3 \end{cases} \quad (3.18)$$

On vient de réaliser un pas d'apprentissage. Il faut recommencer ces opérations pour tous les vecteurs d'apprentissage, puis tester la qualité de l'apprentissage avec les vecteurs de test qui n'ont pas servi à l'apprentissage : ce qui permet de tester les capacités de la généralisation du réseau.

L'algorithme de rétro-propagation du gradient consiste à effectuer une descente de gradient sur la fonction de coût déjà utilisée pour le neurone seul :

$$\varepsilon(\vec{w}, k) = \frac{1}{2} (d(k) - y(k))^2 \quad (3.19)$$

Où y est la sortie du réseau (donc une somme pondérée de sigmoïdes) et non la sortie d'un neurone seul.

En dérivant cette expression par rapport à chaque poids w , on retrouve la règle d'apprentissage donnée dans l'algorithme.

3.10. Mise en œuvre du réseau de neurone multicouche

La mise en œuvre des réseaux de neurones multicouches comporte à la fois une partie de conception, dont l'objectif est de permettre de choisir la meilleure architecture possible, et une partie de calcul numérique, pour réaliser l'apprentissage d'un réseau de neurones. On peut diviser la procédure en quatre étapes :

Étape 01 : Fixer le nombre de couches cachées

Mis à part les couches d'entrée et de sortie, l'analyse doit décider du nombre de couches intermédiaires ou cachées. Sans couche cachée, le réseau n'offre que de faibles possibilités d'adaptation, avec une couche cachée, il est capable, avec un nombre suffisant de neurones, d'approximer toute fonction continue. Une seconde couche cachée prend en compte les discontinuités éventuelles.

Etape 02 : Déterminer le nombre de neurones par couches cachées

Chaque neurone supplémentaire permet de prendre en compte des profils spécifiques des neurones d'entrée. Un nombre plus important permet donc de mieux coller aux données présentées mais diminue la capacité de généralisation du réseau.

Etape 03 : Choisir la fonction d'activation

Le choix de la fonction d'activation dans un réseau de neurones artificiels multicouche (MLP) est un facteur clé qui peut affecter la performance et l'efficacité du modèle. Le choix de la fonction d'activation dépend des caractéristiques spécifiques du problème à résoudre et des données disponibles. Il est aussi courant d'expérimenter plusieurs fonctions d'activation pour déterminer celle qui fonctionne le mieux pour un cas d'usage particulier.

Etape 04 : choisir l'apprentissage

A partir d'une architecture de réseau de neurones donnée et des exemples disponible (la base d'apprentissage), on détermine les poids optimaux, par l'algorithme de la rétropropagation des erreurs, pour que la sortie du modèle s'approche le plus possible au fonctionnement désiré.

Dans le cas général, un réseau de neurones MLP peut posséder un nombre de couches quelconque et un nombre de neurones par couche également quelconque, mais en vue de perfectionner le fonctionnement du réseau et de minimiser au maximum le temps de calcul, on doit chercher une architecture optimale au point de vue nombre de couche et nombre de neurones par couche.

3.11. Domaines d'application des Réseaux de Neurones

Les réseaux de neurones sont aujourd'hui utilisés dans une multitude de domaines pour diverses applications, notamment :

A. Modélisation de données statiques

Les réseaux de neurones sont particulièrement efficaces pour modéliser une large gamme de phénomènes statiques, où une relation déterministe existe entre des causes et des effets. Ils peuvent être utilisés pour approximer ces relations à partir de données expérimentales, à condition que ces dernières soient suffisamment abondantes et représentatives du phénomène à modéliser.

B. Modélisation de processus dynamiques non linéaires

Dans ce cas, le but est de déterminer un ensemble d'équations mathématiques qui décrivent le comportement dynamique d'un processus, en détaillant comment ses sorties évoluent en fonction de ses entrées. Les réseaux de neurones sont capables de résoudre ce type de problème lorsqu'il s'agit de phénomènes non linéaires, offrant ainsi une approche flexible et robuste pour la modélisation dynamique.

C. Identification et commande de processus

L'identification et la commande de systèmes complexes, y compris dans le domaine du control des machines électriques, sont des secteurs où les réseaux de neurones jouent un rôle de plus en plus important. On les utilise par exemple pour aider au pilotage de réacteurs chimiques ou de colonnes à distillation, pour contrôler une voiture autonome suivant une trajectoire donnée sur un terrain accidenté, dans la robotique, ou encore pour réguler le niveau de la lingotière dans une aciérie à coulée continue. Les applications de ce type se multiplient. En général, elles ne nécessitent que des ressources relativement modestes, car les temps de réponse des systèmes sont souvent assez longs. Une simple simulation sur micro-ordinateur suffit fréquemment en phase d'exploitation. L'un des principaux atouts des réseaux de neurones dans ce domaine est la rapidité de leur mise en œuvre initiale, notamment lors de l'identification par apprentissage. Il convient également de noter que les techniques de commande basées sur la logique floue peuvent utiliser les réseaux de neurones pour l'apprentissage des règles.

D. La classification

La classification est un domaine clé dans lequel les réseaux de neurones sont largement utilisés pour attribuer automatiquement un objet à une classe parmi plusieurs possibles. Grâce à leur capacité à approximer des fonctions complexes de manière précise, les réseaux de neurones peuvent estimer la probabilité d'appartenance d'un objet inconnu à une classe donnée. Cette approche est appliquée dans divers secteurs, comme le diagnostic de pannes, le contrôle de qualité, l'analyse du signal, l'élimination du bruit, ou encore la reconnaissance de formes.

Ces applications montrent la polyvalence et l'efficacité des réseaux de neurones dans des problèmes complexes de modélisation, de commande et de classification.

3.12. Conclusion

En conclusion, les réseaux de neurones artificiels représentent une avancée majeure dans le domaine de l'intelligence artificielle et du Machine Learning. Leur capacité à modéliser des relations complexes et non linéaires leur permet de résoudre une large gamme de problèmes, allant de la classification et la régression à la commande de processus dynamiques et la reconnaissance de formes. Grâce à leur architecture flexible et à leur aptitude à apprendre à partir de données, ils sont devenus incontournables dans des domaines variés comme la robotique, la finance, la médecine, et l'industrie. Leur grande capacité d'adaptation, ainsi que leur pouvoir d'approximation universelle, font des réseaux de neurones des outils puissants pour résoudre des problématiques complexes. Toutefois, leur efficacité dépend de la qualité des données, de l'architecture choisie et du temps d'entraînement. Bien que des défis subsistent, tels que l'interprétabilité des modèles ou le besoin en ressources computationnelles, les réseaux de neurones continuent de se développer, ouvrant de nouvelles perspectives pour l'innovation technologique.

4.1. Introduction

Les réseaux de neurones et la logique floue, deux approches intelligentes distinctes, possèdent chacune des avantages uniques ainsi que des limitations. Les réseaux de neurones sont particulièrement efficaces pour l'apprentissage à partir de données, mais leur incapacité à expliquer les décisions prises reste un défi majeur. De son côté, la logique floue excelle dans la gestion d'informations imprécises et l'explication des décisions, mais elle ne permet pas l'acquisition automatique de règles. Cette complémentarité a conduit à l'émergence des systèmes neuro-flous, qui combinent les forces de ces deux approches. L'objectif est de surmonter leurs inconvénients respectifs en tirant parti des capacités d'apprentissage des réseaux de neurones et de la lisibilité ainsi que de la flexibilité des systèmes flous.

Depuis la fin des années 80, de nombreuses recherches ont été consacrées au développement de systèmes hybrides qui intègrent ces deux technologies, aboutissant à des solutions particulièrement performantes dans des domaines tels que la commande de systèmes complexes et la classification de données. Ces systèmes permettent de bénéficier des atouts de chaque technique, notamment l'apprentissage des réseaux de neurones et la capacité de raisonnement des systèmes flous. Dans ce contexte, plusieurs modèles et approches d'hybridation ont vu le jour, proposant différentes manières de combiner ces deux méthodes pour résoudre des problèmes pratiques.

Ce chapitre se propose de définir les systèmes neuro-flous, de présenter les différentes catégories de leur intégration (coopérative, concurrente et hybride) ainsi que leurs principes de fonctionnement. Nous explorerons également leurs avantages et inconvénients, en soulignant leurs applications dans la commande de systèmes complexes et dans d'autres domaines.

4.2. Définition des réseaux neuro-flous

Les systèmes neuro-flous sont des systèmes flous qui utilisent un algorithme d'apprentissage inspiré des réseaux de neurones. Ce processus d'apprentissage se base sur des informations locales et entraîne uniquement des modifications locales au sein du système flou initial. Les règles floues intégrées dans ces systèmes représentent des échantillons imprécis, pouvant être considérées comme des prototypes approximatifs des données d'apprentissage. Cependant, un système neuro-flou ne doit pas être confondu avec un système expert flou, car il ne s'appuie pas sur la logique floue au sens strict. En outre, ces systèmes peuvent servir d'approximateurs universels. La figure 4.1 illustre le principe du système neuro-flou qui représente l'intersection entre les réseaux de neurones et la logique floue. Cette combinaison permet de résoudre efficacement des problèmes

de commande, de classification, et de modélisation dans des domaines variés, tels que l'automatisation industrielle, la robotique, et la gestion de systèmes complexes.

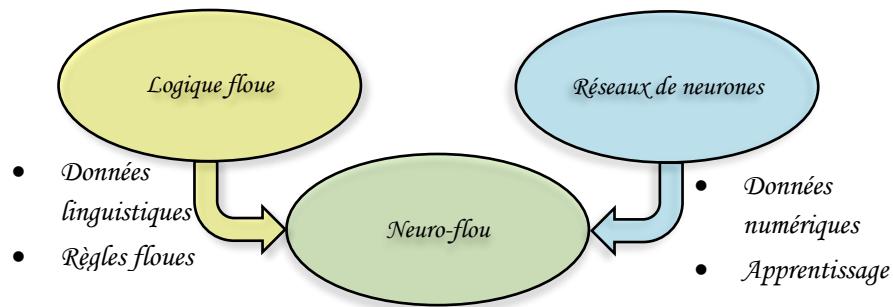


Figure 4.1. Principe du système neuro-flou

Les systèmes neuro-flous sont conçus pour exploiter les avantages et compenser les inconvénients des réseaux de neurones artificiels et des systèmes flous, en combinant leurs points forts respectifs. Le Tableau 4.1 regroupe un rappel sur les avantages et les inconvénients de la logique floue et des réseaux de neurones artificiels.

	Logique floue	Réseaux de neurones artificiels
AVANTAGES	➤ La connaissance antérieure sur les règles peut être utilisée. ➤ Le modèle mathématique non requis. ➤ Une interprétation et implémentation simple.	➤ Aucune connaissance basée sur les règles. ➤ Le modèle mathématique non requis. ➤ Plusieurs algorithmes d'apprentissage sont disponibles.
INCONVÉNIENTS	➤ Ne peut pas apprendre. ➤ Les règles doivent être disponibles. ➤ Adaptation difficile au changement de l'environnement. ➤ Aucune méthodes formelles pour l'ajustement.	➤ Boite noire (manque de traçabilité). ➤ L'adaptation aux environnements différents est difficile et le réapprentissage est souvent obligatoire. ➤ La connaissance antérieure ne peut pas être employée (apprentissage à partir de zéro). ➤ Aucune garantie sur la convergence de l'apprentissage.

Tableau 4.1. Avantages et inconvénients de la logique floue et des réseaux de neurones artificiels.

4.3. Techniques de combinaison neuro-floues

Diverses associations de ces deux approches ont été développées depuis la fin des années 80 et sont le plus souvent orientées vers la commande de systèmes complexe et les problèmes de classification. Plusieurs classifications des systèmes neuro-flous peuvent être trouvées dans la littérature, parmi elles on peut citer celle basée sur leur architecture, leur méthode d'apprentissage

ou même leur application. Dans la plupart des littératures récentes, la taxonomie de Neuro-floue suivante est adoptée :

4.3.1. Réseaux flous neuronaux

Un réseau flou neuronal est une architecture qui permet d'améliorer l'efficacité du traitement des données imprécises ou incertaines. Contrairement aux systèmes neuro-flous classiques où les deux techniques sont étroitement fusionnées, ce modèle utilise principalement des mécanismes flous pour optimiser l'apprentissage du réseau neuronal ou pour prétraiter des entrées floues. Par exemple, des fonctions d'appartenance floues peuvent être intégrées dans les couches d'entrée du réseau, permettant une meilleure gestion des données qualitatives ou linguistiques. Cette approche améliore la flexibilité et l'interprétabilité du système tout en conservant la puissance de modélisation des RNA.

4.3.2. Systèmes neuro-flous coopératifs

Le réseau de neurone est généralement employé pour déterminer les paramètres (les règles et les ensembles flous) d'un système flou (figure 4.2). Après la phase d'apprentissage, le système flou peut fonctionner sans le réseau de neurone.

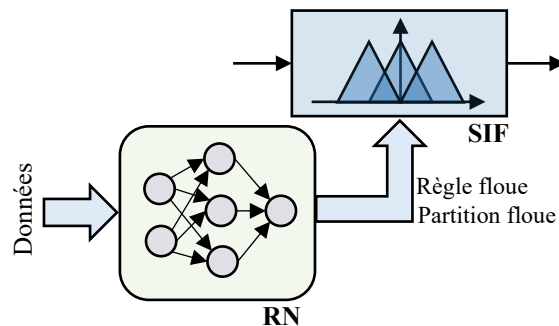


Figure 4.2. Système neuro-flou coopératif

Plusieurs possibilités d'associations des réseaux de neurones et des systèmes flous sont ainsi possibles selon des configurations en série ou en parallèle :

❖ Configuration en série

Dans cette configuration (voir figure 4.3), le réseau de neurones agit en amont ou en aval du système flou. Lorsqu'il fonctionne en amont, le RNA est chargé de traiter les données d'entrée, notamment lorsqu'elles sont complexes ou non directement mesurables, pour en extraire des caractéristiques pertinentes. Ces informations prétraitées servent ensuite d'entrées au système flou, facilitant ainsi la prise de décision floue. Inversement, un RNA placé en aval peut ajuster les sorties du SIF en fonction de nouvelles données ou connaissances, améliorant ainsi la précision et l'adaptabilité du système.

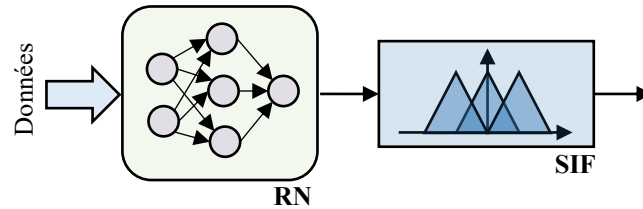


Figure 4.3. Système neuro-flou en série avec prétraitement neuronal

❖ Configuration en parallèle

Dans cette approche, les RNA et les SIF fonctionnent simultanément et de manière indépendante, traitant les informations en parallèle (voir figure 4.4). Leurs résultats peuvent ensuite être fusionnés ou comparés pour obtenir une décision finale plus robuste. Cette configuration est particulièrement utile lorsque les deux systèmes apportent des perspectives complémentaires sur les données traitées.

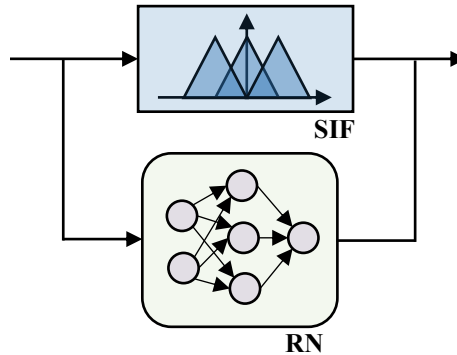


Figure 4.4. Système neuro-flou en parallèle

❖ Post-traitement

Une autre application des systèmes neuro-flous coopératifs est le post-traitement des sorties d'un SIF. Lorsque les résultats du système flou ne sont pas directement utilisables ou nécessitent une adaptation à un environnement externe complexe, un RNA peut intervenir pour transformer ou adapter ces sorties, assurant ainsi leur pertinence et leur applicabilité.

4.3.3. Systèmes neuro-flous concurrents

Le réseau de neurone et le système flou fonctionnent ensemble pour la même tâche, mais indépendamment, c-à-d ni l'un ni l'autre n'est employé pour déterminer les paramètres de l'autre. Habituellement le réseau neuronal traite les entrées, ou post-traite les sorties du système flou (voir figure 4.5).

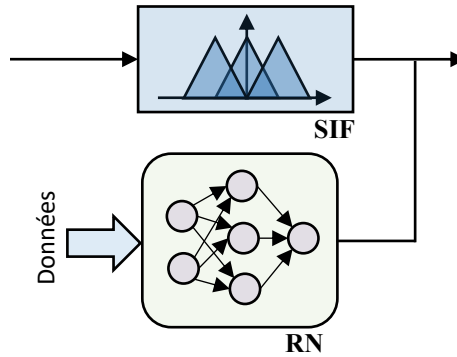


Figure 4.5. Système neuro-flou concurrent

4.3.4. Systèmes neuro-flous hybrides

Le système neuro-flou hybride représente une intégration avancée des réseaux de neurones artificiels (RNA) et de la logique floue au sein d'une architecture unifiée, permettant de combiner leurs avantages complémentaires. Contrairement aux approches coopératives ou concurrentes, cette approche fusionne les deux paradigmes en une seule architecture homogène, le système peut être interprété comme un réseau de neurones utilisant des paramètres flous ou bien un système implémenté selon une logique floue de forme distribuée ou parallèle. Le système d'inférence floue est mis sous la forme d'un réseau multicouche dans lequel les poids correspondent aux paramètres du système. L'architecture du réseau dépend du type de règles et des méthodes d'inférence, d'agrégation et de défuzzification choisie. L'architecture la plus connue, ANFIS (Adaptive Neuro-Fuzzy Inference System), illustre parfaitement ce principe en structurant un système de type Takagi-Sugeno sous forme de couches neuronales adaptables par rétropropagation.

4.4. Quelques modèles neuro-flous

Dans cette partie, nous discuterons brièvement quelques modèles neuro-flous qui font usage des complémentarités de réseaux neuronaux et systèmes d'inférence flou rendant effectif un SIF de type Mamdani ou Takagi Sugeno.

Nous citons quelques travaux dans ce domaine tel que GARIC, FALCON, ANFIS, NEFCON, NEFCLASS, NEFPROX, etc.

4.4.1. ANFIS (Adaptive-Network-based Fuzzy Inference System)

L'architecture ANFIS proposée par J. R. Jang a pour objectif d'exploiter les capacités d'apprentissage des RNA pour la détermination d'une base optimale des règles floues du raisonnement approximatif du type Takagi et Sugeno avec 5 couches.

C'est un modèle, qui est très utilisé dans les applications de contrôle et de classification, c'est le modèle de système flou implémenté selon une topologie de réseau de neurones. Il consiste à exploiter les capacités d'apprentissage et de généralisation des réseaux de neurones pour générer les paramètres du système flou. Le système flou est implémenté dans une topologie de réseau de

neurone, où les poids de connexion représentent les paramètres de la fonction d'appartenance des variables floues faisant partie de la base de règles.

La figure 4.6 présente l'architecture d'un ANFIS formalisant le raisonnement de Takagi et Sugeno du premier ordre, à deux entrées x_1, x_2 , une sortie f et une base de règles constituée de deux règles :

Règle 1 : **SI** x_1 est A_1 **ET** x_2 est B_1 **ALORS** $f_1 = r_1 + p_1x_1 + q_1x_2$

Règle 2 : **SI** x_1 est A_2 **ET** x_2 est B_2 **ALORS** $f_2 = r_2 + p_2x_1 + q_2x_2$

Où : A_1, A_2, B_1 et B_2 sont les ensembles flous. p_i, q_i et r_i : sont des paramètres du conséquent de la règle i déterminés pendant le processus d'apprentissage.

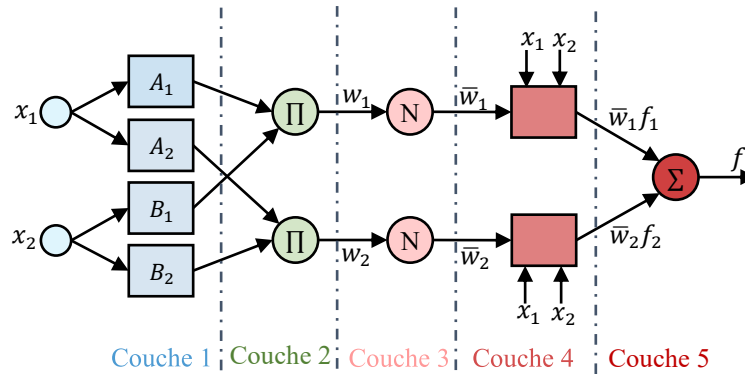


Figure 4.6. Modèle ANFIS

Le réseau ANFIS est un réseau multicouche dont les nœuds sont de deux types différents selon leur fonctionnalité, ceux qui contiennent des paramètres (nœuds carrés) et ceux qui ne contiennent pas (nœuds circulaires).

Dans ce qui suit, on donne les fonctions réalisées par les cinq couches de l'architecture ANFIS :

➤ **Couche 01 (la fuzzification)**

Chaque nœud de cette couche fait le calcul des degrés d'appartenance des valeurs d'entrées. Ces degrés sont donnés par :

$$\begin{cases} O_i^1 = \mu_{A_i}(x_1) & \text{pour } i = 1, 2 \\ O_i^1 = \mu_{B_i}(x_2) & \text{pour } i = 3, 4 \end{cases} \quad (4.1)$$

Avec :

x_1 et x_2 sont les entrées des nœuds (1, 2) et (3, 4) respectivement.

A_i, B_i les termes linguistiques associés aux fonctions d'appartenance $\mu_{A_i}(x_1)$ et $\mu_{B_i}(x_1)$.

Les sorties O_i^1 de la première couche représente donc les degrés d'appartenance des variables d'entrée aux ensemble flous. Dans le modèle de Jang, les fonctions d'appartenance sont des cloches généralisées avec les paramètres (a_i, b_i, c_i) , données par :

$$\mu_{B_i}(x) \text{ ou } \mu_{A_i}(x) = \frac{1}{1 + \left[\left(\frac{(x_i - c_i)}{a_i} \right)^2 \right]^{b_i}} \quad (4.2)$$

➤ *Couche 02 (les règles floues)*

Cette couche est formée d'un nœud pour chaque règle floue et génère les poids synaptiques. Ces nœuds sont de type fixe notés Π et chacun d'eux engendre en sortie le produit de ses entrées (opérateur ET de la logique floue), ce qui correspond au degré d'activation de la règle considérée :

$$O_i^2 = w_i = \min(\mu_{A_i}(x_1), \mu_{B_i}(x_2)) = \mu_{A_i}(x_1) \cdot \mu_{B_i}(x_2) \quad \text{pour } i = 1, 2 \quad (4.3)$$

➤ *Couche 03 (la Normalisation)*

Les nœuds de cette couche sont également fixes et réalise la normalisation des poids des règles floues selon la relation :

$$O_i^3 = \bar{w}_i = \frac{w_i}{w_1 + w_2} \quad \text{pour } i = 1, 2 \quad (4.4)$$

Chaque nœud i de cette couche est un nœud circulaire appelé N. La sortie du nœud i est le degré d'activation normalisé de la règle i .

➤ *Couche 04 (Conséquence)*

Chaque nœud de cette couche est adaptatif et calcule les sorties des règles en réalisant la fonction :

$$O_i^4 = \bar{w}_i \cdot f_i = \bar{w}_i \cdot (r_i + p_i x_1 + q_i x_2) \quad \text{pour } i = 1, 2 \quad (4.5)$$

Les paramètres $(r_i, p_i \text{ et } q_i)$ sont les paramètres de sortie de la règle i (la partie conclusion).

➤ *Couche 05 (la Sommation)*

Elle comprend un seul neurone qui fournit la sortie de l'ANFIS en calculant la somme des sorties précédentes. Sa sortie qui est également celle du réseau est déterminée par la relation suivante :

$$O_i^5 = f = \sum_i \bar{w}_i \cdot f_i \quad \text{pour } i = 1, 2 \quad (4.6)$$

L'architecture de l'ANFIS contient donc deux couches adaptatives :

- ❖ La première couche présente trois paramètres modifiables (a_i, b_i, c_i) liées aux fonctions d'appartenance des entrées, appelés paramètres des prémisses.
- ❖ La quatrième couche contient également trois paramètres modifiables (r_i, p_i, q_i) appelés paramètres conséquents (conclusion).

4.4.1.1. Apprentissage de l'ANFIS

L'apprentissage est la phase de développement du réseau neuro-flou en optimisant les paramètres d'un système flou : les paramètres de la partie prémisses (fonctions d'appartenance) et la partie conclusion (les coefficients de sortie). Cette adaptation est basée sur la capacité d'apprentissage des réseaux de neurones artificiels multicouches à partir d'un ensemble de données. Pour l'identification des paramètres, la structure du réseau doit être fixée et les paramètres des fonctions d'appartenance et des conclusions seront ajustés en utilisant l'algorithme d'apprentissage de rétro-propagation avec une combinaison avec l'algorithme des moindres carrés. Le système ANFIS est défini par deux ensembles de paramètres : S_1 et S_2 tels que :

S_1 : représente les paramètres des ensembles flous utilisés pour la fuzzification dans la première couche de l'ANFIS :

$$S_1 = ((a_{11}, b_{11}, c_{11}), (a_{12}, b_{12}, c_{12}), \dots, (a_{1p}, b_{1p}, c_{1p}), \dots, (a_{np}, b_{np}, c_{np}))$$

Où p est le nombre des partitions floues de chacun des variables d'entrées et n est le nombre de variables d'entrées.

S_2 : représente les coefficients des fonctions linéaires (les paramètres conséquents) :

$$S_2 = (p_1, p_2, p_3, \dots, q_1, q_2, q_3, \dots, r_1, r_2, r_3, \dots)$$

Jang a proposé que la tâche d'apprentissage de L'ANFIS se faite en deux passages en utilisant un algorithme d'apprentissage hybride, comme illustré sur le tableau 4.2.

	Passage vers l'avant	Passage en arrière
Paramètres des prémisses	Fixe	Rétro- propagation
Paramètres des conséquences	Moindres carrés	Fixe

Tableau 4.2. Méthodes utilisées pour l'apprentissage de l'ANFIS.

Une autre alternative pour l'apprentissage des réseaux ANFIS est l'emploi des méthodes d'optimisation d'ordre zéro à savoir les AG, le recuit simulé et le PSO.

4.4.2. NEFCLASS (Neuro-Fuzzy CLASSification)

Modèle utilisé généralement en classification. Il est constitué de 3 couches : une couche d'entrée avec les fonctions d'appartenance, une couche cachée représentée par des règles et une couche de sortie définissant les classes (Figure 4.7).

Ce modèle est facile à mettre en application, il évite l'étape de défuzzification, tout en étant précis dans le résultat final, avec une rapidité bien supérieure aux autres architectures.

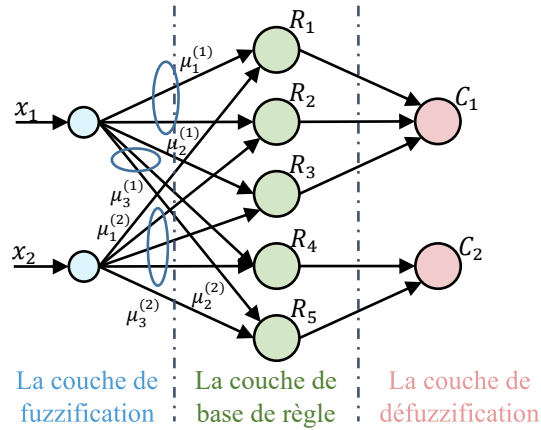


Figure 4.7. Architecture de NEFCLASS

4.4.3. NEFCON (Neuro-Fuzzy Controller)

Modèle formé de 3 couches. Une couche cachée formée par des règles, une couche d'entrée incluant les nœuds d'entrée avec les sous-ensembles flous d'antécédences et une couche de sortie avec un nœud de sortie et les sous-ensembles des conséquences.

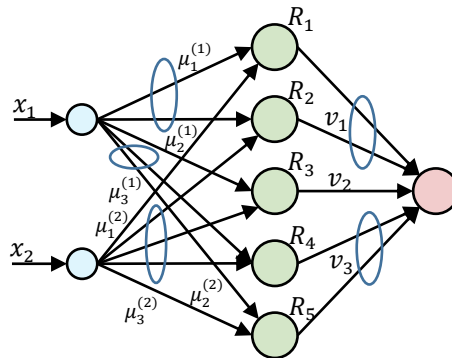


Figure 4.8. Architecture de NEFCON

L'élaboration des règles est similaire à l'architecture NEFCLASS, avec une différence en sortie. Cette architecture est généralement utilisée en approximation de fonctions et en contrôle flou (Figure 4.8).

Le processus d'apprentissage du NEFCON peut être divisé en deux phases. La première phase consiste à trouver les règles de base initiale. Si les connaissances antérieures ne sont pas disponibles, les règles de base seront apprises avec difficulté. Et si cette règle est définie par un expert l'algorithme les complète. Dans la seconde phase, les règles de base sont optimisées par modification des sous-ensembles flous des règles. Les deux phases utilisent l'erreur floue, cette erreur peut être trouvée avec la différence entre la sortie désirée et celle obtenue.

4.4.4. NEFPROX (Neuro Fuzzy function apPROXimator)

Modèle obtenu par l'association des deux architectures : NEFCLASS et NEFCON, il est utilisé dans différentes applications comme la classification et l'approximation de fonctions. Le NEFCLASS utilise un algorithme supervisé pour définir les règles floues, le NEFCON utilise un algorithme d'apprentissage non supervisé avec le calcul de l'erreur de sortie. Les deux modèles emploient la rétro propagation afin de définir les sous-ensembles flous.

Comparé au modèle ANFIS, NEFPROX (figure 4.9) est beaucoup plus rapide, mais ANFIS donne de meilleurs résultats en approximation.

Le NEFPROX est le premier système interprétable et lisible, dédié à l'approximation de fonction. Néanmoins, ses résultats en classification restent moins bons que ceux donnés par le NEFCLASS.

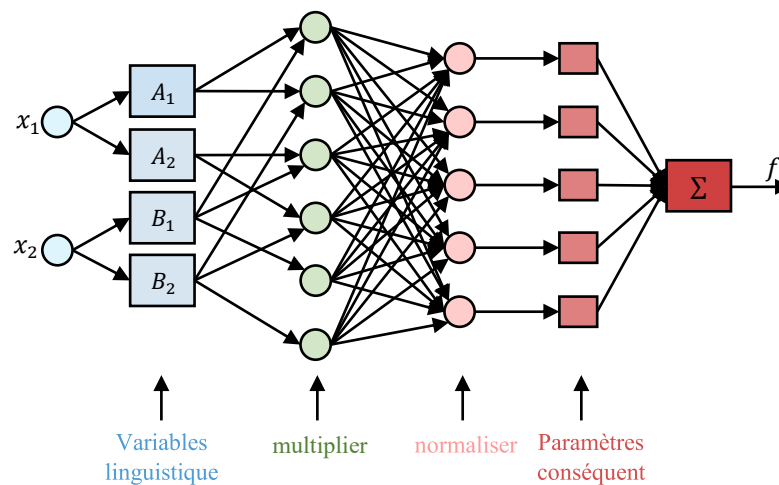


Figure 4.9. Architecture de NEFPROX

Nous pouvons initialiser le système NEFPROX si nous savons déjà des règles convenables ou autrement le système est capable à d'apprendre d'une manière incrémentale toutes les règles.

4.5. Conclusion

En conclusion, les réseaux neuro-flous constituent un outil puissant pour l'ingénierie des systèmes intelligents, offrant un équilibre entre performance et interprétabilité. La caractéristique principale de ces réseaux revient aux propriétés intrinsèques de la logique floue, auxquels deux propriétés des réseaux neuronaux sont ajoutées : Les possibilités d'apprentissage, permettant l'optimisation des règles floues et la mise en évidence d'une structure parallèle.

Cependant, leur conception et leur optimisation restent des défis majeurs, notamment en ce qui concerne le choix des fonctions d'appartenance, la réduction de la complexité du modèle et l'interprétabilité des règles générées. Malgré ces limitations, les réseaux neuro-flous continuent d'évoluer, bénéficiant des avancées en intelligence artificielle et en apprentissage automatique.

5.1. Introduction

L'être humain est de nature exigeant, il adore la perfection, il cherche toujours à améliorer son quotidien, tout en essayant bien sûr de minimiser ses charges, sa facture d'électricité ou la consommation du carburant de sa voiture, etc. donc il cherche toujours à maximiser ses gains et minimiser ses dépenses à vrai dire optimiser.

Accroître le taux de production tout en réduisant le coût est une requête très sollicitée dans le milieu industriel, ce qui a fortement contribué à l'émergence de l'optimisation qui est considérée comme une branche très importante des mathématiques. Les travaux faisant appel aux méthodes d'optimisation ont connu une croissance très rapide ces dernières années dans plusieurs domaines de la science tels que l'économie, les sciences sociales, la classification, la théorie de la décision et bien sûr l'engineering.

Utiliser une méthode pour la résolution d'un problème d'optimisation dépend de plusieurs facteurs tels que la nature du problème, de son degré de complexité et aussi des outils mathématiques et numériques disponibles. L'usage des méthodes dites classiques est limité aux cas où la modélisation du problème à résoudre peut-être analytiquement et/ou convexe, ce qui n'est pas le cas malheureusement pour de nombreuses situations.

Les méthodes stochastiques connues sous le nom de méta-heuristiques sont des outils puissants et efficaces pour résoudre le problème d'optimisation dans de telles situations. Ces algorithmes méta-heuristiques sont formellement définis comme des processus itératifs qui combinent intelligemment différents concepts pour explorer et exploiter l'espace de recherche. Des stratégies d'apprentissage sont utilisées pour structurer l'information afin de trouver efficacement des solutions quasi optimales. Parmi ces algorithmes nous pouvons citer les colonies de fourmis, les algorithmes génétiques, le recuit simulé, les essaims de particules (PSO), etc.

Les algorithmes génétiques (AGs) sont une méthode d'optimisation s'appuyant sur des techniques dérivées de la génétique et de l'évolution naturelle, inspirés de la théorie darwinienne à savoir les mutations, les croisements, la sélection, etc. Ces algorithmes sont basés sur des mécanismes très simples, ils sont robustes car ils peuvent résoudre des problèmes fortement non linéaires et discontinus, et efficaces car ils font évoluer non pas une solution mais toute une population de solutions potentielles et donc ils bénéficient d'un parallélisme puissant.

A l'image des AGs, l'optimisation par les PSO a connu un grand succès ces dernières années dans divers domaines. Cette technique inspirée du comportement social des animaux a été largement utilisée en Automatique pour optimiser les paramètres des contrôleurs comme le.

Dans ce chapitre nous aborderons quelques définitions générales des méthodes d'optimisation. Nous donnons aussi quelques concepts théoriques de deux méthodes d'optimisation qui sont les AGs et le PSO.

5.2. Optimisation

5.2.1. Définition

L'optimisation consiste à trouver les meilleures solutions pour un problème donné dans des conditions préétablies. Le mathématicien vient dans ce cas pour concrétiser le vœu de l'ingénieur qui cherche à maximiser le bénéfice et/ou minimiser l'effort requis, en modélisant les différents problèmes sous des fonctions coûts dépendantes de certaines variables. Donc l'optimisation peut être définie comme étant le processus qui sert à trouver les variables donnant le minimum ou le maximum de cette fonction.

D'un point de vue mathématique, optimiser une fonction $f(\xi)$ où $\xi \in \delta$ (avec δ est un ensemble de solutions) revient à trouver une solution $\xi_{opt} \in \delta$, dite solution optimale, tel que $\forall \xi \in \delta$:

- Dans un problème de maximisation $f(\xi_{opt}) \geq f(\xi)$;
- Dans un problème de minimisation $f(\xi_{opt}) \leq f(\xi)$;

La fonction coût f reflète la qualité des solutions, elle est appelée aussi fitness ou fonction objectif.

Chaque problème de maximisation peut se transformer sous la forme d'un problème de minimisation et vis-versa.

Deux types de minima peuvent être distingués dans le domaine de l'optimisation :

- **Minimum local** : S'il existe un ensemble $N \subset \delta$, contenant ξ_{loc} , tel que $\forall \xi \in N, f(\xi_{loc}) < f(\xi)$, avec $\xi \neq \xi_{loc}$, alors ξ_{loc} représente un minimum local.
- **Minimum global** : Si on peut prouver que $\forall \xi \in \delta, f(\xi_{glob}) < f(\xi)$, alors ξ_{glob} représente un minimum global.

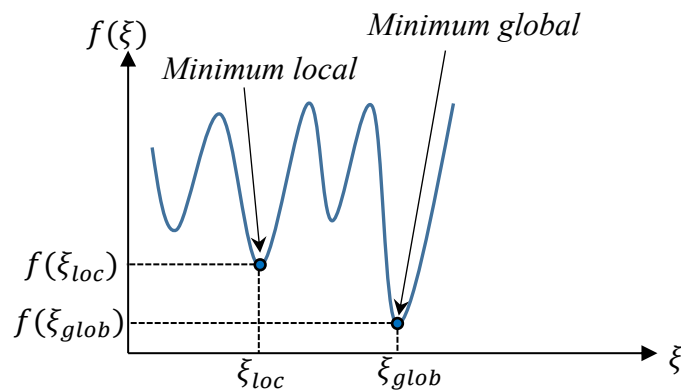


Figure 5.1. Minimum local vs minimum global

5.2.2. Problème d'optimisation

Nous pouvons définir un problème d'optimisation par un ensemble de variables, avec un ensemble de contraintes si elles existent et une fonction objectif à minimiser (ou maximiser). La résolution de ce problème revient donc à trouver la ou les meilleures solutions avec satisfaction de l'ensemble des contraintes. Le problème peut être formulé de la façon suivante :

- *Problème sans contrainte :*

$$\min_{\xi \in R^n} J(\xi) \quad (5.1)$$

- *Problème avec contrainte :*

$$\min_{\xi \in V} J(\xi) \quad (5.2)$$

Où $\xi = [\xi_1, \xi_2, \dots, \xi_n]^T$ représente le vecteur des n variables de décision, et $J(\xi)$ est la fonction objectif. Minimiser $J(\xi)$ est équivalent à maximiser $-J(\xi)$.

Les contraintes peuvent être de deux types :

- *Type inégalités :*

$$V = \{\xi \in R \text{ tels que } : w_j(\xi) \leq 0, j = 1, \dots, m\} \quad (5.3)$$

Où m représente le nombre de contrainte de type inégalité.

- *Type égalités :*

$$V = \{\xi \in R \text{ tels que } : h_j(\xi) = 0, j = 1, \dots, k\} \quad (5.4)$$

Où k représente le nombre de contrainte de type égalité.

Un problème d'optimisation doit posséder un espace de recherche qui représente l'ensemble des solutions possibles du problème.

Nous pouvons classer un problème d'optimisation de plusieurs façons :

- Existe-t-il des contraintes ou pas.
- Statique ou dynamique (un changement avec le temps de la fonction objectif est possible : les bourses économiques, le routage dans les réseaux, etc.).
- Mono-objectif ou multi-objectif (plusieurs fonctions objectifs à optimiser).

Actuellement les problèmes d'optimisations interviennent dans divers domaines et représentent des axes de recherche importants, en automatique, l'électronique, la mécanique, l'économie, le traitement d'image, la recherche opérationnelle, l'informatique, etc.

5.2.3. Méthodes d'optimisation

En dépit de l'abondance des méthodes d'optimisation proposées, nous pouvons suivant leurs approches les discerner des traits caractéristiques. Ainsi nous proposons la classification suivante : Les méthodes déterministes (ou classiques) et les méthodes non-déterministes (méta heuristiques).

5.2.3.1. Méthodes déterministes (classiques)

Lorsque la méthode de résolution évolue d'une manière prévisible ne laissant aucune place au hasard, alors celle-ci est qualifiée de *déterministe*. Tout d'abord ces méthodes ont été conçues pour résoudre d'une façon exacte des problèmes particuliers tels que les problèmes continus et linéaires avec des contraintes linéaires. Les pionniers à qui revient l'existence des méthodes d'optimisation classiques sont Newton, Lagrange, Euler Leibnitz et Cauchy. Malgré leurs contributions, il n'y a pas eu de très grands progrès, ce n'est qu'à partir de la deuxième moitié du vingtième siècle avec l'utilisation des ordinateurs numériques, permettant ainsi la mise en œuvre des procédures d'optimisation qu'on a pu voir des progrès spectaculaires. Mais de l'autre côté les problèmes à optimiser sont devenus plus complexes.

Généralement les méthodes déterministes sont adoptées lorsque la fonction objectif du problème possède certaines propriétés telles que la dérivabilité, la continuité ou la convexité. La qualité principale de ces méthodes est qu'elles n'ont pas besoin de point de départ et permettent d'avoir, à la convergence, une solution exacte du problème d'optimisation traité avec une garantie absolue, ainsi qu'une bonne gestion des contraintes. Cependant, il faut noter que ces méthodes restent d'usage tant que le nombre de variables n'est pas important (une vingtaine maximum).

Parmi les méthodes déterministes les plus réputées, nous pouvons trouver : la méthode du simplexe de Dantzig, la méthode du gradient et aussi la méthode de Newton.

5.2.3.2. Méthodes non-déterministes (Métaheuristiques)

Les méthodes déterministes peuvent être limitées dans la résolution de certains problèmes d'optimisation complexes, Parmi ces limites, nous avons la non-dérivabilité, la discontinuité, l'absence de convexité ou même des difficultés pour définir avec précision la fonction objectif. De plus, la plupart des problèmes d'optimisation sont NP-difficile, et souffre d'un temps de résolution trop long. Donc leur utilisation est limitée à certains problèmes de petite taille.

Les approches métaheuristiques paraissent comme dans ce cas comme un moyen intéressant pour résoudre les problèmes d'optimisation. Ces méthodes non déterministes appelées aussi stochastiques font appel à des tirages de nombre aléatoires. Elles permettent aussi d'explorer plus efficacement l'espace de recherche.

Le mot métaheuristique introduit pour la première fois par Fred Glover en 1986, est composé de deux mots d'origine Grec :

- Méta : un suffixe qui signifie « au-delà ».
- Heuristique : dérivé du verbe « heuriskein » qui signifie « trouver ».

Plusieurs définitions ont été proposées dans la littérature pour les métaheuristiques. Nous proposons celle donnée par : Une métaheuristique est formellement définie comme un processus de génération itératif qui guide une heuristique subordonnée en combinant intelligemment différents concepts pour explorer et exploiter l'espace de recherche, des stratégies d'apprentissage sont utilisées pour structurer l'information afin de trouver efficacement des solutions quasi optimales.

Les métaheuristiques sont caractérisées par le caractère stochastique, c.à.d. que la recherche de des solutions est partiellement conduite d'une manière aléatoire, et sont généralement itératives, permettant d'améliorer successivement les solutions obtenues. Elles sont inspirées de processus naturels, tel que la biologie (les algorithmes génétiques, recherche tabou...), l'éthologie (colonies de fourmis, PSO...) ou la physique (recuit simulé...).

Selon Blum et Roli les métaheuristiques ont en commun les propriétés suivantes :

- Les approches métaheuristiques vont de procédures simples de recherche locale à des procédures très complexe.
- L'objectif visé est d'explorer efficacement tous l'espace de recherche dans le but de trouver des solutions (presque) optimales.
- Les métaheuristiques sont des procédures permettant de guider le processus de recherche.
- Peuvent utiliser des mécanismes pour éviter la stagnation dans des optimaux locaux.
- Les métaheuristiques sont généralement approximatives et non-déterministes.
- Les métaheuristiques peuvent utiliser l'expérience accumulée pendant la recherche de l'optimum, pour mieux orienter la suite du processus de recherche.

5.2.3.2.1. Classification des métaheuristiques

Plusieurs classifications des métaheuristiques sont possibles, selon la nature de l'algorithme en lui-même, on peut distinguer :

A. Inspirées de phénomènes naturels vs non-inspirées de phénomènes naturels

D'une manière intuitive une classification des métaheuristiques est possible en séparant les méthodes inspirées de la nature, des autres qui ne le sont pas. A titre d'exemple, les algorithmes des colonies de fourmis et les algorithmes génétiques sont inspirées respectivement du comportement naturel des fourmis en cherchant la nourriture et la théorie de l'évolution. En contrepartie, l'algorithme de recherche tabou n'est pas inspiré d'un phénomène naturel.

Cette classification est parfois difficile à réaliser, et ne semble pas significative. En effet, il y a de nombreuses métaheuristiques hybrides récentes qui sont difficiles à classer dans l'une des deux classes. Par exemple, si l'usage d'une mémoire dans l'algorithme de recherche tabou n'est pas inspiré de la nature.

B. Méthodes de trajectoire vs Méthodes basées sur une population de points

On peut également séparer les métaheuristiques travaillant avec une population de solutions de celles à solution unique. Ces dernières qui ne manipulent qu'une seule solution à la fois sont appelées aussi méthodes de trajectoires. L'algorithme de recherche tabou, la Recherche à Voisins Variables et le Recuit Simulé sont des exemples typiques de méthodes de Recherche Locale à solution unique. En revanche, les métaheuristiques à population de solutions au fur et à mesure des itérations, améliorent une population des solutions. Ces méthodes utilisent la population comme un facteur de diversité. Les algorithmes génétiques, l'optimisation par essaim particules et la recherche par dispersion sont des exemples de méthodes appartenant à cette catégorie.

C. Fonction objectif statique vs fonction objectif dynamique

La manière d'utiliser la fonction objectif peut être considérée comme un critère pour classer les métaheuristiques. Lorsqu'elles travaillent directement sur la fonction à optimiser f , ces

métaheuristiques sont dites statiques. Par contre si elles utilisent une fonction g obtenue à partir de f avec l'ajout de quelques composantes permettant ainsi de modifier la topologie de l'espace des solutions, ces composantes additionnelles ont la possibilité de varier pendant le processus de recherche.

D. Usage de mémoire à court terme vs mémoire à long terme

L'utilisation de l'historique de la recherche peut être considérée comme un critère de classification. Certains algorithmes utilisent l'historique, alors que d'autres n'ont aucune mémoire du passé. Ces derniers sont des processus markoviens puisque la situation courante c'est elle qui détermine totalement l'action à réaliser. L'utilisation de l'historique de la recherche peut être effectuée de différentes manières. On peut faire généralement la différence entre les méthodes ayant une mémoire à long terme de celles qui ont une mémoire à court terme.

E. Diversification vs intensification

Un autre critère de classification selon l'utilisation de la diversification ou de l'intensification. Le terme diversification, signifie une exploration assez large de l'espace de recherche, alors que l'intensification veut dire l'exploitation de l'information accumulée durant la recherche.

Dans ce qui suit, nous nous sommes intéressés particulièrement à deux méthodes basées sur des populations de solutions et qui ont trouvé un large domaine d'utilisation en engineering. Ces méthodes sont « les algorithmes génétiques » et « les essaims de particules PSO ».

5.3. Les algorithmes génétiques

L'objectif visé par tous les algorithmes d'optimisation c'est de trouver une solution optimale tout en respectant un ensemble de contraintes.

Inspirées des phénomènes naturels, les algorithmes génétiques sont des algorithmes informatiques qui reposent dans leurs conceptions sur la théorie de Darwin qui date des années 1800. Cette théorie repose sur deux principes. Le premier est qu'au sein d'un environnement, il n'y a que les espèces qui sont les mieux adaptées qui ont la possibilité de perdurer au cours du temps, pour le reste elles sont menacées de disparaître. Le deuxième principe de cette théorie repose sur le fait que dans chaque espèce, la population est renouvelée à l'aide des meilleurs individus, c'est ce qu'on appelle l'évolution et elle est opérée selon Darwin sur les chromosomes contenus dans leurs ADN.

Mais il fallait attendre les années cinquante du vingtième siècle pour voir les premiers travaux sur les algorithmes génétiques, lorsque des biologistes américains arrivent à simuler des structures biologiques sur ordinateur. Ensuite le professeur John Holland de l'Université du Michigan commença à s'intéresser à ce qui allait devenir les algorithmes génétiques au début des années soixante. Un premier aboutissement de ses travaux fut en 1975 en publiant son livre intitulé *Adaptation in Natural and Artificial Systems*. Malheureusement à cette époque les ordinateurs n'étaient pas assez puissants pour pouvoir envisager une application sur des problèmes concrets de grande taille.

La publication de l'ouvrage de Goldberg qui décrit d'une manière concrète l'utilisation des algorithmes génétiques dans le cadre de la résolution des problèmes réels a permis de mieux faire

connaître ces algorithmes et a signé le début d'un nouvel intérêt pour cette technique d'optimisation.

Pratiquement la performance des algorithmes génétiques est basée sur une approche empirique du problème et repose sur le savoir-faire de l'utilisateur.

5.3.1. Terminologie et concepts de base des algorithmes génétiques

Les algorithmes génétiques sont des méthodes d'optimisation s'appuyant sur des techniques issues de la génétique et de l'évolution naturelle selon les règles de Mendel et la théorie de Darwin, à savoir le croisement, la mutation, la sélection, etc. Durant l'évolution, les individus ou les organismes qui y participent interagissent et vivent ensemble dans un environnement où ces ils partagent les mêmes ressources et se reproduisent pour créer de nouvelles générations. Les individus les plus puissants possèdent plus de chance de se reproduire et de subsister que les individus les plus faibles. De ce fait, les gènes des individus qui se sont bien adaptés se transmettront à d'autres individus au cours des générations et l'espèce s'adaptera à son environnement sans cesse.

Puisque ces algorithmes emploient une terminologie empruntée à celle de la génétique, nous devons définir les concepts suivants :

5.3.1.1. Génotype et phénotype

En termes de la génétique, le génotype est défini comme étant l'ensemble des valeurs des gènes situés sur des chromosomes tandis que le phénotype indique la solution réelle après la modification du chromosome en d'autres termes le phénotype représente l'ensemble des caractères d'un individu. Ces caractères ont la possibilité d'être acquis d'une manière héréditaire (c.à.d. transmission d'une génération à une autre) ou modifiés par l'environnement (cicatrice, musculature, etc.). Pour générer une nouvelle population, plusieurs opérateurs génétiques interviennent tels que le croisement, la mutation, et la sélection pour manipuler les chromosomes.

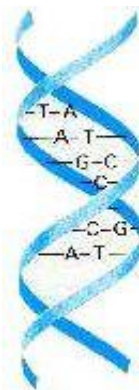


Figure 5.2. Le génotype

5.3.1.2. La population

Dans un problème d'optimisation les algorithmes génétiques vont faire évoluer un ensemble de solutions candidates, celles-ci sont appelées population. Donc la population est l'ensemble des individus (chromosomes) qui représentent chacun une solution possible d'un problème donnée. À tout instant t , cette population est nommée génération.

5.3.1.3. Les individus

Tout point de l'espace d'évolution est appelé individu (une solution). Ces individus sont caractérisés par une architecture génétique "génotype du chromosome ou individu" (codage des solutions du problème) contenant ainsi toute information nécessaire pour évaluer la fonctionnelle à optimisée. Classiquement un chromosome possède N gènes qui sont dans un problème binaire N bits de valeur 0 ou 1.

5.3.1.4. Les chromosomes

Un chromosome est un ensemble de gènes. Dans le langage de la génétique c'est la structure qui donne les caractéristiques d'un individu.

5.3.1.5. Les gènes

L'élément de base constituant les chromosomes est appelé un gène (en génétique il donne le caractère de la chaîne ADN). Chaque chromosome est constitué d'une suite de gènes permettant de coder les fonctionnalités de l'organisme.

La composition d'une population dans un algorithme génétique est donnée par la figure 5.3.

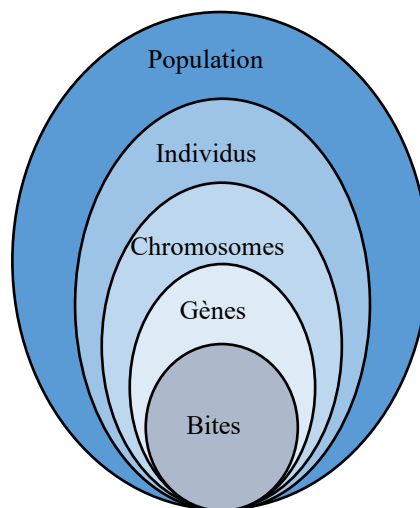


Figure 5.3. Les niveaux d'organisation d'un Algorithme Génétique

5.3.2. Méthodologie adoptée

D'une façon générale, pour optimiser une fonction $f(\xi)$ quelconque, il suffit de calculer sa dérivée par rapport à la variable ξ et ensuite calculer la valeur de cette variable pour laquelle l'expression résultante s'annule. Par la suite pour vérifier si cette solution optimale correspond à un minimum ou un maximum, la dérivée seconde est requise.

$$\max f(\xi) \rightarrow \frac{df}{d\xi} = 0, \quad \frac{d^2f}{d\xi^2} < 0 \quad (5.5)$$

Pratiquement toutes les techniques d'optimisation conventionnelles se basent sur ce genre de procédure. Malgré la simplicité dans la conception de cette méthode, elle présente des limites en termes de pratique. Plusieurs problèmes existent malheureusement, dont il est quasiment impossible de calculer d'une manière analytique une solution pour ξ . Les algorithmes génétiques apparaissent comme une alternative pour résoudre tous ces problèmes.

La recherche du ou des extrema d'une fonction définie sur un espace de données en utilisant les algorithmes génétiques se fait d'une manière itérative (génération), jusqu'à ce qu'un test d'arrêt soit vérifié. Pour pouvoir utiliser un algorithme génétique pour résoudre un problème d'optimisation, plusieurs étapes doivent être établies :

- Le codage des différents éléments de la population ;
- La génération d'une population initiale avec une taille fixe ;
- Définir la fonction objectif (fitness) pour évaluer les solutions ;
- Appliquer les opérateurs génétiques (croisement, mutation, etc.) pour générer de nouvelles solutions ;
- Un mécanisme pour choisir les solutions qui doivent rester et celles qui doivent disparaître, avec un test d'arrêt.

Le principe de fonctionnement des algorithmes génétiques est montré sur la figure 5.4, L'algorithme débute avec un ensemble de solutions possibles du problème (les individus), pour constituer une population. Des éléments variables forment les individus, qui représentent dans un dispositif les paramètres à ajuster. La conception de cette population se fait d'une façon aléatoire dans un intervalle prédéfini (les limites dictées par l'aspect pratique du problème d'optimisation). Par la suite, certaines solutions sont sélectionnées à partir de la première population pour former une nouvelle population, en utilisant les opérateurs génétiques (la sélection, le croisement et la mutation). Le but escompté à travers ceci est que la nouvelle population soit nettement meilleure que la population qui la précède. Une sélection aléatoire permet de choisir les solutions qui vont servir à créer de nouvelles solutions, en se basant sur leurs fonctions objectifs construites en concordance avec le problème d'optimisation. Les meilleurs individus auront plus de chance de se reproduire (c'est-à-dire, la probabilité d'être choisi pour subir les différents opérateurs génétiques sera plus grande). Cette procédure sera répétée jusqu'à la satisfaction d'un critère de convergence (un nombre prédéfini de générations, un temps de simulation, pas d'amélioration sur la performance des solutions, etc.).

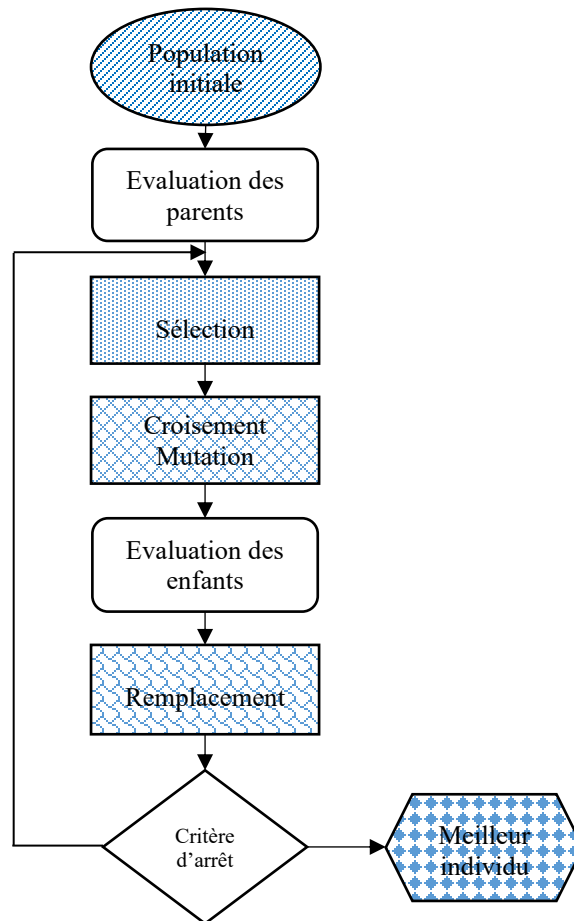


Figure 5.4. Schéma fonctionnel d'un Algorithme Génétique

La structure d'un algorithme génétique peut aussi être présentée sous forme de l'algorithme suivant :

Algorithme 5.1 : Algorithme génétique

Début

Initialiser la population

Evaluer les individus de la population

Répéter

Sélection

Reproduction des individus sélectionnés (croisement et mutation)

Evaluer les enfants

Remplacer certains parents par les enfants

Jusqu'à condition d'arrêt

Fin

5.3.2.1. Le codage

La première étape consiste à définir et à coder d'une manière convenable le problème. La représentation d'un individu sous une forme de chromosome est appelée codage. Ce chromosome est formé d'un ensemble de gènes, à qui sont attribués des valeurs dans un alphabet qui dépend de la nature du codage en lui-même. Ceci permettra d'établir une liaison entre les individus de la population et la valeur de la variable, de façon à imiter la version génotype-phénotype existante dans le monde réel. Cette étape demande un grand soin, car l'efficacité des algorithmes génétiques dépend du choix du codage d'un chromosome. C'est la spécificité du problème traité qui conditionne le choix du codage des données. Différentes manières existent pour coder un chromosome qui dépend de l'alphabet utilisé. Les plus connus sont le codage binaire, le codage réel et un peu moins le codage de Gray.

5.3.2.1.1. Le codage binaire

Dans ce type de codage les gènes sont codés par des caractères binaires 0 ou 1. C'est le premier codage qui a été appliqué sur les algorithmes génétique, et c'est le plus couramment utilisé. Le codage binaire peut facilement être utilisé pour coder différentes sortes d'objets (les entiers, les réels, les chaînes de caractères, etc.), il suffit juste d'utiliser des fonctions de codage et de décodage pour assurer le passage d'une représentation à une autre.

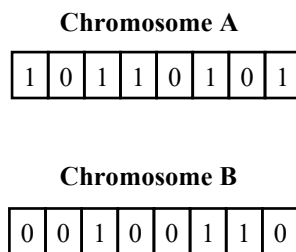


Figure 4.5. Codage binaire

5.3.2.1.2. Le codage réel

Dans certain cas, il est plus efficace de coder les chromosomes en utilisant un codage réel. Ainsi les gènes sont représentés par des nombres réels au lieu d'avoir une étape de transcoding (du réel vers le binaire et vice versa). Ce type de codage permet une plus grande efficacité et une évaluation plus rapide des chromosomes. En effet, les chromosomes codés en réel sont plus courts que ceux codés en binaires.

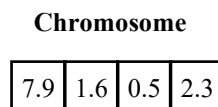


Figure 5.6. Codage réel

5.3.2.2. La fonction objectif (Fitness)

Pour concrétiser au mieux le processus d'évolution, il est essentiel de pouvoir faire la distinction entre les chromosomes les moins adaptés et les plus adaptés. Une valeur d'adaptation est assignée à chaque chromosome pour distinguer les mieux adaptés à leur environnement des autres. Ceci permet de faire la comparaison entre les différents individus.

Souvent, la formulation de cette fonction spécifique à un problème est simple lorsque le nombre des paramètres n'est pas élevé. Cependant, définir cette fonction devient difficile lorsque le nombre de paramètre est élevé ou dans le cas où ils sont corrélés. La fonction peut devenir dans ce cas une somme avec pondération de plusieurs fonctions.

Mettre au point une bonne fonction objectif (fitness function) impose un respect de plusieurs critères qui se ramènent à satisfaire les contraintes du problème ainsi que sa complexité. S'il y a des contraintes à satisfaire et que les chromosomes issus des opérateurs de recherche codent les individus invalides, une solution parmi d'autres consiste à attribuer une mauvaise fitness à l'individu qui a violé les contraintes afin de favoriser la reproduction des individus valides.

5.3.2.3. Générer une population initiale

Une fois le codage établi, nous devons déterminer une population initiale formée d'un ensemble de solutions admissibles du problème d'optimisation. Cette étape conditionne fortement la rapidité de l'algorithme. Cette population est générée d'une manière aléatoire dans le cas où la position du minimum (ou maximum) est inconnue dans l'espace de recherche, en effectuant des tirages uniformes dans tous les domaines associés aux composantes de l'espace et en respectant les contraintes sur les individus. Si par contre, il existe des informations a priori sur le problème, il paraît évident de générer les individus dans un sous domaine particulier, ce qui permettra d'accélérer la convergence.

Disposant d'une population non homogène, la diversité de la population doit être entretenue au cours des générations pour pouvoir parcourir le plus largement possible l'espace de recherche. C'est le rôle des opérateurs de reproduction.

Par ailleurs, à cette étape le choix de la taille de la population pose problème. En effet, une population trop petite conduit probablement à obtenir un optimum local peu intéressant, alors qu'une population trop grande augmentera excessivement le temps de calcul ainsi que l'espace mémoire requis. Le choix de la taille de la population doit être effectué de tel sorte à avoir un bon compromis entre qualité des solutions et temps de calcul.

Mais il faut savoir aussi que la puissance de calcul dont nous disposons, les méthodes utilisées dans les opérateurs de reproduction, la fonction objectif et le nombre de variables considérées, affecte grandement la taille de la population. Si à titre d'exemple, une fonction qu'on cherche à optimiser comporte un optimum global clair et peu d'optimums locaux, une petite taille de la population sera suffisante, ce qui n'est pas le cas pour une fonction plus compliquées qui comporte un nombre élevé d'optimums locaux.

5.3.2.4. L'opérateur de sélection

L'opérateur de sélection permet l'identification statistique des individus d'une population courante qui auront l'autorisation à se reproduire. Ce qui offre aux individus sélectionnés une capacité à se diffuser et à persister dans la population. Cet opérateur est fondé sur la performance de chaque individu au sein de la population, estimée en utilisant la fitness. Cela interprète d'une manière approchée le concept de la sélection naturelle, où les gènes qui ont une performance relativement faible ont tendance à disparaître tandis que les gènes les plus performants tendent à se diffuser. Ce procédé va permettre de donner aux meilleurs individus, une probabilité plus élevée de contribuer aux générations futures. Donc la sélection est opérée à partir de la fonction objectif, après avoir effectué l'évaluation d'une génération de la population à un instant donné t .

De nombreuses techniques de sélection existent dans la littérature, les plus courantes sont :

5.3.2.4.1. Sélection par la roulette

Ce type de sélection est basé sur le principe de la roulette utilisée dans les casinos. On associe à Chaque individu de la population un segment dans cette roulette. La fitness de l'individu détermine la largeur du segment associé d'une manière proportionnelle, donc la probabilité d'être sélectionné est proportionnelle à sa fitness. En tournant la roue et en faisant lancer la boule dans cette roulette, la case où la boule tombera correspond à l'individu sélectionné. Les individus qui ont une meilleur fitness possèdent donc plus de chance d'être tirés au sort au déroulement de ce jeu. Ce système favorisera les grands segments, c'est-à-dire les bons individus seront plus souvent sollicités que les moins bons. Cependant, les individus les moins bons gardent toujours une chance d'être sélectionnés, pour maintenir une certaine diversité.

Si le nombre d'individus participants dans la population est égal à N , alors un individu ξ_j a une probabilité de sélection $P(\xi_j)$ égale à :

$$P(\xi_j) = \frac{F(\xi_j)}{\sum_{k=1}^N F(\xi_k)} \quad (5.6)$$

Avec $F(\xi_j)$ est la fonction objectif de l'individu ξ_j .

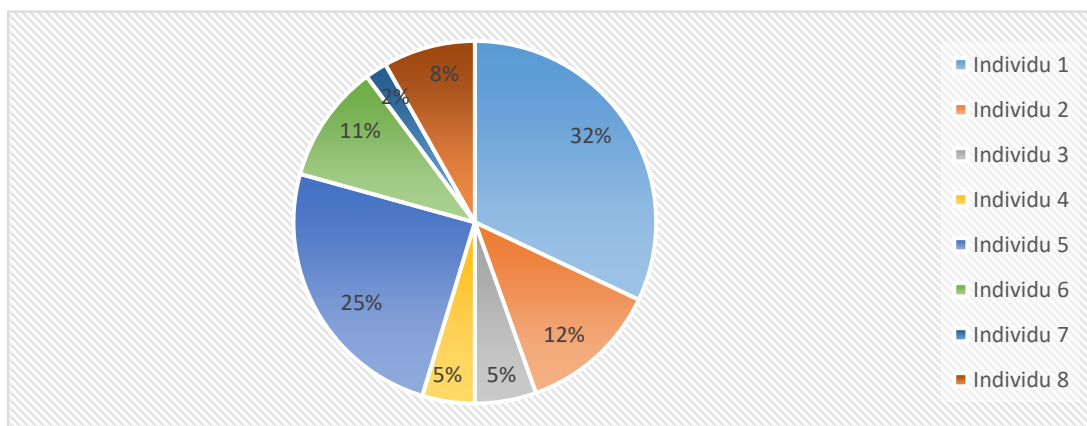


Figure 5.7. Sélection par roulette

5.3.2.4.2. Sélection par tournoi

Dans ce mode de sélection, la procédure se fait par un choix de deux ou de plusieurs individus au hasard qui vont se combattre (comparaison de leurs fitness) pour participer à la prochaine étape. Les individus de mauvaise qualité vis-à-vis à leurs fitness ont plus de chance de participer dans l'amélioration de la population.

En réalité c'est une compétition qui opposer a les individus d'une sous-population prise au hasard dans la population, de taille égale ou inférieure à la taille de cette population et qui est fixée à priori par l'utilisateur. Le vainqueur dans cette compétition est considéré comme le meilleur individu dans la sous-population et sera donc sélectionné pour lui appliquer l'opérateur de croisement.

Plusieurs possibilités existent pour faire varier le mode de fonctionnement de la compétition, comme la variation du nombre d'individus qui devront s'affronter au départ, ou encore la permission ou non à un même individu d'être éligible plusieurs fois dans un même tournoi.

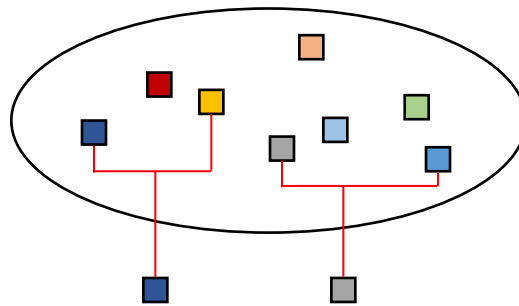


Figure 5.8. Sélection par tournoi

5.3.2.4.3. Sélection par rang

Dans cette sélection les individus de la population sont rangés soit en ordre croissant ou décroissant suivant l'objectif et par la suite ne retenir qu'un nombre fixe de chromosomes. Dans ce cas on conserve les individus les plus forts. Ce qui parfois peut représenter un inconvénient majeur pour cette méthode, parce qu'il est nécessaire parfois de garder quelques individus faibles pour avoir une diversité dans la population. Un autre problème dans la limite à fixer à la sélection, ce qui peut empêcher parfois de garder de bons individus pour les prochaines générations.

5.3.2.5. L'opérateur de croisement

Pour créer de nouveaux individus, il est nécessaire de prendre aléatoirement une partie des chromosomes de chacun des deux parents et de les mélanger. Ce phénomène, inspiré de la nature est appelé croisement (crossover en anglais). Le but du croisement est d'enrichir la diversité de la population à travers la manipulation des structures des chromosomes. D'une manière classique, le croisement est appliqué en utilisant deux parents et génère deux enfants comme le montre la figure 5.9. Les couples d'individus sélectionnés subissent un croisement avec une probabilité P_c , un nombre aléatoire α est généré dans un intervalle $[0,1]$, et les individus subissent un croisement si et seulement si $\alpha \leq P_c$, sinon le couple procède sans croisement. Les valeurs typiques de P_c sont

de 0,4 à 0,9. Si par exemple, $P_c=0,5$, alors la moitié de la population sera formée par sélection et croisement, et l'autre moitié par sélection seule .

Initialement, le croisement à découpage de chromosome est associé au codage par chaînes de bits. Ce type de croisement est appliqué sur des chromosomes formés de M gènes, en tirant d'une façon aléatoire une position dans chacun des parents. Par la suite, on effectue un changement des deux sous-chaines terminales par rapport à cette position de chacun des deux chromosomes, produisant ainsi deux enfants. Ce principe peut être étendu non seulement à un découpage du chromosome en deux sous-chaines mais en trois, quatre, etc. le croisement à découpage de chromosome est très pratique pour les problèmes discrets.

En présence d'un problème continu, un croisement « barycentrique » est souvent sollicité. Dans ce cas deux gènes $P_1(j)$ et $P_2(j)$ sont sélectionnés à la même position j dans chacun des parents. Ils vont définir deux nouveaux gènes par pondération, en créant deux nouveaux points sur la droite reliant ces derniers. Les enfants $E_1(j)$ et $E_2(j)$ sont créés de la manière suivante :

$$\begin{cases} E_1(j) = \alpha P_1(j) + (1 - \alpha) P_2(j) \\ E_2(j) = (1 - \alpha) P_1(j) + \alpha P_2(j) \end{cases} \quad (5.7)$$

Avec α un coefficient de pondération aléatoire adapté au domaine d'extension des gènes (pas forcément compris entre 0 et 1).

Ce type de croisement permet de générer des points à l'extérieur, ou entre les deux gènes considérés.

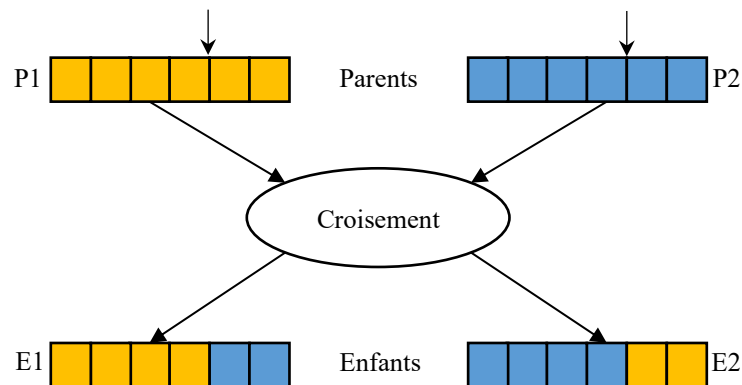


Figure 5.9. Croisement en un seul point

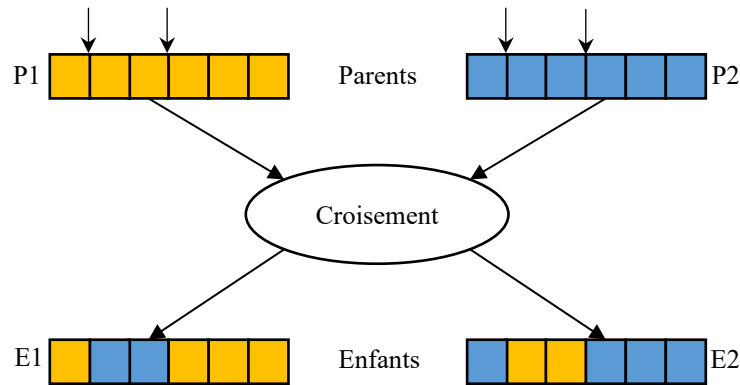


Figure 5.10. Croisement en deux points

5.3.2.6. L'opérateur de mutation

La mutation est un autre opérateur utilisé dans les algorithmes génétiques. Il s'agit de l'altération occasionnelle de la valeur d'un chromosome avec une certaine probabilité p_m . Même si d'autres types de mutations existent dans la littérature, la plus couramment utilisée est la mutation de bits. Dans la mutation de bits, chaque bit de la chaîne est muté avec une probabilité p_m . Pour une chaîne codée en binaire, la mutation est définie comme la conversion de 1 en 0, et vice versa, tel qu'il est montré sur la figure 5.11.

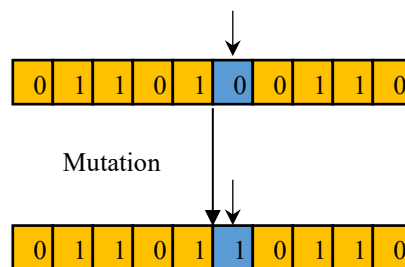


Figure 5.11. Une mutation dans un chromosome

La mutation est un opérateur très important, car elle apporte aux algorithmes génétiques l'aléa essentiel à une exploration efficace. Elle garantit aussi que l'algorithme génétique sera capable d'atteindre la majorité des points du domaine de recherche. C'est aussi une assurance contre une convergence prématurée, en évitant les optimums locaux par la diversification qu'elle donne. Sans mutation, des chaînes similaires sont traitées à chaque génération, ce qui peut provoquer une convergence vers des optimums locaux.

L'opérateur de mutation intervient généralement avec une probabilité fixée assez faible (de l'ordre de 1%). Si la probabilité de mutation est notée p_m , et r_m un nombre tiré au hasard tel que $r_m \in [0,1]$, et que $r_m \leq p_m$ alors l'individu subira une mutation. D'autre variante existe aussi où la mutation peut être appliquée sur plusieurs bits dans un même chromosome.

5.3.2.7. L'opérateur d'élitisme

En créant une nouvelle population, il y a un grand risque que les meilleurs individus soient perdus après l'application des deux opérateurs de reproduction. La méthode d'élitisme permet d'éviter cela, en copiant un ou plusieurs des meilleurs individus dans la nouvelle génération. Par la suite le reste de la population sera généré suivant l'algorithme de reproduction. Cet opérateur permet d'améliorer considérablement l'algorithme génétique, car les meilleures solutions seront gardées.

5.3.2.8. Test d'arrêt

Le test d'arrêt joue un rôle primordial dans le jugement de la qualité des individus. Son but est de nous assurer l'optimalité, de la solution finale obtenue par l'algorithme génétique.

Les critères d'arrêts sont de deux natures :

1. Arrêt après un nombre fixé a priori de générations. C'est la solution retenue lorsqu'une durée maximale de temps de calcul est imposée.
2. Arrêt lorsque la population cesse d'évoluer ou n'évolue plus suffisamment. Nous sommes alors en présence d'une population homogène dont on peut penser qu'elle se situe à la proximité de l'optimum. Ce test d'arrêt reste le plus objectif et le plus utilisé.

Il est à noter qu'aucune certitude concernant la bonne convergence de l'algorithme n'est assurée. Comme dans toute procédure d'optimisation l'arrêt est arbitraire, et la solution en temps fini ne constitue qu'une approximation de l'optimum.

5.3.3. Paramètres de l'algorithme génétique

Plusieurs paramètres peuvent conditionner la convergence des algorithmes génétiques :

- La taille de la population ;
- Le nombre max de génération que doit faire l'algorithme ;
- La probabilité de croisement ;
- La probabilité de mutation.

L'ajustement de ces paramètres dépend fortement du problème étudié. Donc il n'y a pas de paramètres valables pour résoudre tous les problèmes posés. Cependant, il existe des valeurs qui sont souvent employées (définies par la littérature) et peuvent être utilisées comme un point départ pour lancer la recherche de solutions.

- La valeur de la probabilité de croisement est choisie dans un intervalle $[0.4, 0.9]$;
- La valeur de la probabilité de mutation est choisie dans un intervalle $[0.001, 0.1]$

Trouver des valeurs à ces paramètres qui donnent de bons résultats est parfois une tâche délicate.

5.3.4. Exemple de résolution à base d'algorithme génétique

Afin de présenter les étapes à suivre dans un algorithme génétique classique, nous allons reprendre l'exemple cité par Goldberg. Il consiste à trouver le maximum de la fonction $f(x) = x^2$, pour une plage de $x = 0$ jusqu'à $x = 31$.

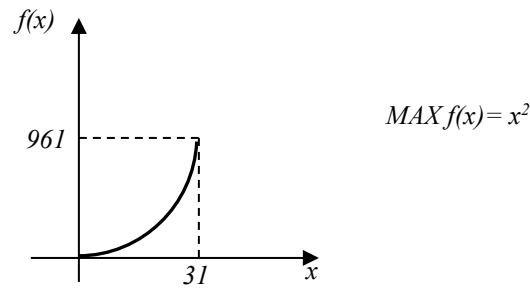


Figure 5.12. Représentation de la fonction $f(x)$

Comme première étape nous allons générer au hasard une population de n chromosome. Nous avons choisi de coder l'information en utilisant cinq gènes de 0 ou 1. Nous procédons ensuite à l'évaluation des performances de chaque chromosome en calculant la valeur de $f(x)$. La taille de la population est choisie, $n=6$. Le résultat obtenu de la première génération est montré au tableau suivant (tableau 5.1).

Chromosome					x	$f(x)=x^2$	n
0	1	1	0	0	12	144	1
1	0	0	1	1	19	361	2
1	0	0	1	1	19	361	3
0	1	0	0	0	8	64	4
1	0	0	0	1	17	289	5
1	1	0	0	0	24	576	6

Tableau 5.1. Choix aléatoire des individus

Deux chromosomes sont sélectionnés d'une façon aléatoire, pour comparer leurs performances. Nous conservons l'individu le plus performant des deux. Nous répétons cette sélection par compétition dans le but de créer la nouvelle population qui participera à la prochaine génération.

chromosome					x	$f(x)=x^2$	comparaison	n
1	0	0	1	1	19	361	2-1	1
1	1	0	0	0	24	576	6-5	2
0	1	1	0	0	12	144	1-4	3
1	0	0	1	1	19	361	3-4	4
1	0	0	0	1	17	289	5-4	5
1	0	0	0	1	17	289	5-1	6

Tableau 5.2. Résultats obtenus d'une sélection par tournoi

Alors deux chromosomes vont subir un croisement. Nous tirons aléatoirement deux parents Pn1 et Pn2. Au hasard, nous choisissons les points de croisement, ainsi les groupes de gènes à croiser et nous les accouplons. Ceci produit deux nouveaux enfants, En1 et En2.

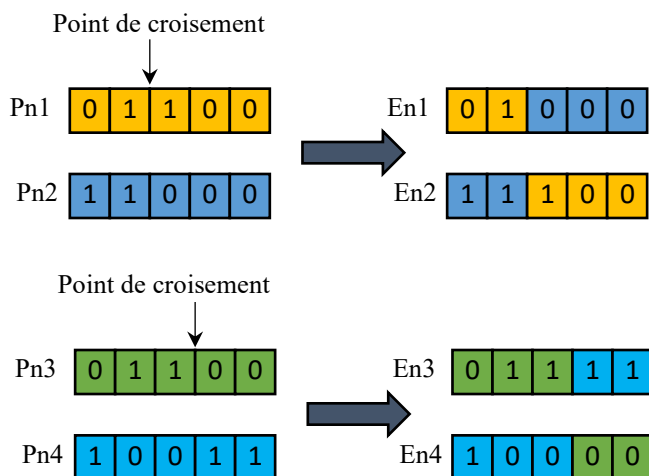


Figure 5.12. Le croisement

La mutation implique un changement aléatoire d'un gène d'un chromosome. Ceci implique un changement de 0 à 1 d'un gène choisi de façon aléatoire.

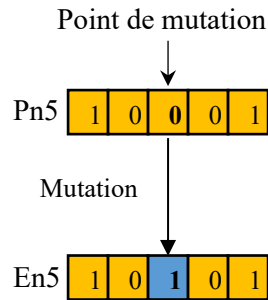


Figure 5.13. La mutation

L'élitisme correspond à transmettre l'individu le plus performant à la prochaine génération. Dans cet exemple, l'individu à $n=2$ est le plus performant alors, il est transmis directement à la prochaine génération. Maintenant, nous devons refaire une nouvelle sélection par compétition avec la nouvelle génération de solution. Les deux prochains tableaux démontrent respectivement le résultat de la première génération et le résultat d'une re-sélection par compétition.

Chromosome					x	$f(x)=x^2$	n
0	1	0	0	0	8	64	1
1	1	1	0	0	28	784	2
0	1	1	1	1	15	225	3
0	1	0	1	1	11	121	4
1	0	0	0	0	16	256	5
1	1	0	0	0	24	576	6

Tableau 5.3. Individus de la première génération

Chromosome					x	$f(x)=x^2$	Comparaison	n
1	1	1	0	0	28	784	2-1	1
1	1	0	0	0	24	576	6-4	2
1	1	1	0	0	28	784	2-4	3
0	1	1	1	1	15	225	3-4	4
1	1	1	0	0	28	784	2-3	5
1	0	0	0	0	16	256	5-1	6

Tableau 5.4. Re-sélection par tournoi

Nous accouplons aléatoirement les chromosomes Pn4 à Pn6 et Pn2 à Pn5.

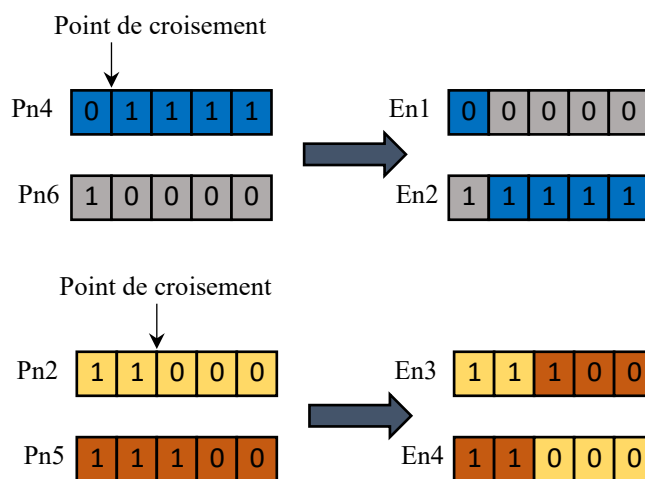


Figure 5.14. Le croisement

Nous effectuons alors la mutation au parent Pn3.

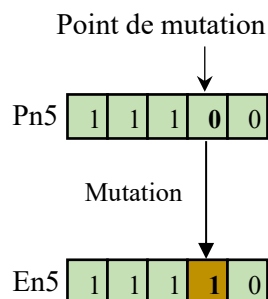


Figure 5.15. La mutation

L'individu le plus performant est Pn1, ce chromosome élite sera directement transmis à la prochaine génération. Le tableau 5.5 démontre le résultat de la sélection par tournoi effectuée pour générer la deuxième génération. Nous pouvons voir que les performances de chaque individu augmentent. C'est-à-dire, les plus faibles seront éliminés. Nous nous approchons de plus en plus vers la valeur maximale de x . Les chromosomes de la prochaine itération auront sûrement tous convergé vers la valeur maximale de 31.

Chromosome					x	$f(x)=x^2$	Comparaison	n
1	1	1	1	1	31	961	2-1	1
1	1	1	0	0	28	784	6-4	2
1	1	1	1	1	31	961	2-4	3
1	1	1	0	0	28	784	3-4	4
1	1	1	1	1	31	961	2-3	5
1	1	1	1	0	30	900	5-4	6

Tableau 5.5. Sélection par tournoi

Cet exemple nous a permis de montrer le concept de base des différentes étapes, notamment la sélection par tournoi, le croisement, la mutation et l'élitisme, de l'algorithme génétique classique.

5.4. Optimisation par Essaim de particules (PSO)

L'algorithme d'optimisation par l'essaim de particules (en anglais PSO pour Particle Swarm Optimization), est une technique d'optimisation stochastique inspirée en analogie avec le comportement des animaux lorsqu'ils se déplacent en collectif. Elle a été proposée pour la première fois par l'ingénieur en Electricité Russell Eberhart et le socio-psychologue James Kennedy en 1995. Au début Eberhart et Kennedy cherchaient à simuler le comportement des oiseaux et leurs capacités à voler d'une manière synchrone ainsi que leurs aptitudes à varier brusquement de direction tout en gardant une formation optimale, c'est ce modèle qui fut étendu par la suite à un algorithme d'optimisation.

Le PSO est basé sur le comportement social des animaux lorsqu'ils évoluent en essaim, tels que les vols des nuées d'oiseaux et les bancs de poissons. Les insectes, les poissons, les animaux, en particulier les oiseaux, etc., voyagent toujours en groupe de membres en ajustant leurs positions et vitesses à leurs informations de groupe (voir figure 5.16). Cette méthode réduit leur effort individuel pour rechercher la nourriture, un abri, ou même éviter un prédateur, etc. Dans cet algorithme, la population est appelée l'essaim et chacun de ses éléments la particule.



Figure 5.16. Exemple d'une nuée d'oiseaux (étourneaux)

5.4.1. Origine et inspiration du PSO

L'idée principale de cet algorithme, s'appuie sur les travaux de Reynolds et Heppner et Grenander, qui ont présentés des simulations de vol d'oiseaux. Reynolds était intrigué par l'esthétique de la chorégraphie d'une nuée d'oiseaux, et Heppner, zoologiste, souhaitait découvrir les règles sous-jacentes qui permettaient à un grand nombre d'oiseaux de se rassembler de manière synchrone, changeant souvent d'une manière soudaine de direction, se dispersant et se regroupant, etc. Les modèles proposés dans ces travaux reposaient fortement sur la manipulation des distances interindividuelles ; c'est-à-dire que la synchronisation du comportement des essaims était considérée comme une fonction des efforts des oiseaux pour maintenir une distance optimale entre eux et leurs voisins.

Dans une nuée d'oiseaux en vol, pour garantir la stabilité de l'essaim, les différents agents obéissent à trois règles locales :

- Cohésion : se déplacer vers le centre des voisins pour se maintenir dans l'essaim.
- Séparation : si les voisins sont trop proches, il faut s'éloigner.
- Alignement : le déplacement se fait dans la même direction que l'essaim en moyennant les directions et les vitesses des voisins.

Les trois règles citées ci-dessus, permettent la répulsion et l'attraction de chacun des individus et ainsi maintenir la cohésion de l'essaim.

Le PSO correspond donc à une population constituée d'agents simples, qu'on appelle particules. Toute particule appartenant à cette population est considérée comme une solution possible du problème, possédant une position et une vitesse. De plus, toutes les particules possèdent une mémoire, permettant ainsi à chacune de se souvenir de la meilleure performance atteinte (en position et en valeur) et de la meilleure performance réalisée par les autres particules (voisines) «informatrices» : en effet, chaque particule dispose d'un groupe de particules informatrices, appelé historiquement son voisinage.

5.4.2. Topologies de voisinage

Le voisinage représente la structure du réseau social. La topologie du voisinage définit le réseau de communication de chaque particule. Plusieurs topologies de voisinage ont été proposées et sont considérées en fonction des identificateurs des particules et non des informations topologiques comme les distances euclidiennes dans l'espace de recherche, les plus connus sont (voir figure 5.17) :

- **Topologie en anneau ou *lbest*** : cette topologie est basée sur un voisinage local. Dans cette approche, chaque particule ne partage les informations qu'avec les n voisins directs. Chaque particule tend dans son voisinage local à se déplacer vers la meilleure solution notée *lbest*. La probabilité de localiser l'optimum global est plus importante, mais la convergence de l'algorithme est lente.
- **Topologie en étoile ou *gbest*** : les particules dans cette topologie partagent les informations en utilisant une structure entièrement connectée. Ainsi, chaque particule peut recevoir les informations par la totalité des particules. Un mécanisme de voisinage global est utilisé dans ce cas, où la meilleure position trouvée par n'importe quelle particule de la population va influencer la trajectoire de recherche de chaque membre. Avec cette topologie, toutes les particules sont attirées au même temps vers la meilleure solution ce qui conduit à une convergence rapide du PSO. En revanche, si l'optimum global est loin de la meilleure particule, l'exploration n'est pas suffisante et l'algorithme risque de stagner dans un optimum local.
- **Topologie en rayon** : toutes les particules sont connectées à une seule particule centrale. Cette particule centrale peut ajuster sa position vers la meilleure, si cela donne une amélioration, l'information est propagée aux autres.

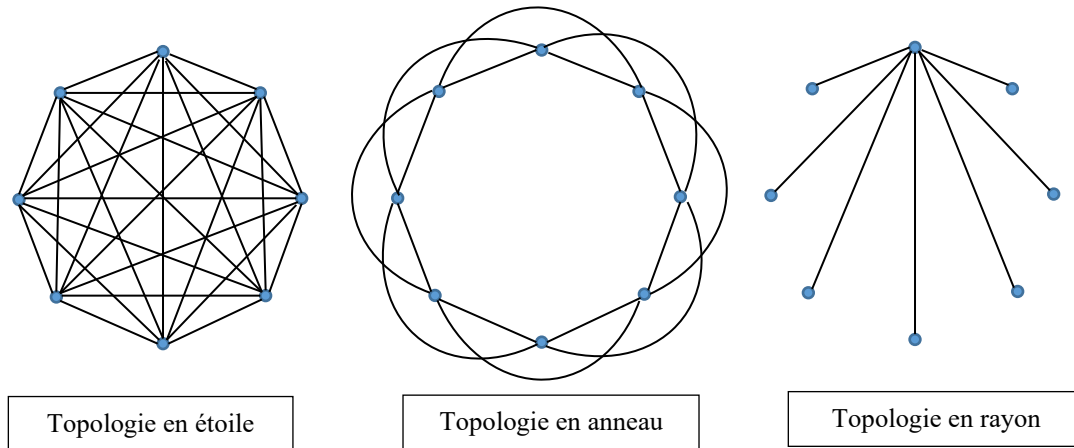


Figure 5.17. Différents types de topologie de voisinage

D'autres topologies ont été proposées dans la littérature, afin d'augmenter les performances du PSO. Ces topologies peuvent être caractérisées par rapport à deux facteurs, le premier est le degré de connectivité K représentant le nombre de voisins d'une particule, et le deuxième est le nombre de voisins d'une particule faisant partie également à d'autres voisinages. Parmi ces topologies nous pouvons citer : Von Neumann, Four-clusters, arbre, etc.

Le voisinage géographique n'est pas nécessairement pertinent, bien que ce soit le premier auquel nous sommes amenés à penser, parce que d'une part c'est un voisinage trop local, et d'autre part parce que la socialisation des particules rend tout voisinage social en voisinage géographique. En plus, en matière de calcul c'est un voisinage très lourd car à chaque itération on doit recalculer le voisinage de chaque particule.

5.4.3. Principe de fonctionnement de l'algorithme du PSO

Le principe sur lequel l'algorithme du PSO est fondé ressemble à celui d'un essaim d'oiseaux, d'une façon particulière du fait qu'un oiseau possédant une certaine capacité de mémorisation et d'analyse, lorsqu'il capte un site intéressant (source d'eau, source de nourriture, etc.), il passe cette information à certains de ces voisins qui vont la prendre en considération pour leur prochain déplacement. En effet, chaque oiseau possède individuellement une connaissance locale et une intelligence limitée. Donc, les différentes interactions entre les oiseaux qui forment l'essaim vont créer une intelligence globale. La somme des performances de l'ensemble des oiseaux est inférieure à la performance de l'essaim dont ils font partie.

Les particules dans un essaim qui représentent potentiellement des solutions au problème d'optimisation, vont parcourir l'espace de recherche afin de trouver l'optimum global. Au départ de l'algorithme, pour chaque particule la position est fixée au hasard dans l'espace de recherche, la vitesse prend une valeur aléatoire, elle est dotée d'une mémoire qui va lui permettre de mémoriser le meilleur point par lequel elle est déjà passée, et également d'un ensemble de voisines informatiques (le voisinage) pour être en courant du meilleur point atteint de son voisinage. À chaque pas, chaque particule peut faire :

- L'évaluation de la qualité sa position actuelle (fitness) qui interprète la distance à l'optimum global et mémoriser la meilleure performance atteinte jusqu'ici.
- Recevoir des informations des particules voisines et ainsi obtenir la meilleure performance de chacune d'entre elles.
- Le choix de la meilleure parmi les meilleures performances dont elle a connaissance.

Donc trois composantes peuvent influencer le déplacement d'une particule :

1. *Une composante d'inertie* : la particule à tendance à suivre sa propre direction de déplacement.
2. *Une composante cognitive* : la particule est attirée par sa propre expérience et à tendance à se diriger vers la meilleure position par laquelle elle est déjà passée.
3. *Une composante sociale* : la particule est attirée par l'expérience de ses paires et à tendance à se diriger vers la meilleure position atteinte par ses voisins.

5.4.3.1. Formalisation

Dans le cas d'un problème d'optimisation la performance de chaque particule appartenant à l'essaim est mesurée par une fonction objectif notée dans notre cas f .

Dans un espace de recherche de dimension N , chaque particule P_k est exprimée en termes des caractéristiques suivantes :

- Vecteur position $\vec{x}_k = [x_{k1}, x_{k2}, \dots, x_{kN}]$
- Vecteur vitesse $\vec{v}_k = [v_{k1}, v_{k2}, \dots, v_{kN}]$
- La meilleure position visitée par la particule k

$\vec{Pbest}_k = [Pbest_{k1}, Pbest_{k2}, \dots, Pbest_{kN}]$, à l'itération i elle est calculée comme suit :

$$\vec{Pbest}_k(i) = \begin{cases} \vec{Pbest}_k(i-1) & \text{si } f(\vec{x}_k(i)) \geq f(\vec{Pbest}_k(i-1)) \\ \vec{x}_k(i) & \text{sinon} \end{cases} \quad (5.8)$$

- La meilleure position du voisinage de la particule k

$\vec{Gbest}_k = [Gbest_{k1}, Gbest_{k2}, \dots, Gbest_{kN}]$, et calculée comme suit :

$$\vec{Gbest}_k(i) = \arg \min_{\vec{Pbest}_k} f(\vec{Pbest}_k(i)), \quad 1 \leq k \leq L \quad (5.9)$$

Avec L qui représente le nombre de particule dans l'essaim.

Au cours du processus d'optimisation chaque particule de l'essaim change sa position dans l'espace de recherche en fonction de sa dernière position, sa dernière vitesse, sa meilleure position trouvée au cours des itérations passées $Pbest$ et la meilleure position trouvée par l'essaim $Gbest$, à chaque itération la vitesse et la position sont mises à jour en utilisant les équations suivantes :

$$v_k(i) = w * v_k(i-1) + r_1 \varphi_1 * (Pbest_k - x_k(i-1)) + r_2 \varphi_2 * (Gbest - x_k(i-1)) \quad (5.10)$$

$$x_k(i) = x_k(i-1) + v_k(i) \quad (5.11)$$

Où w est une constante, nommée coefficient d'inertie, r_1 et r_2 sont des nombres aléatoires compris entre 0 et 1 et φ_1 et φ_2 sont des constantes, nommés coefficients d'accélération, qui sont positives

et généralement sont comprises entre 0 et 2. En se basant sur ces relations le principe de déplacement d'une particule est illustré par la figure 5.18.

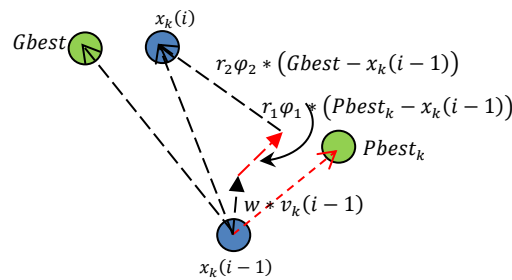


Figure 5.18. Principe de déplacement d'une particule

$w * v_k(i-1)$ correspond à la composante d'inertie introduite par Shi et Eberhart. Le paramètre w contrôle l'impact des directions de déplacement sur les déplacements futures. Une petite valeur de w favorise une recherche locale (l'exploitation), tandis qu'une grande valeur favorise une recherche globale (l'exploration).

$r_1 \phi_1 * (Pbest_k - x_k(i-1))$ correspond à la composante cognitive, qui représente l'attraction linéaire de la particule k vers la meilleure position déjà trouvée.

$r_2 \phi_2 * (Gbest - x_k(i-1))$ correspond à la composante sociale, qui représente l'attraction linéaire de la particule k vers la meilleure position trouvée par le voisinage de cette particule.

La structure d'un algorithme PSO présentée ci-dessous peut aussi être représentée sous forme de l'organigramme présenté dans la figure 5.19.

Algorithme 5.2 : PSO

Début

Initialiser aléatoirement les positions et les vitesses de chaque particule

Pour chaque particule P_k , $Pbest_k = x_k$

Tant que le critère d'arrêt n'est pas satisfait **faire**

Pour $k=1$ à L **faire**

 Déplacement de la particule P_k selon eq 5.10 et eq 5.11

 Evaluer les position

Si $f(x_k) < f(Pbest_k)$ **alors**

$Pbest_k = x_k$

Fin si

Si $f(Pbest_k) < f(Gbest)$ **alors**

$Gbest = Pbest_k$

|
 |
 | Fin si
 | Fin pour
 | Fin Tant que
 |
 Fin

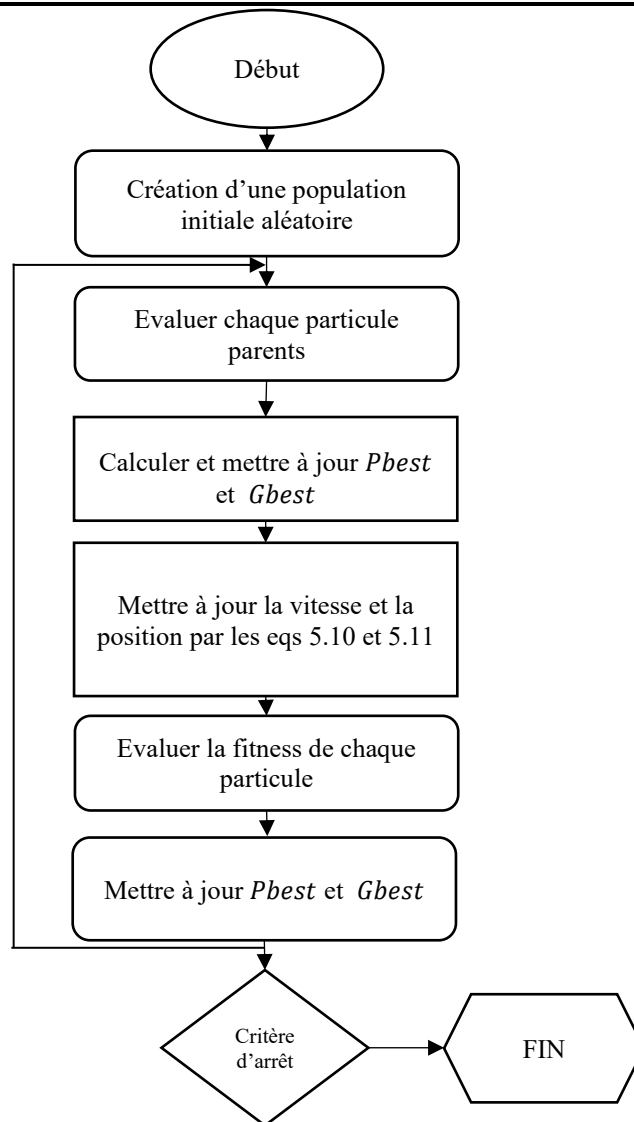


Figure 5.19. Organigramme d'un algorithme PSO

5.4.3.2. Choix des paramètres de l'algorithme

Les paramètres de l'algorithme PSO ont une grande influence sur la conduite des particules et donc sur la convergence de l'algorithme ; et même si des résultats satisfaisants sont obtenus par cette méthode, un bon choix des paramètres de la méthode demeure comme un point critique pour s'assurer du succès de l'algorithme PSO.

5.4.3.2.1. Nombre de particules L

Le nombre de particules réservées pour résoudre un problème d'optimisation est dépendant principalement de la taille de l'espace de recherche. Il n'existe pas de règle fixe pour la détermination de ce paramètre, pour avoir une bonne valeur il faut faire plusieurs essais.

Généralement, augmenter le nombre de particules va augmenter la diversité, ce qui minimise le risque d'être orienter vers des minimums locaux. La valeur attribuée à L ne doit pas être trop petite selon.

5.4.3.2.2. L'initialisation de l'algorithme

L'initialisation des positions des particules est aléatoire, mais d'une manière à couvrir tout l'espace de recherche. Il faut noter que l'algorithme du PSO peut trouver des difficultés pour localiser l'optimum, si les positions des particules ne couvrent pas initialement tout l'espace de recherche, de risque que cet optimum se trouve à l'extérieur de la zone initiale.

5.4.3.2.3. Coefficient d'inertie w

Le coefficient d'inertie w a été introduit par Shi et Eberhart pour contrôler l'influence des directions actuelles des particules de l'essaim sur les déplacements futur de ces derniers. Ce paramètre permet d'avoir un compromis entre l'exploration (la recherche globale) et l'exploitation (la recherche locale). Une petite valeur du coefficient d'inertie mène à une recherche locale sur un espace réduit tandis qu'une grande valeur provoque une exploration étendue de l'espace de recherche.

Les meilleures valeurs de w pour avoir une meilleure convergence selon l'étude de sont dans l'intervalle $[0.8, 1.2]$. Au-delà des valeurs indiquées dans l'intervalle l'algorithme peut avoir des difficultés pour converger.

5.4.3.2.4. Coefficients d'accélération φ_1 et φ_2

φ_1 et φ_2 sont deux constantes positives, qui contrôlent l'attraction vers la meilleure position de soi-même et l'attraction vers la meilleure position globale respectivement. D'une manière empirique elles sont déterminées suivant la relation $\varphi_1 + \varphi_2 \leq 4$. Combiner les trois coefficients w , φ_1 et φ_2 permet de régler la balance entre une recherche globale et locale.

5.4.3.2.5. Critère d'arrêt

Différents critères d'arrêts existent pour terminer l'exécution de l'algorithme PSO :

- **Convergence vers la solution recherchée** : un seuil ϵ est souvent utilisé, si la valeur de ϵ est trop petite la recherche est lente, si par contre elle est trop grande les solutions trouvées sont mauvaises.
- **Nombre maximum d'itération** : fixer un nombre maximum d'itération à l'avance. Si ce nombre est petit les solutions générées par l'algorithme ne sont pas optimales.

- **Aucune amélioration/nombre d'itération** : plusieurs façons existent pour savoir si la recherche s'améliore ou pas, à titre d'exemple la variation des positions des particules est trop petite ou la variation de la vitesse est proche de zéro.

5.5. Conclusion

A travers ce chapitre, quelques méthodes d'optimisation ont été présentées en se basant sur les caractéristiques fondamentales des métaheuristiques. Ces dernières sont réputées d'être efficace dans la résolution des problèmes d'optimisation difficiles sans avoir recours à la modification de l'architecture de base de l'algorithme utilisé. Ces dernières années elles sont devenues très sollicitées par les scientifiques à cause de leurs simplicités d'emploi dans divers domaines. Malgré leur succès remarquable, les métaheuristiques souffrent néanmoins de quelques difficultés auxquelles sont confrontés les utilisateurs dans le cas d'un problème concret, tel que le choix d'une approche efficace pour trouver la solution optimale et le réglage des paramètres. Parmi les métaheuristiques qui existent dans la littérature notre intérêt s'est orienter vers deux méthodes à savoir les algorithmes génétiques et l'optimisation par l'essaim de particule (PSO).

L'algorithme génétique est une technique d'intelligence artificielle permettant la résolution des problèmes d'optimisation par une approche itérative qui impose une série de solutions aléatoires se rapprochant d'une manière progressive de la solution réelle du problème. Cette technique est basée sur le principe de l'évolution naturelle énoncé par Darwin, c'est-à-dire la capacité des espèces à survivre et à s'adapter à leur environnement par des processus naturels comme le croisement et la mutation. Mais les algorithmes génétiques ont un désavantage du point de vue temps de calcul pour les problèmes ayant des fonctions objectifs complexes à plusieurs variables. Il est parfois nécessaire d'utiliser une population d'individu assez grande et un nombre important de générations afin de s'assurer que la solution trouvée est optimale à cause de la nature aléatoire de l'algorithme.

Malgré sa récente apparition, l'algorithme PSO inspiré particulièrement du comportement social des animaux qui évoluent en essaim, a connu un intérêt croissant ces dernières années. Ceci est dû aux avantages qu'offre cet algorithme : une excellente robustesse, une facilité dans la réalisation d'un calcul parallèle, une convergence rapide vers la valeur optimale et peut s'hybrider avec d'autres techniques pour accroître ses performances.

6.1. Introduction

La connaissance acquise n'est pas toujours certaine (établie à 100%) pour permettre au système de prendre la décision la plus appropriée. L'incertitude est une connaissance dont on ne connaît pas les propriétés quantitatives et qui n'a pas une représentation arithmétique ou logique formelle. La modélisation de l'incertitude en Intelligence Artificielle implique l'élaboration d'un formalisme et d'une méthode pour la représenter et la manipuler. L'existence de l'information incertaine s'explique notamment par le manque de précision éventuel des données fournies par l'utilisateur.

Parmi les raisons qui nous pousse à introduire de l'incertitude :

- Conclusions disjonctives (comme dans le cas du diagnostic médical).
- Modélisation qualitative
- Informations insuffisantes

Les approches probabilistes s'adaptent mieux au raisonnement avec la connaissance (ou croyance) incertaine et à la structure de la représentation de la connaissance.

La théorie des probabilités traite les incertitudes qui sont stochastiques concernant l'occurrence de certains événements. Cette incertitude est donnée ou mesurée ou évaluée par un degré de probabilité.

La probabilité conditionnelle conduit au théorème de Bayes. Les réseaux dits Bayésiens utilisent ce théorème pour trouver de nouvelles connaissances à partir de données préalables. Ces réseaux sont appelés aussi réseaux de croyance (belief network).

Les réseaux Bayésiens sont actuellement une des techniques les plus intéressantes de l'intelligence artificielle car ils allient la lisibilité d'une représentation de la connaissance par un graphe causal intuitif à l'efficacité d'une représentation « distribuée » des données qui tient compte de l'incertitude dans le raisonnement. Ils sont utilisés dans bon nombre d'applications.

Nous définissons, dans ce chapitre, quelques notions de base sur le raisonnement probabiliste et les réseaux Bayésiens.

6.2. Raisonnement probabiliste

Commençons par un exemple illustrant la notion de l'incertitude.

Soit A_t l'action d'aller à l'université t minute avant le commencement du cours.

- ❖ A_t me permettra-t-il d'arriver à temps pour assister au cours ?

Problèmes :

On peut identifier plusieurs problèmes qui sont essentiellement liés à l'incertitude, tel que :

- Observabilité partielle (conditions routières, etc.)
- Senseurs bruités (annonces du trafic, etc.)
- Incertitude dans l'effet des actions (bus en pannes, crevaisons, etc.)
- Immense complexité pour modéliser les actions et le trafic.

Un raisonnement purement logique et déterministe :

- ❖ Risque de tirer des conclusions erronées :
 - « A_{20} me permettra d'arriver à temps » (pas de garantie)
- ❖ Risque de tirer des conclusions peu exploitables du point de vue de la prise de décision :
 - « A_{20} me permettra d'arriver à temps, s'il ne pleut pas, si le bus ne tombe pas en panne, si mes pneus ne crèvent pas, etc. »
 - « A_{1440} me permettra presque certainement d'arriver à temps, mais je devrai passer une nuit devant les portes de l'université. »

6.2.1. Théorie des probabilités en intelligence artificielle

La théorie des probabilités sera utilisée comme un outil pour faire du raisonnement dans un contexte où on a un environnement incertain ou stochastique. Donc, elle permet de modéliser des connaissances ayant de l'incertitude (vraisemblance d'événements).

L'information sur la vraisemblance est dérivée :

- ❖ Des croyances/certitudes d'un agent, ou
- ❖ D'observations empiriques de ces événements

Donne un cadre théorique pour :

- ❖ **Mettre à jour les connaissances** (vraisemblance d'événements) après l'acquisition d'observations
- ❖ **Prendre de décisions** basées sur des connaissances avec incertitude

Facilite la modélisation en permettant de considérer l'influence de phénomènes complexes comme du « bruit ».

6.2.2. Rappels sur des notions de probabilités

Le triplet (Ω, Z, P) , appelé espace probabilisé, est formé de l'ensemble Ω appelé univers, d'une tribu Z sur Ω et d'une mesure de probabilité P sur Z dans $[0,1]$. Les éléments de Z sont appelés événements. Le nombre $P(A)$ est appelé la probabilité de l'événement $A \in Z$.

Si deux événements A et B sont incompatibles ou mutuellement exclusifs, alors $P(A \cup B) = P(A) + P(B)$ et $P(A \cap B) = 0$. Par contre, si A et B sont deux événements quelconques, alors $P(A \cup B) = P(A) + P(B) - P(A \cap B)$.

On dit que deux événements A et B sont indépendants si et seulement si :

$$P(A \cap B) = P(A).P(B).$$

Si B est une conséquence logique de A i.e. $A \subset B$ ou $B \Leftrightarrow A$, alors $P(A) \leq P(B)$ et $P(B \setminus A) = P(B) - P(A)$.

Si A et B sont deux événements quelconques avec $P(B) > 0$, la probabilité de A conditionnellement à B est noté $P(A|B)$ et vaut :

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad (6.1)$$

Dans le cas où les événements A et B sont indépendants et $P(B) > 0$, alors $P(A|B) = P(A)$. D'après la définition de la probabilité conditionnelle on a :

$$P(A \cap B) = P(A|B).P(B) = P(B|A).P(A)$$

Et donc

$$P(A|B) = \frac{P(B \cap A)}{P(B)} \quad (6.2)$$

Le théorème de Bayes (1763) découle de la généralisation de l'équation 6.2 à des ensembles d'événements A et B :

$$P(A|B) = \frac{P(B|A).P(A)}{\sum_a P(B|A=a)*P(A=a)} \quad (6.3)$$

Dans la suite, on note $P(A, B)$ au lieu de $P(A \cap B)$.

6.2.3. Exemple illustratif

Pour mieux comprendre ces définitions nous allons utiliser un exemple de détection de pourriels.

On souhaite raisonner, à partir des connaissances, sur la possibilité qu'un **courriel** soit un **pourriel** tenant compte de l'incertitude associée à une telle classification.

Pour ce faire, notre modèle (« base de connaissances») est une distribution conjointe des probabilités de **variables aléatoires** :

- ❖ **Inconnu** : l'adresse de l'expéditeur n'est pas connue du destinataire
- ❖ **Mot_sensible** : le courriel contient un mot sensible
- ❖ **Pourriel** : le courriel est un pourriel

Inconnu	Mot_sensible	Pourriel	Probabilité
vrai	vrai	vrai	0.108
vrai	vrai	faux	0.016
vrai	faux	vrai	0.012
vrai	faux	faux	0.064
faux	vrai	vrai	0.072
faux	vrai	faux	0.144
faux	faux	vrai	0.008
faux	faux	faux	0.576

Tableau 6.1. Probabilités qu'un courriel soit un pourriel

Chaque rangée du tableau 6.1 correspond à un état possible de mon environnement, et on va avoir pour chaque état possible une probabilité associée (on suppose que ces probabilités sont définies à priori). Si on prend le cas de la première rangée nous aurons :

La probabilité que *Inconnu=vrai et Mot_sensible=vrai et Pourriel=vrai* est égale à **0.108** ou **10.8%**.

Remarque : Toutes ces probabilités somment à 1 et sont entre 0 et 1.

- En théorie de probabilité on a la notion d'un **événement élémentaire**, on le note par le symbole ω , et c'est l'état possible de l'environnement. C'est une rangée du tableau ??, un événement au niveau le plus simple (par exemple : *Inconnu=vrai et Mot_sensible=vrai et Pourriel=vrai*).

- on définit aussi l'**univers** Ω comme étant l'ensemble des événements élémentaires possibles. Dans notre cas c'est l'ensemble des toutes les rangées.

- on peut définir une **variable aléatoire** comme une fonction d'un événement élémentaire ω (exemple : Inconnu est vrai si ω est un état où l'expéditeur du courriel reçu n'est pas connu). On pourrait définir des variables plus complexes, c.à.d. des variables impliquant plusieurs aspects de l'état. Souvent, on définit les variables aléatoires individuelles avant l'état ω (on définit alors ω comme étant une assignation de toutes ces variables. Une variable aléatoire joue alors le rôle d'une fenêtre sur l'état de l'environnement.

Dans ce cours on va se concentrer sur des variables aléatoires Booléennes ou Binaires (le domaine, c-à-d. l'ensemble des valeurs possibles de la variable, était toujours « vrai, faux »).

On pourrait avoir d'autres types de variables, avec des domaines différents :

- **Discrètes** : le domaine est énumérable
- ♦ Météo $\in$ {soleil, pluie, nuageux, neige}
- ♦ Lorsqu'on marginalise, on doit sommer sur toutes les valeurs :

$$P(\text{Température}=X) = \sum_{y \in \{\text{soleil, pluie, nuageux, neige}\}} P(\text{Température}=X, \text{Météo}=y)$$

- **Continues** : le domaine est continu (par exemple, l'ensemble des réels)
- ♦ Exemple : PositionX=4.2
- ♦ Le calcul des probabilités marginales nécessite des intégrales.

 **Probabilité conjointe** : probabilité d'une assignation de valeurs à toutes les variables.

- $P(\text{Inconnu=vrai}, \text{Mot_sensible=vrai}, \text{Pourriel=vrai}) = 0.108(10.8\%)$
- $P(\text{Inconnu=faux}, \text{Mot_sensible=faux}, \text{Pourriel=vrai}) = 0.008(0.8\%)$

 **Probabilité marginale** : probabilité sur un sous-ensemble des variables.

- $P(Y) = \sum_z P(Y, Z=z)$ Pour n'importe quel ensemble de variable **Y** et **Z**
- $P(\text{Inconnu=vrai}, \text{Pourriel=vrai})$

$$= P(\text{Inconnu=vrai}, \text{Mot_sensible=vrai}, \text{Pourriel=vrai}) + P(\text{Inconnu=vrai}, \text{Mot_sensible=faux}, \text{Pourriel=vrai})$$

$$= \sum_{x \in \{\text{vrai}, \text{faux}\}} P(\text{Inconnu}=x, \text{Mot_sensible=vrai}, \text{Pourriel=vrai}) = 0.108 + 0.012 = \mathbf{0.12}$$

- $P(\text{Pourriel=vrai})$

$$= \sum_{x \in \{\text{vrai}, \text{faux}\}} \sum_{y \in \{\text{vrai}, \text{faux}\}} P(\text{Inconnu}=x, \text{Mot_sensible}=y, \text{Pourriel=vrai})$$

$$= 0.108 + 0.012 + 0.072 + 0.008 = \mathbf{0.2}$$

✚ Probabilité de disjonction (« ou ») d'événements :

- Formule générale : $P(X \text{ ou } Y) = P(X) + P(Y) - P(X \text{ et } Y)$
- $P(\text{Pourriel}=\text{vrai} \text{ ou } \text{Inconnu}=\text{faux})$ Six états (mondes) possibles

$$= 0.108 + 0.012 + 0.072 + 0.008 + 0.144 + 0.576$$

$$= \mathbf{0.92}$$

- $P(\text{Pourriel}=\text{vrai} \text{ ou } \text{Inconnu}=\text{faux})$ Une autre façon de le calculer

$$= P(\text{Pourriel}=\text{vrai}) + P(\text{Inconnu}=\text{faux}) - P(\text{Pourriel}=\text{vrai}, \text{Inconnu}=\text{faux})$$

$$= 1 - P(\text{Pourriel}=\text{faux}, \text{Inconnu}=\text{vrai}) = 1 - 0.016 - 0.064$$

$$= \mathbf{0.92}$$

✚ Probabilité d'un événement arbitraire : On peut calculer la probabilité d'événements arbitrairement complexes, il suffit d'additionner les probabilités des événements élémentaires associés.

- $P((\text{Pourriel}=\text{vrai}, \text{Inconnu}=\text{faux}) \text{ ou } (\text{Mot_sensible}=\text{faux}, \text{Pourriel}=\text{faux}))$

$$= 0.072 + 0.008 + 0.064 + 0.576 = \mathbf{0.72}$$

✚ Probabilité conditionnelle :

- Formule générale : $P(X/Y) = P(X, Y) / P(Y)$ si $P(Y) \neq 0$
- $P(\text{Pourriel}=\text{faux} | \text{Inconnu}=\text{vrai})$

$$= P(\text{Pourriel}=\text{faux}, \text{Inconnu}=\text{vrai}) / P(\text{Inconnu}=\text{vrai})$$

$$= (0.016 + 0.064) / (0.016 + 0.064 + 0.108 + 0.012) = \mathbf{0.4}$$

✚ Distribution de probabilités : l'énumération des probabilités pour toutes les valeurs possibles de variables aléatoires

- $P(\text{Pourriel}, \text{Inconnu}, \text{Mot_sensible})$
- $P(\text{Pourriel}) = [P(\text{Pourriel}=\text{faux}), P(\text{Pourriel}=\text{vrai})] = [0.8, 0.2]$
- $P(\text{Pourriel}, \text{Inconnu})$

$$= [[P(\text{Pourriel}=\text{faux}, \text{Inconnu}=\text{faux}), P(\text{Pourriel}=\text{vrai}, \text{Inconnu}=\text{faux})],$$

$$[P(\text{Pourriel}=\text{faux}, \text{Inconnu}=\text{vrai}), P(\text{Pourriel}=\text{vrai}, \text{Inconnu}=\text{vrai})]]$$

La somme est toujours égale à 1

On utilise le symbole **P** pour les distributions et P pour les probabilités ($P(\text{Pourriel})$ désignera la probabilité $P(\text{Pourriel}=x)$ pour une valeur x non-spécifiée, et c'est un élément quelconque de $P(\text{Pourriel})$).

✚ Distribution conditionnelle : on peut faire la même chose pour le cas conditionnel. Une distribution conditionnelle peut être vue comme une distribution renormalisée afin de satisfaire les conditions de sommation à 1.

- $P(\text{Pourriel} | \text{Inconnu}=\text{faux})$

$$= [P(\text{Pourriel}=\text{faux} | \text{Inconnu}=\text{faux}), P(\text{Pourriel}=\text{vrai} | \text{Inconnu}=\text{faux})] = [0.9, 0.1]$$

- $P(\text{Pourriel} | \text{Inconnu})$

$$= [[P(\text{Pourriel}=\text{faux} \mid \text{Inconnu}=\text{faux}), P(\text{Pourriel}=\text{vrai} \mid \text{Inconnu}=\text{faux}), [P(\text{Pourriel}=\text{faux} \mid \text{Inconnu}=\text{vrai}), P(\text{Pourriel}=\text{vrai} \mid \text{Inconnu}=\text{vrai})]]]$$

$$= [[0.9, 0.1], [0.4, 0.6]]$$

Chaque sous-ensemble de probabilités associé aux mêmes valeurs des variables sur lesquelles on conditionne somme à 1

$P(\text{Pourriel} \mid \text{Inconnu})$ contient deux distributions de probabilités sur la variable Pourriel : une dans le cas où Inconnu=faux, l'autre lorsque Inconnu=vrai

✚ Règle du produit :

- $P(X,Y)=P(X|Y)P(Y)$
- $P(\text{Pourriel}=\text{faux}, \text{Inconnu}=\text{vrai})$

$$= P(\text{Pourriel}=\text{faux} \mid \text{Inconnu}=\text{vrai}) P(\text{Inconnu}=\text{vrai})$$

$$= P(\text{Inconnu}=\text{vrai} \mid \text{Pourriel}=\text{faux}) P(\text{Pourriel}=\text{faux})$$

- En général :

$$P(\text{Pourriel}, \text{Inconnu}) = P(\text{Pourriel} \mid \text{Inconnu}) P(\text{Inconnu})$$

$$= P(\text{Inconnu} \mid \text{Pourriel}) P(\text{Pourriel})$$

✚ Règle de chaînage (chain rule) pour n variables $X_1 \dots X_n$:

- $P(X_1, \dots, X_n) = P(X_1, \dots, X_{n-1}, X_n)$

$$= P(X_1, \dots, X_{n-1}) P(X_n \mid X_1, \dots, X_{n-1})$$

$$= P(X_1, \dots, X_{n-2}) P(X_{n-1} \mid X_1, \dots, X_{n-2}) P(X_n \mid X_1, \dots, X_{n-1})$$

$$= \dots$$

$$= \prod_{i=1..n} P(X_i \mid X_1, \dots, X_{i-1})$$

- La règle du chaînage est vraie, quelle que soit la distribution de $X_1 \dots X_n$
- ♦ Plutôt que de spécifier toutes les probabilités jointes $P(X_1, \dots, X_n)$, on pourrait plutôt spécifier $P(X_1), P(X_2 \mid X_1), P(X_3 \mid X_1, X_2), \dots, P(X_n \mid X_1, \dots, X_{n-1})$
- Exemple, on aurait pu spécifier :
- ♦ $P(\text{Pourriel}=\text{faux}) = 0.8, P(\text{Pourriel}=\text{vrai}) = 0.2$
- ♦ $P(\text{Inconnu}=\text{faux} \mid \text{Pourriel}=\text{faux}) = 0.9, P(\text{Inconnu}=\text{vrai} \mid \text{Pourriel}=\text{faux}) = 0.1$
 $P(\text{Inconnu}=\text{faux} \mid \text{Pourriel}=\text{vrai}) = 0.4, P(\text{Inconnu}=\text{vrai} \mid \text{Pourriel}=\text{vrai}) = 0.6$
- On aurait tous les ingrédients pour calculer les $P(\text{Pourriel}, \text{Inconnu})$:
- ♦ $P(\text{Pourriel}=\text{faux}, \text{Inconnu}=\text{vrai}) = P(\text{Inconnu}=\text{vrai} \mid \text{Pourriel}=\text{faux}) P(\text{Pourriel}=\text{faux})$

$$= 0.1 * 0.8 = 0.08$$

- ♦ $P(\text{Pourriel}=\text{vrai}, \text{Inconnu}=\text{vrai}) = P(\text{Inconnu}=\text{vrai} \mid \text{Pourriel}=\text{vrai}) P(\text{Pourriel}=\text{vrai})$

$$= 0.6 * 0.2 = 0.12$$

✚ Règle de Bayes :

- $P(X|Y) = P(Y|X)P(X)/P(Y)$
- Donne une probabilité diagnostique à partir d'une probabilité causale :
- ♦ $P(\text{Cause}|\text{Effet}) = P(\text{Effet}|\text{Cause}) P(\text{Cause}) / P(\text{Effet})$
- On pourrait calculer $P(\text{Pourriel}=\text{faux} \mid \text{Inconnu}=\text{vrai})$:
- ♦ $P(\text{Pourriel}=\text{faux} \mid \text{Inconnu}=\text{vrai}) = P(\text{Pourriel}=\text{faux}, \text{Inconnu}=\text{vrai}) / P(\text{Inconnu}=\text{vrai})$

$$\begin{aligned}
&= P(\text{Inconnu}=\text{vrai} \mid \text{Pourriel}=\text{faux}) P(\text{Pourriel}=\text{faux}) / P(\text{Inconnu}=\text{vrai}) \\
&= \frac{P(\text{Inconnu}=\text{vrai} \mid \text{Pourriel}=\text{faux}) P(\text{Pourriel}=\text{faux})}{P(\text{Inconnu}=\text{vrai}, \text{Pourriel}=\text{faux}) + P(\text{Inconnu}=\text{vrai}, \text{Pourriel}=\text{vrai})} \\
&= 0.08 / (0.08 + 0.12) = 0.4
\end{aligned}$$

- On appelle $P(\text{Pourriel})$ une probabilité a priori
- ♦ C'est notre croyance p/r à la présence d'un Pourriel avant toute observation
- On appelle $P(\text{Pourriel}|\text{Inconnu})$ une probabilité a posteriori
- ♦ C'est notre croyance mise à jour après avoir observé que l'auteur du courriel est Inconnu
- La règle de Bayes lie ces deux probabilités ensemble.

 **Indépendance** : c'est le concept d'indépendance entre les variables aléatoires.

- Soit les variables A et B, elles sont indépendantes si et seulement si
- ♦ $P(A|B) = P(A)$ ou
- ♦ $P(B|A) = P(B)$ ou
- ♦ $P(A, B) = P(A) P(B)$
- Exemple : $P(\text{Pluie}, \text{Pourriel}) = P(\text{Pluie}) P(\text{Pourriel})$

Pluie	Pourriel	Probabilité
vrai	vrai	0.03
vrai	faux	0.27
faux	vrai	0.07
faux	faux	0.63

$P(\text{Pluie} = \text{vrai}) = 0.3$ et $P(\text{Pourriel} = \text{vrai}) = 0.1$

- L'indépendance totale est puissante mais rare
- L'indépendance entre les variables permet de réduire la taille de la distribution de probabilités et rendre les inférences plus efficaces
- ♦ Dans l'exemple précédent, on n'a qu'à stocker en mémoire $P(\text{Pluie} = \text{vrai}) = 0.3$ et $P(\text{Pourriel} = \text{vrai}) = 0.1$, plutôt que la table au complet
- Mais il est rare d'être dans une situation où toutes les variables sont réellement indépendantes

 **Indépendance conditionnelle** :

- Si je sais déjà que le courriel est un pourriel, ma croyance (probabilité) qu'il contienne un mot sensible ne dépend plus du fait que l'expéditeur me soit inconnu ou non :
- ♦ $P(\text{Mot_sensible} \mid \text{Inconnu}, \text{Pourriel}=\text{vrai}) = P(\text{Mot_sensible} \mid \text{Pourriel}=\text{vrai})$
- On dit que *Mot_sensible* est conditionnellement indépendante de Inconnu étant donné *Pourriel*, puisque :
- ♦ $P(\text{Mot_sensible} \mid \text{Inconnu}, \text{Pourriel}) = P(\text{Mot_sensible} \mid \text{Pourriel})$
- Formulations équivalentes :
- ♦ $P(\text{Inconnu} \mid \text{Mot_sensible}, \text{Pourriel}) = P(\text{Inconnu} \mid \text{Pourriel})$

$$\diamond P(\text{Inconnu}, \text{Mot_sensible} \mid \text{Pourriel}) = P(\text{Inconnu} \mid \text{Pourriel}) P(\text{Mot_sensible} \mid \text{Pourriel})$$

6.3. Les réseaux bayésiens

Les réseaux bayésiens sont la combinaison des approches probabilistes et de la théorie de graphes. Autrement dit, ce sont des modèles qui permettent de représenter des situations de raisonnement probabiliste à partir de connaissances incertaines. Ils sont une représentation efficace pour les calculs d'une distribution de probabilités, de plus ils constituent une technique d'acquisition, de représentation et de manipulation de connaissances, et on les utilise pour leur capacité d'effectuer des inférences dans un contexte d'incertitude et aussi pour leurs algorithmes d'apprentissage afin de prévoir, contrôler et simuler le comportement d'un système, analyser des données et prendre des décisions.

6.3.1. Définition

Un réseau bayésien (RB) est un système représentant la connaissance et permettant de calculer des probabilités conditionnelles apportant des solutions à différentes sortes de problématiques.

Les réseaux bayésiens sont initialement connus comme « réseaux de relations », mais sont nommés "Bayes" après la mort de Révérend Thomas Bayes, 1702-1761, théologien et mathématicien britannique qui a écrit une loi de base de probabilité, maintenant appelée le théorème de Bayes. Les RB sont basés sur ce théorème, ce sont des modèles qui permettent de représenter des situations de raisonnement probabilistes exprimés par la formule :

$$P(A|B) = \frac{P(B|A).P(B)}{P(A)} \quad (6.4)$$

Ainsi, les RB associent une partie qualitative que sont les graphes et une partie quantitative représentant les probabilités conditionnelles associées à chaque nœud du graphe relativement au parent. La partie qualitative exprime des indépendances conditionnelles entre variables et des liens de causalités et ce grâce à un graphe orienté acyclique dont les nœuds correspondent à des variables aléatoires. La partie quantitative est constituée de tables de probabilités dans le cas discret ou de distribution gaussiennes dans le cas continu. Un réseau bayésien $B = \{G, P\}$ est donc défini par un graphe dirigé, un espace probabiliste est un ensemble de variables aléatoires. Le graphe est sans circuit $G = (X, E)$ où X est l'ensemble des nœuds (ou sommets) et E , l'ensemble des arcs. L'espace probabiliste est tel que (Ω, P) où Ω est l'univers des probabilités et P l'ensemble de variables aléatoires $X = \{X_1, \dots, X_n\}$ associées aux nœuds du graphe et tel que:

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i | Pa(X_i)) \quad (6.5)$$

Où $Pa(X_i)$ est l'ensemble des parents du nœud X_i dans G

Ainsi, les réseaux bayésiens sont la combinaison des approches probabilistes et la théorie de graphes. Autrement dit, ce sont des modèles qui permettent de représenter des situations de raisonnement probabiliste à partir de connaissances incertaines.

La structure de ce type de réseau est représentée par un **Graphe orienté acyclique** (DAG, Directed Acyclic Graph) constitué de :

- **Nœuds** : représentant l'ensemble des variables aléatoires,

- **Arcs orientés** : représentant des dépendances (causalités) probabilistes entre les variables et des distributions de probabilités conditionnelles pour chaque variable étant donné ses parents.
- une **table de probabilité** pour chaque nœud.

Notons que le graphe est **acyclique** : il ne contient pas de boucle. Les arcs représentent des relations entre variables qui sont soit déterministes, soit probabilistes. Ainsi, l'observation d'une ou plusieurs causes n'entraîne pas systématiquement l'effet ou les effets qui en dépendent, mais modifie seulement la probabilité de les observer.

6.3.2. Exemple illustratif des RB

Afin de mieux comprendre le concept, nous avons instauré un exemple qui confèrera au modèle bayésien une image plus réelle. Considérons la situation suivante :

- Je suis au travail, et mes voisins Marie et Jean m'ont promis de m'appeler chaque fois que mon alarme sonne
- Mon voisin Jean m'appelle pour me dire que mon alarme sonne
» Parfois il confond l'alarme avec la sonnerie du téléphone
- Par contre ma voisine Marie ne m'appelle pas toujours
» Parfois elle met la musique trop fort
- Parfois mon alarme se met à sonner lorsqu'il y a de légers séismes
- Comment conclure qu'il y a ou non un cambriolage chez moi ?

On peut représenter ce problème par un RB sachant que :

 **Variables aléatoires** : on va commencer par définir les nœuds de notre graphe, donc définir les variables aléatoires qui sont :

- ◆ Cambriolage
- ◆ Séisme
- ◆ Alarme
- ◆ Jean_Appelle
- ◆ Marie_Appelle

Ensuite on va introduire des arcs entre les nœuds pour obtenir un graphe (figure 6.1). La topologie du RB modélise les relations de causalité.

- ◆ Un cambriolage peut déclencher l'alarme
- ◆ Un séisme aussi
- ◆ L'alarme peut inciter Jean à appeler
- ◆ Idem pour Marie

Un arc d'un nœud X vers un nœud Y signifie que la variable X influence la variable Y.

- ◆ X est appelé le parent de Y
- ◆ Parents(Y) est l'ensemble des parents de Y

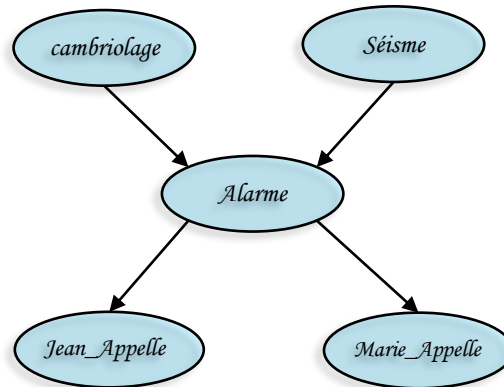


Figure 6.1. Exemple d'un graphe

Une table de probabilités conditionnelles (TPC) donne la probabilité pour chaque valeur du nœud étant donnée les combinaisons des valeurs des parents du nœud (c'est l'équivalent d'une distribution) (voir figure 6.2).

- Si X n'a pas de parents, sa distribution de probabilités est dite inconditionnelle ou a priori.
- Si X a des parents, sa distribution de probabilités est dite conditionnelle.

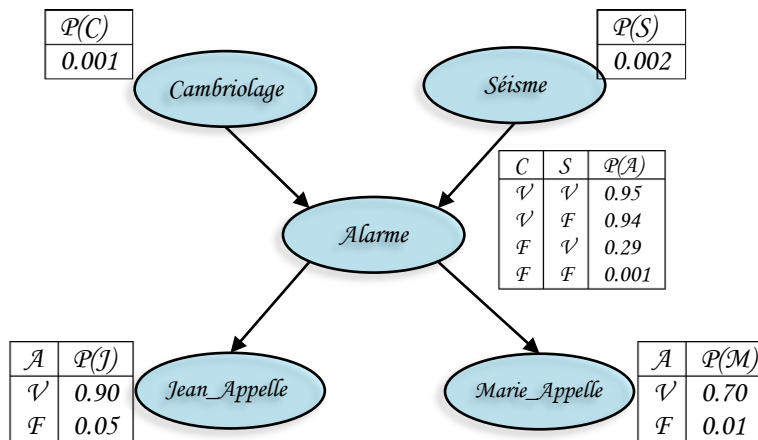


Figure 6.2. Exemple d'un graphe avec TPC

Dans ce cours, on considère uniquement des RB avec des variables discrètes : les TPC sont spécifiées en énumérant toutes les entrées.

Mais les RB peuvent aussi supporter les variables continues (figure 6.3) :

- Les probabilités conditionnelles sont spécifiées par des fonctions de densité de probabilités (PDF)

Exemples :

- » Distance entre voleur et le capteur de mouvement.
- » Force du séisme sur l'échelle de Richter.

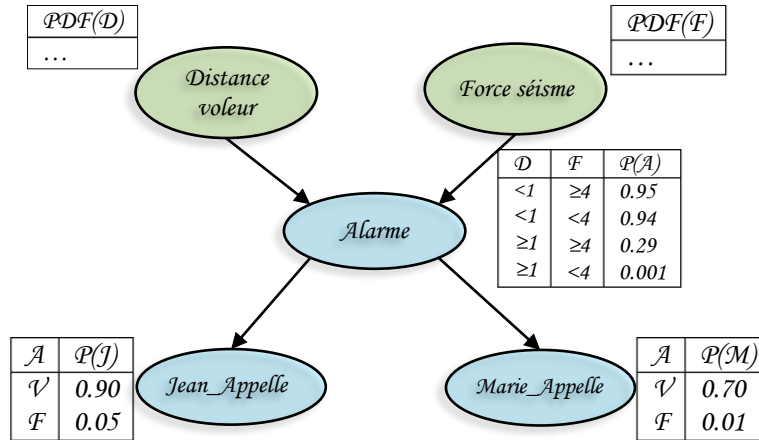


Figure 6.3. RB avec des variables continues

Il y a d'autres appellations pour les RB :

- Réseaux de croyance (belief networks)
- Modèle graphique dirigé acyclique

Ces réseaux font partie de la classe la plus courante des modèles graphiques.

✚ Calcul de la probabilité conjointe

Un RB est une façon compacte de représenter des probabilités conjointes. Par définition, la probabilité conjointe de X_1 et X_2 est donnée par la distribution $P(X_1, X_2)$, pour une valeur donnée de X_1 et X_2 .

La distribution conditionnelle de X_1 sachant X_2 est notée $P(X_1 | X_2)$

$$P(X_1, X_2) = P(X_1 | X_2) P(X_2) \quad (6.6)$$

Soit $X = \{X_1, \dots, X_n\}$, l'ensemble des variables d'un RB :

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i | \text{Parents}(X_i)) \quad (6.7)$$

En d'autres mots, la distribution conjointe des variables d'un RB est définie comme étant le produit des distributions conditionnelles (locales).

Nous avons vu que, quelque soit l'ensemble de variables $X = \{X_1, \dots, X_n\}$, par définition :

$$\begin{aligned} P(X_1, \dots, X_n) &= P(X_n | X_{n-1}, \dots, X_1) P(X_{n-1}, \dots, X_1) \\ &= P(X_n | X_{n-1}, \dots, X_1) P(X_{n-1} | X_{n-2}, \dots, X_1) \dots P(X_2 | X_1) P(X_1) \\ &= \prod_{i=1}^n P(X_i | X_{i-1}, \dots, X_1) \end{aligned} \quad (6.8)$$

Pour un RB : $P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i | \text{Parents}(X_i))$

Ceci est cohérent avec l'assertion précédente pour autant que $\text{Parents}(X_i)$ soit l'ensemble de $\{X_{i-1}, \dots, X_1\}$. Ainsi, un RB est en fait une façon de représenter les indépendances conditionnelles

Faisant maintenant le calcul de la probabilité conjointe pour notre exemple :

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i | \text{Parents}(X_i))$$

$$\begin{aligned} P(J = V, M = V, A = V, C = F, S = F) \\ = P(J = V | A = V) P(M = V | A = V) P(A = V | C = F, S = F) P(C = F) P(S = F) \\ = 0.90 * 0.70 * 0.001 * 0.999 * 0.998 = 0.00062 \end{aligned}$$

✚ Calcul de la probabilité marginale

$$\begin{aligned} P(C=F, A=V) &= \sum_m \sum_j \sum_s P(J=j, M=m, A=V, C=F, S=s) \\ &= \sum_m \sum_j \sum_s P(j|A=V) P(m|A=V) P(A=V|C=F, s) P(C=F) P(s) \\ &= \sum_s \sum_j \sum_m P(j|A=V) P(m|A=V) P(A=V|C=F, s) P(C=F) P(s) \\ &= \sum_s \sum_j P(j|A=V) P(A=V|C=F, s) P(C=F) P(s) \sum_m P(m|A=V) \\ &= \sum_s P(A=V|C=F, s) P(C=F) P(s) \sum_j P(j|A=V) \\ &= P(A=V|C=F, S=V) P(C=F) P(S=V) + P(A=V|C=F, S=F) P(C=F) P(S=F) \\ &= 0.29 * 0.999 * 0.002 + 0.001 * 0.999 * 0.998 \\ &\approx 0.0016 \end{aligned}$$

$$\begin{aligned} P(C=F, A=V) &= \sum_m \sum_j \sum_s P(J=j, M=m, A=V, C=F, S=s) \\ &= \sum_s P(A=V|C=F, s) P(C=F) P(s) \end{aligned}$$

Pour les probabilités marginales, on peut ignorer les nœuds dont les descendants ne sont pas les nœuds observés.

- Jean_Appelle et Marie_Appelle et leurs descendants ne sont pas observés, alors on peut les ignorer.
- Séisme est un ancêtre de Alarme, alors on doit le marginaliser explicitement.

6.3.3. Indépendance conditionnelle et d-séparation

Les connexions entre les nœuds définissent des lois de circulation de l'information dans le graphe. On distingue trois types de connexions :

- **Connexion en serie** (Figure 6.4(a)) : l'information ne peut circuler entre X et Y que si la valeur de Z n'est pas connue, sinon c'est directement la connaissance sur le nœud Z qui influe ;
- **Connexion divergente** (Figure 6.4(b)) : comme précédemment, l'information ne peut circuler entre X et Y que si la valeur de Z n'est pas connue ;
- **Connexion convergente** ou **connexion en V** (Figure 6.4(c)) : l'information ne peut circuler entre X et Y que si la valeur de Z est connue.

Ainsi, la circulation de l'information à l'intérieur d'un graphe dépend du type des connexions, plutôt que du sens des flèches.

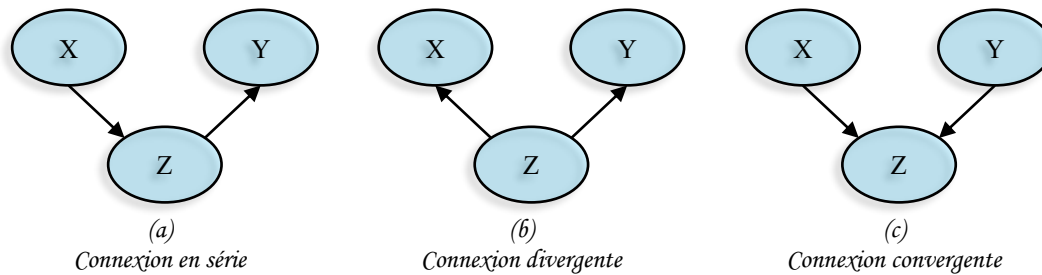


Figure 6.4. Types de connexions entre les nœuds

La notion de d-séparation est essentielle pour le calcul des probabilités a posteriori car elle permet de définir l'indépendance conditionnelle entre certains nœuds.

Soient X, Y et Z trois nœuds du graphe, on dit que X et Y sont d-séparés par Z (et on note $\langle X|Z|Y \rangle$) si, pour tous les chemins entre X et Y, l'une au moins des deux conditions suivantes est vérifiée :

- Le chemin converge en un nœud W, tel que $W \neq Z$, et Z n'est pas un descendant de W.
- Le chemin passe par Z avec une connexion divergente ou en série.

Dans la figure 6.4(a) et (b), X et Y sont d-séparés par Z, alors que dans la figure 6.4(c) Z empêche la d-séparation. En d'autres termes, lorsque Z est connu (on dit observé), l'information circule entre X et Y dans les deux sens. La définition de *d-séparation* s'étend à des sous-ensembles de variables.

Le théorème fondamental des réseaux bayésiens est défini comme suit :

« si X et Y sont d-séparés par Z, alors X et Y sont indépendants sachant Z » :

$$\langle X|Z|Y \rangle \Rightarrow P(X|Y, Z) = P(X|Z)$$

Ainsi la structure du graphe d'un réseau bayésien encode un certain nombre d'indépendances conditionnelles. Cette propriété est exploitée dans les algorithmes d'inférence pour réduire la partie du graphe à considérer.

En revanche, la réciproque est fautive : certaines indépendances conditionnelles, présentées dans la distribution de probabilité, peuvent ne pas se traduire dans le graphe. Certains graphes sont "meilleurs" que d'autres pour une même distribution de probabilité jointe sur un ensemble de variables X, car ils représentent plus d'indépendances conditionnelles. Par ailleurs, différents graphes peuvent encoder les mêmes indépendances conditionnelles, puisque seules les V-structures se différencient des deux autres types de connexion.

6.3.4. Construction d'un réseau bayésien

La construction d'un réseau bayésien peut se faire selon les trois méthodes suivantes :

- **Manuelle** : avec l'aide d'experts humains. Les spécialistes en ingénierie de la connaissance interrogent les experts et ajoutent les nœuds, les liens et les probabilités conditionnelles au réseau bayésien sur la base de la connaissance recueillie. Dans ce type de construction, il est plus fréquent de définir un graphe causal.
- **Automatique** : par application d'un algorithme d'apprentissage à une base de données. Les algorithmes d'apprentissage peuvent identifier à la fois la structure du graphe et les

paramètres (les distributions de probabilités conditionnelles). Les données peuvent être complètes ou incomplètes. On peut aussi avoir des variables non observables ;

- **Hybride** : cette approche combine les deux précédentes. Les données peuvent être exploitées pour améliorer un premier modèle élaboré à partir des connaissances disponibles. Elles peuvent permettre d'élaborer ou de modifier la structure du graphe du réseau bayésien ou les probabilités conditionnelles sur les nœuds du graphe.

Les algorithmes d'apprentissage de réseau bayésien permettent d'exploiter des données pour proposer un modèle qui reflète le mieux les données utilisées. On distingue deux types d'apprentissage : l'un permet d'obtenir la structure du graphe associé au réseau bayésien et l'autre permet d'obtenir la distribution de probabilité.

6.3.4.1. Apprentissage de la structure

L'objectif de l'apprentissage de la structure est de trouver une structure du graphe à partir des données disponibles et qui représente le mieux un problème.

Une solution naïve pour trouver la meilleure structure d'un réseau bayésien, est de parcourir tous les graphes possibles, de leur associer un score, puis de choisir le graphe qui a le score le plus élevé. Cependant, le nombre de structures différentes pour un réseau bayésien de n nœuds est super exponentiel. Il est donc impossible d'effectuer un parcours exhaustif en un temps raisonnable. Pour cette raison, la plupart des méthodes d'apprentissage de structure utilisent une heuristique de recherche dans l'espace des graphes acycliques dirigés.

Les méthodes d'apprentissage de structure qui se basent sur un calcul de score maximisent le score de la structure G qui décrit le mieux les données D . Par exemple,

$$\text{score}(G, D) = P(G|D) = \frac{P(D|G)P(G)}{P(D)} \quad (6.9)$$

Plusieurs méthodes et critères de calculs des scores ont été présentés dans la littérature.

6.3.4.2. Apprentissage des paramètres

L'apprentissage des paramètres pour une structure particulière consiste à estimer les distributions de probabilités a priori ou les paramètres des lois de probabilités à partir des données disponibles. Dans le cas où l'on dispose de données complètes, on peut utiliser le maximum de vraisemblance qui utilise la fréquence d'apparition d'un événement dans les données. Au contraire, si les données sont incomplètes, l'algorithme Expectation-Maximisation peut être utilisé. Cet algorithme itératif part du modèle, infère le modèle pour calculer la distribution de probabilité. Puis, sur la base de cette distribution, l'algorithme construit un modèle meilleur. Le processus se répète jusqu'à obtenir le modèle le plus vraisemblable.

6.3.5. Inférence dans les réseaux bayésiens

L'inférence est le calcul de la probabilité de n'importe quelle variable d'un modèle probabiliste à partir de l'observation d'une ou de plusieurs autres variables. Il consiste à propager une ou plusieurs informations au sein de ce réseau, pour en déduire comment sont modifiées les croyances

concernant les autres nœuds. La structure du graphe joue un rôle important dans la complexité de ces calculs ainsi dans le choix de la méthode d'inférence. On peut distinguer deux catégories d'algorithmes d'inférence : l'inférence exacte et l'inférence approchée. Plusieurs méthodes ou algorithmes conçues spécialement pour les problèmes d'inférence exacte pour les réseaux bayésiens. On peut citer l'algorithme de passage de messages de Pearl, dont le principe est le suivant, à chaque nœud est associé un processeur qui peut envoyer des messages à ses voisins, jusqu'à ce qu'un équilibre soit atteint, en un nombre fini d'étapes. Le problème de l'algorithme de passage de messages de Pearl est qu'il ne s'applique qu'aux arbres, l'algorithme d'arbre de jonction, qui est une généralisation de l'algorithme de passage de messages de Pearl permet de faire de l'inférence sur n'importe quel type de graphe. Cette méthode est divisée en cinq étapes qui sont : la moralisation du graphe, la triangulation du graphe moral, la construction de l'arbre de jonction, l'inférence dans l'arbre de jonction en utilisant l'algorithme des messages locaux et la transformation des potentiels de clique en lois conditionnelles mises à jour. A rappeler qu'une clique est un sous graphe du graphe G dont tous les nœuds sont connectés deux à deux. On peut citer encore l'Algorithme d'élimination des variables ou l'algorithme d'élimination de Bucket. Qui consiste à marginaliser la distribution de probabilité jointe d'un réseau, en procédant variable par variable. Chaque marginalisation sur une variable X_i donne lieu à une somme des probabilités de cette variable. Parfois, cette somme vaudra 1, ce qui conduit à l'élimination de la variable X_i . On procédera alors à la marginalisation sur une des variables restantes et ainsi de suite jusqu'à ce que la distribution soit marginalisée. Le problème de cet algorithme est que l'ordre dans lequel les variables sont éliminées détermine la quantité de calcul nécessaire pour marginaliser la distribution de probabilités jointe et donc la complexité de l'algorithme. Dans le cas de la classification on peut avoir soit des variables discrètes ou bien un mélange de variables discrètes représentant les classes et les variables continues représentant les caractéristiques du modèle. Dans les deux cas le problème de l'inférence revient à calculer les probabilités à posteriori, ainsi la classe choisie est celle qui maximise ces probabilités :

Dans le cas où les probabilités conditionnelles ne sont pas exactes, pour effectuer une inférence exacte avec ces valeurs approximatives n'est plus probant. Il est alors intéressant d'effectuer une inférence approchée, en utilisant d'autres algorithmes tels que l'Algorithme de Métropolies Hastings, l'échantillonneur de Gibbs, Loopy belief propagation...etc.

6.5. Conclusion

En conclusion, le raisonnement probabiliste offre un cadre rigoureux pour modéliser l'incertitude et prendre des décisions dans des environnements complexes. Les réseaux bayésiens, en combinant théorie des probabilités et théorie des graphes, permettent une représentation intuitive des dépendances entre variables aléatoires, facilitant ainsi l'inférence et l'apprentissage à partir de données. Leur flexibilité en fait des outils puissants en intelligence artificielle, en diagnostic médical, ou encore en finance. Cependant, leur construction requiert une bonne connaissance du domaine et des données pour éviter les biais. Malgré certaines limites, comme la complexité calculatoire pour les grands réseaux, leur capacité à intégrer des connaissances expertes et des observations en temps réel en fait une méthode incontournable. Les avancées récentes en apprentissage automatique étendent encore leurs applications, notamment grâce aux techniques

d'approximation et d'optimisation. Ainsi, maîtriser les réseaux bayésiens permet de résoudre des problèmes où l'incertitude et les relations causales sont centrales. Leur étude ouvre également la voie à des modèles plus sophistiqués, comme les réseaux dynamiques ou les modèles hybrides. En définitive, ce chapitre a mis en lumière l'importance du raisonnement probabiliste et des réseaux bayésiens pour modéliser et raisonner sous incertitude, offrant des perspectives prometteuses pour la recherche et les applications pratiques.

7.1. Introduction

Les systèmes experts (SE) représentent l'un des domaines les plus marquants de l'intelligence artificielle (IA). Apparus au milieu des années 1970, ils ont été conçus pour simuler le raisonnement humain d'un expert dans un domaine spécifique. Contrairement aux logiciels traditionnels basés sur des algorithmes prédéfinis, les systèmes experts s'appuient sur une base de connaissances et un moteur d'inférence pour résoudre des problèmes complexes nécessitant une expertise humaine.

Le premier système expert célèbre, MYCIN, développé à l'université de Stanford en 1972, a marqué un tournant dans l'histoire de l'IA. Conçu pour diagnostiquer des infections bactériennes et recommander des antibiotiques, MYCIN a démontré qu'un programme informatique pouvait atteindre une performance comparable, voire supérieure, à celle d'un médecin humain dans un domaine restreint. Ce système a ouvert la voie à une nouvelle génération de logiciels capables de raisonner, d'expliquer leurs décisions et de s'adapter à des situations nouvelles.

Depuis, les systèmes experts ont été déployés dans de nombreux domaines : médecine, ingénierie, finance, géologie, chimie, robotique, etc. Leur objectif principal est de capitaliser, transmettre et automatiser l'expertise humaine, en particulier lorsqu'elle est rare, coûteuse ou difficilement disponible.

Ce chapitre présente une vue d'ensemble complète des systèmes experts, en détaillant leurs fondements, leur architecture, leur fonctionnement, leurs méthodologies de conception, ainsi que leurs domaines d'application.

7.2. Définition d'un système expert

Un système expert (SE) est un logiciel intelligent qui reproduit le comportement d'un expert humain dans un domaine spécifique. Il collecte, formalise et restitue des connaissances pour aider à la prise de décision, au diagnostic, à la planification ou à la résolution de problèmes complexes.

D'autres définitions ont été attribuées aux systèmes experts dans la littérature, dont on peut citer quelques-unes :

- Un système expert est un programme conçu pour simuler le comportement d'un humain qui est un spécialiste ou un expert dans un domaine très restreint. (P Denning 1986).
- un logiciel intelligent qui utilise des connaissances et des inférences logiques pour résoudre des problèmes qui sont suffisamment difficiles pour nécessiter une expertise humaine importante pour trouver une solution (Feigenbaum, 1982).

- Un système expert est un logiciel qui sait donner des recommandations (pour un domaine et une application bien défini) au même niveau d'un expert humain de ce domaine.

Les systèmes experts sont donc des programmes conçus pour résoudre des problèmes généralement traités par des experts humains. Pour y parvenir, ils nécessitent un accès à une base de connaissances conséquente, élaborée de manière efficiente. Ces systèmes doivent également proposer différents modes de raisonnement et être capables de justifier les conclusions obtenues.

7.3. Quelques notions fondamentales

Un système expert ne calcule pas comme un programme classique. Il raisonne à partir de faits et de règles. Il est donc programmé pour raisonner sur des connaissances qui lui sont fournies par l'Expert Humain et qui sont appliquées dans le cas particulier de l'utilisateur. Il favorise la prise de décision des experts humains en se basant sur des axiomes. Dans son ensemble, il s'agira d'une évaluation par :

« Si prémisse alors conclusion »

Exemple : Si la température > 38°C ET frissons = oui ALORS suspicion de grippe.

Avec comme prémisse une disjonction ou conjonction de faits connus pour valider la conclusion. Les inférences de choix et de preuves pourront partir des faits connus établis ou alors de conclusions connues pour déduire les uns ou les autres. Le système Expert est basé sur une approche déterministe. Il est donc utile d'avoir des faits connus pour les exploiter et d'avoir des règles pour valider ou invalider (par déduction) les faits.

7.3.1. Les faits

Les faits, qui peuvent prendre des formes très diverses (connaissances), servent à établir des conditions préalables pour exploiter des connaissances opérationnelles. Ils constituent généralement les conditions initiales de résolution à partir desquelles des connaissances pourront être utilisées.

Ce sont des affirmations vraies dans un contexte donné. Ils constituent les données d'entrée du système.

Exemples :

- ❖ Amine habite Sidi Bel Abbès.
- ❖ La pression du circuit hydraulique est de 25 bars.
- ❖ Le client a un revenu de 40000 DA par mois.

Les faits peuvent être :

- ☞ *Initiaux* : fournis par l'utilisateur.
- ☞ *Dérivés* : déduits par le moteur d'inférence.

Les faits élémentaires sont :

- ☞ *Booléens* : vrai, faux

☞ *Symboliques* : c'est-à-dire appartenant à un domaine fini de symboles

☞ *Réels* : pour représenter les faits continus.

Par exemple, actif est un fait booléen, profession est un fait symbolique et rémunération est un fait réel.

7.3.2. Les règles

Une règle a pour construction une prémisse qui implique une conclusion. Les prémisses et les conclusions sont constituées de faits. Une conclusion peut être une prémisse d'une règle. Dans ce cas, pour valider une prémisse, il faudra alors valider la conclusion par la validé de sa prémisse. Ainsi, une règle est composée de faits liés avec des opérateurs logiques. Les règles représentent les connaissances opératoires qui permettent la déduction pour un système expert. Ce sont des règles déductives appelées règles de production. Les règles permettent de déduire d'autres faits. Les règles sont fournies par l'expert humain suite à son expérience dans le domaine.

Une règle contient généralement deux parties : une partie « *déclencheur* » de la règle, et une partie « *effets* » de la règle.

Pour sélectionner une règle particulière, on ne fournit pas un nom propre individuel à cette règle, mais un groupe de faits susceptibles d'être compatible avec les conditions de déclenchement de la règle. De plus, dans une règle, on ne désigne jamais une autre règle par un nom individuel.

Exemple de Prémisse et Conclusion

Si <fait connu> ALORS <fait déduit>

A. Cas d'une Prémisse unique

SI <condition> ALORS <conclusion>

Exemple : *SI <il fait froid> ALORS <je porte ma veste>*

Nous pouvons remarquer que la règle peut aussi être exploitée en sens inverse (Sachant qu'il a porté sa veste, nous déduisons qu'il fait froid).

B. Prémisses multiples (conjonction ou disjonction de conditions)

SI (condition1) ET (condition2) ALORS (conclusion)

Exemple : *SI (la pluie continue) ET (les égouts sont pleins) ALORS (la route est inondée)*

Enfin, la principale description est d'établir les liens entre les faits connus, les faits déduits pour proposer une règle.

Selon la description du domaine. *<fait connu> : <fait déduit>*, (*il fait froid : je porte ma veste*)

Exemple

SI <le malade présente de la fièvre> ET SI <il a une douleur intense à la gorge, surtout en avalant > Alors <il a une angine >

De là, on va faire ressortir les conditions de déclenchement de la règle :

<Le malade présente de la fièvre>, et <il a une douleur intense à la gorge, surtout en avalant >.

Alors l'effet de la règle est décrit par : *<il a une angine >*

L'application de cette règle se traduira alors par la déduction d'un nouveau fait désormais considéré établi : *<il a une angine >*

7.3.3. Le diagnostic

Pour un système expert, le diagnostic est le but initial à valider ou invalider en fonction des liens avec des faits connus, déduits ou observés. Le diagnostic indique si le comportement d'un système est conforme aux demandes. Une séquence de diagnostic permettra de proposer différents comportements pour arriver à un résultat. Le diagnostic est à la fois une référence et un résultat qu'un système expert doit prouver lors de son fonctionnement. Si la preuve n'est pas établie, le système passe au diagnostic suivant et si l'ensemble des diagnostics ne prouve rien, alors il y a échec de la recherche de solution. Il s'agit d'un fait référence à valider ou invalider. Il s'agit du point de validité d'un comportement dans le système.

Exemple :

Le système doit déterminer si un client est éligible à un prêt.

7.4. Architecture d'un système expert

L'architecture d'un système expert est modulaire, séparant clairement les connaissances du mécanisme de raisonnement. Il est principalement composé de (figure 7.1) :

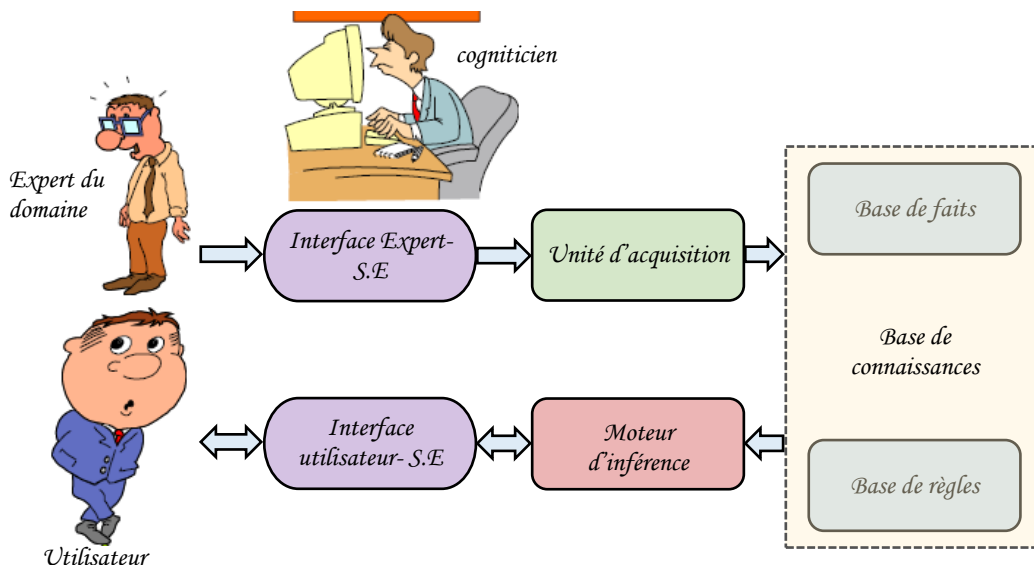


Figure 7.1. Architecture d'un système expert

7.4.1. La base de connaissances

C'est le cœur du système. Elle est formée de deux types d'informations constitutants :

A. La base de faits : traduit les faits constatés, qui sont des assertions (déclarations, affirmations) indéniables (vraies) constatées dans le domaine considéré. Il s'agit d'une base de données qui détermine l'ensemble de faits. Elle représente une partie de la connaissance d'un système expert sous formes déclaratives relatives à des faits connus.

B. La base de règles : traduit les déductions qu'il est possible d'effectuer, chaque règle étant constituée de prémisses et de conclusions ; les prémisses sont des assertions à partir desquelles on peut affirmer les conclusions. Il s'agit de l'ensemble des règles de production donné sous forme d'une base de données.

Alors, une base de règles contient les connaissances expertes, c'est à dire qu'elles représentent les raisonnements effectués par un expert. Ces connaissances sont appelées les unes à la suite des autres afin de créer des enchaînements de raisonnements et tous ces raisonnements peuvent être représentés sous forme de règles de production de type *< Si la condition est vraie alors exécuter action >*.

7.4.2. Le moteur d'inférence

Le moteur d'inférence est le cerveau du système. Il applique les règles aux faits pour produire de nouvelles conclusions.

Il s'agit donc d'un programme qui parcourt la base de connaissance en appliquant les règles aux faits connus donnés. Le moteur d'inférence est l'équivalent de l'esprit logique de l'expert humain qui raisonne sur des faits connus pour en sortir de nouvelles vérités. Il n'est pas lié à l'expertise et peut et doit être un programme qui raisonne sur diverses et plusieurs bases de connaissances.

7.4.2.1. Les caractéristiques d'un moteur d'inférence

Les caractéristiques d'un moteur d'inférence sont :

A. Un cycle de base

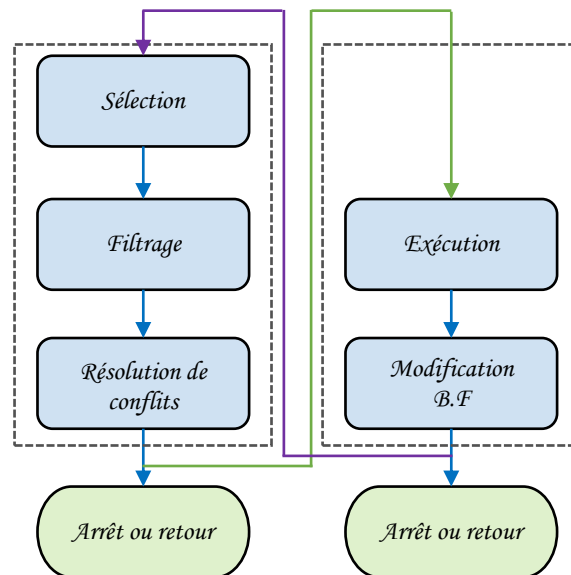


Figure 7.2. Cycle de base d'un moteur d'inférence

➤ Phase de sélection (restriction) :

L'objectif de cette tâche est de trier et de rassembler un sous-ensemble des faits et des règles de base de connaissance, cette phase permet une économie de temps au profit de la phase suivante (Figure 7.2).

➤ Phase de filtrage :

Déterminer l'ensemble des règles applicables (on l'appelle aussi ensemble conflit) (Figure 7.2).

➤ Phase de résolution de conflit :

Le choix de la règle à appliquer (Figure 7.2).

➤ Phase d'exécution :

Dans cette phase on applique la règle choisie à la phase précédente, cette étape permet d'ajouter un ou plusieurs faits dans BF (Base de faits) (Figure 7.2).

7.4.2.2. Stratégie et raisonnement du moteur d'inférence

Comme le moteur d'inférence est un programme indépendant chargé d'exploiter les connaissances et les faits pour mener un raisonnement. Il s'arrêtera lorsqu'il aura acquis les faits nécessaires pour aboutir à une conclusion.

A. Stratégie

Autrement, le moteur d'inférence est le cœur de tout système expert. Il peut être décrit par :

- ❖ Une stratégie de recherche, selon différentes stratégies, le moteur d'inférence utilise des règles, les interprète, les enchaîne jusqu'à arriver à un état représentant une condition d'arrêt,

- ❖ Une méthode de chaînage,
- ❖ Une tentative et des méthodes de résolutions des conflits (par exemple favoriser les règles dont les prémisses sont les plus courtes. favoriser les règles dont les résultats antérieurs sont les plus fructueux).

B. Raisonnement

Appelé également démonstrateur de théorème, le moteur d'inférence est chargé d'appeler et d'utiliser les informations contenues dans la base de connaissances, correctement, pour répondre à une question ou résoudre un problème. Il effectue des déductions grâce à deux principales stratégies de raisonnement qui sont :

i) *Stratégie relative au chaînage :*

- Chaînage avant.
- Chaînage arrière.
- Chaînage mixte.

ii) *Stratégie relative au parcours :*

- Parcours en largeur d'abord.
- Parcours en profondeur d'abord.

Pour que la prémisse déclenche la règle, il faut qu'elle s'apparente à l'un des faits connus. Nous disons que la recherche de la conclusion est guidée par les faits. Pour que la conclusion déclenche la règle il faut qu'elle s'apparente au but recherché. Nous disons que la recherche est guidée par le but.

❖ Chaînage avant

Son principe est de déterminer une réponse à partir des faits constatés, contenus dans la base de faits, en utilisant les règles de déductions de la base de connaissance et en déduisant des conclusions à partir des prémisses vérifiées.

Exemple

Le moteur d'inférence fonctionne dans ce mode lorsque les faits de la base de faits représentent des informations dont la vérité a été prouvée. C'est à dire que ce mode de fonctionnement va des faits vers les buts. La règle est déclenchée par la partie prémisse, de ce fait la conclusion en résulte.

❖ Chaînage arrière

Son principe consiste, à partir d'une question posée appelée but, de remonter jusqu'aux faits de la base de connaissance, grâce aux règles.

Exemple

Le moteur d'inférence part d'un fait que l'on souhaite établir, il recherche toutes les règles qui concluent sur ce fait, il établit la liste des faits qu'il suffit de prouver pour qu'elles puissent se déclencher puis il applique récursivement le même mécanisme aux autres faits contenus dans cette

liste. La règle est déclenchée par la partie conclusion et pour valider la règle, il faut que la prémisse soit un fait connu.

❖ Chaînage mixte

Comme son nom l'indique, cette approche utilise la combinaison des deux approches en alliant leurs avantages pour essayer de pallier leurs inconvénients.

Exemple

Utilise les deux chaînages présentés ci-dessus. On peut donc partir des données pour aller vers les buts et également d'un fait inconnu pour aller vers les données permettant de le retrouver. Combinaison des 2 stratégies précédentes.

L'efficacité du moteur d'inférence dépend non seulement du type de chaînage, mais aussi de la stratégie relative au type de parcours (en profondeur ou en largeur). La comparaison de deux moteurs d'inférence ne peut être réalisée correctement que si nous disposons des caractéristiques relatives aux règles (nombre de conclusions redondantes, profondeur et largeur de l'arbre des relations).

Exemple

Nous avons les règles de décisions suivantes :

Règle 1 : A et B

Règle 2 : B et C

Si nous connaissons le fait de départ A, nous pouvons à l'aide de ces règles par chaînage avant, trouver la conclusion C. Si nous connaissons la conclusion C, nous pouvons à l'aide de ces règles par chaînage arrière, trouver la cause A.

7.5. Méthodologie de réalisation d'un système expert

La réalisation d'un système expert nécessite une méthodologie qui offrira un fil conducteur auquel il conviendra de s'attacher tout au long du projet, de plus d'une documentation riche qui doit être rédigée afin de permettre la mise à jour ultérieure du projet.

Ainsi, la réalisation d'un système expert est un travail long, et un peu complexe dans sa programmation, la collection et la formalisation des connaissances. En plus de son intérêt scientifique, le système expert apporte un intérêt financier aux entreprises qui l'utilise, chaque entreprise devra réaliser un tel système lorsqu'elle possède un expert compétent dans son domaine ; Même si cet expert vient de quitter l'entreprise, les connaissances acquises ne disparaissent plus, parce que le transfert d'expertise est réalisé, et l'entreprise ne perdra pas du temps et d'argent.

La méthodologie de réalisation d'un système expert passe par les étapes suivantes :

- ☞ l'ingénieur de la connaissance (vous) appelé aussi cognitif qui est en charge de l'extraction de la connaissance auprès des experts ; en travaillant en étroite collaboration avec eux.

- ☞ les développeurs, ce sont des informaticiens qui se chargent de la programmation, en utilisant un outil de développement choisi suite aux études approfondies.

- ☞ les experts qui fournissent la connaissance (le savoir-faire) du futur système.
- ☞ les utilisateurs apportent leurs points de vue et leurs critiques afin que leurs désirs soient pris en compte dans le futur système.
- ☞ les managers qui suivent le projet et dont la tâche principale est de susciter l'intérêt de tous les intervenants.

Ainsi, la méthodologie de développement d'un système expert se résume en deux étapes importantes :

- ❖ une étude préalable qui servira à vérifier que les conditions du système expert sont réunies ; c'est également l'étude de la faisabilité de l'application du point de vue technique en s'assurant de la disponibilité des ressources matérielles et humaines et enfin, faire l'étude économique du projet (coûts de réalisation, coûts engendrés par le nouveau système) sans oublier la rentabilité du nouveau système.

- ❖ la réalisation qui consiste à définir d'abord un planning, puis le réaliser.

Les principales étapes de réalisation d'un système expert sont :

*1ère étape : **Choix d'un moteur d'inférence***

Le choix d'un moteur d'inférence se fait selon l'ordre du moteur d'inférence. Un moteur d'ordre zéro peut suffire (si les faits sont des propositions, des faits booléens). par contre, si les problèmes mis en jeu sont plus complexes, un moteur d'ordre un est nécessaire (si on va calculer les prédicats). De ce fait, C'est à l'informaticien de choisir le type.

*2ème étape : **Travail de l'expert***

L'expert doit arriver à extraire de ses méninges ses connaissances et les traduire sous une forme accessible par le moteur. En général, il n'est pas habitué à ce genre de réflexion et quelquefois, son propre raisonnement ne lui est pas complètement accessible. Cette étape aboutit à la réalisation d'une maquette système expert sur une partie restreinte du domaine choisi, suit une série de tests permettant à l'expert et au cognitifien de vérifier la véracité des diagnostics proposés par le système.

*3ème étape : **Extension au domaine***

Il s'agit d'étendre la version prototype au domaine initialement choisi. Là, l'expert doit continuer le travail de mise à plat de sa connaissance et également tester le travail du système expert.

*4ème étape : **Vers un produit fini***

Un système expert peut être utilisé par des non-experts ou des non-informaticiens. Par conséquent, il est indispensable de lui associer un ensemble de logiciels d'interface du type dialogue en langage naturel en respectant l'ergonomie et l'interface homme/machine, et par l'explication de raisonnement à suivre.

7.6. Avantages et inconvénients d'un système expert

7.6.1. Avantages d'un système expert

La puissance d'un système Expert réside dans :

- la quantité de connaissance traitée.
- la manière de représenter :
 - la base de faits.
 - la base de règles.
- la manière de les exploiter :
 - le moteur d'inférence.

7.6.2. Inconvénients d'un système expert

Les inconvénients des systèmes experts résident dans :

- Changement des besoins ; sortir d'un domaine d'application spécifique,
- Limites de la représentation des connaissances,
- Mauvaises tentatives de reproduction.

Enfin, La représentation des connaissances conditionne l'efficacité du raisonnement dans l'être humain aussi bien que dans la machine. Il s'agit d'un modèle interne qui représente sous forme codée l'environnement sur lequel nous agissons. C'est un domaine de recherche très développé en IA.

7.7. Quelques domaines d'applications des systèmes experts

La diversité des domaines d'application montre la mesure dans laquelle l'IA a été intégrée dans les secteurs d'activité, citons quelques exemples :

➤ **secteur médical** en 1982 ABEL (Acid-Base and Electrolyte disorders diagnosis) est un programme utilisant des modèles physiopathologiques à plusieurs niveaux pour le diagnostic des troubles acido-basiques et électrolytiques. En 2017, Easydiagnosis dernière actualisation d'une expertise en fonction de symptômes.

➤ **Secteur bancaire** en 1988, American Express développe de l'analyse financière pour l'octroi d'un prêt, détermination des montants et taux appropriés.

➤ En 2018, un système expert automatise le métier dans tous les domaines. Cela permet notamment une recherche précise de termes, de concepts sur les placements, les tenues de comptes, etc.

➤ **Secteur humanoïde et robotique** en 2014 GEMINOID développement d'un androïde.

➤ **Secteur de démonstrations mathématiques** : Dès le début et toujours en 2018, ALC2 (A Computational Logic for Applicative Common Lisp) permet de prouver les propriétés d'un modèle donné.

➤ Par exemple un SIAD (Système Interactif d'Aide à la Décision intelligents) inclut un système Expert, etc.

➤ COLOSSUS®. Système expert utilisé par plusieurs compagnies d'assurances dans le monde afin d'aider l'assureur dans des réclamations pour dommages corporels. <http://www.csc.com/industries/insurance/mds/mds221/408.s.html>

➤ HEPAXPERT III 1 WWW. Système expert médical permettant l'analyse et l'interprétation de sérologie d'hépatite A et d'hépatite B. <http://medexpert.imc.akh-wien.ac.at/hepax1>

➤ PEPID (<< Portable Emergency Physician Information Database >>). Ce système expert a été conçu pour aider les médecins à diagnostiquer rapidement les problèmes médicaux et de drogue arrivant en urgence à l'hôpital. Faisant suite aux diagnostics, des recommandations des premiers traitements médicaux nécessaires à donner sont proposés par le système.

7.8. Exemple d'un système expert

La figure 7.3 montre un exemple d'un système expert.

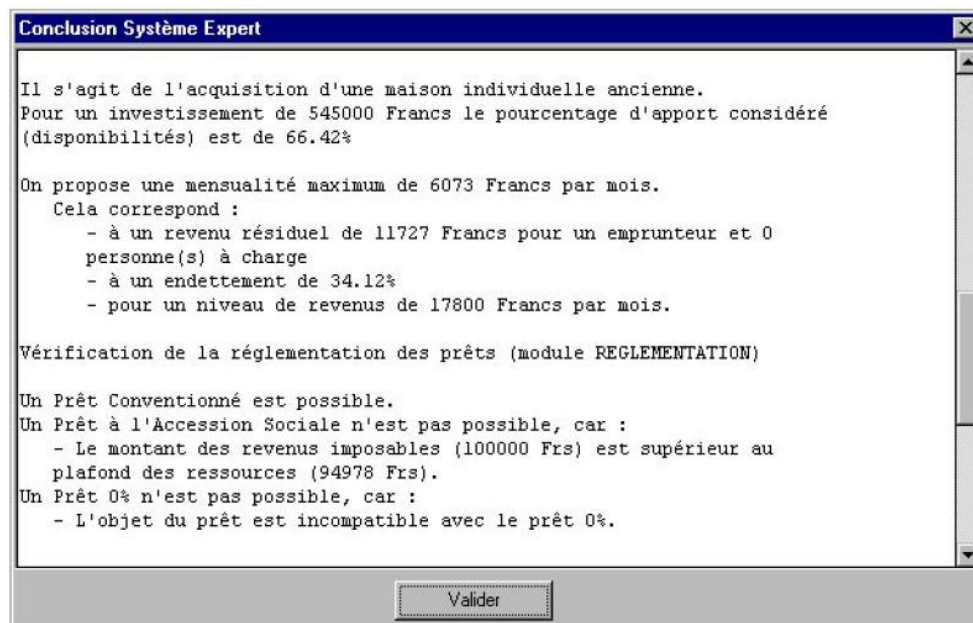


Figure 7.3. Exemple d'un système expert

7.9. Conclusion

Un système expert tend donc vers le remplacement du raisonnement humain, à la différence qu'il essaie d'être plus rapide et plus efficace que le raisonnement humain qui est bien souvent trop lent devant des situations trop complexes. L'élément central d'un système expert est la base de connaissances, dans laquelle se retrouve toute l'expertise qu'un expert humain possède sur le domaine d'application du système expert. Cette base de connaissances est jumelée à un moteur d'inférence (ou de raisonnement) qui permet au système expert de raisonner sur son domaine et ainsi tendre vers le raisonnement de l'expert humain.

Conclusion générale

À travers ce polycopié, nous avons présenté un ensemble de techniques d'intelligence artificielle qui trouvent aujourd'hui de nombreuses applications dans les domaines de l'ingénierie, en particulier en électrotechnique et en automatique. L'objectif poursuivi était de fournir aux étudiants une base méthodologique et conceptuelle solide, leur permettant non seulement de comprendre les fondements théoriques, mais également d'identifier les potentialités pratiques de ces approches.

Au fil des sept chapitres abordés, vous avez découvert comment la logique floue permet un raisonnement linguistique et une commande robuste basée sur l'expertise humaine ; comment les réseaux de neurones excellent dans l'approximation de fonctions complexes et l'identification de systèmes à partir de données ; et comment leur hybridation au sein des réseaux neuro-flous allie la puissance d'apprentissage à la transparence sémantique. Les métaheuristiques d'optimisation (algorithmes génétiques, PSO) vous fournissent désormais des moyens robustes pour résoudre des problèmes d'optimisation multidimensionnels et non convexes, cruciaux pour le réglage fin des contrôleurs ou la conception de systèmes. Enfin, les fondements du raisonnement probabiliste et des systèmes experts ont jeté les bases d'une IA capable de gérer l'incertitude et de formaliser l'expertise métier pour le diagnostic et l'aide à la décision.

Il convient de rappeler que ce document ne prétend pas à l'exhaustivité. L'intelligence artificielle est un domaine en constante évolution, marqué par des avancées rapides et des applications toujours plus variées. Ce polycopié constitue ainsi une porte d'entrée qui doit inciter l'étudiant à approfondir ses connaissances à travers des lectures complémentaires, des projets pratiques et des recherches plus spécialisées.

Nous espérons que ce support aura contribué à :

- ☞ Clarifier les concepts fondamentaux de l'intelligence artificielle,
- ☞ éveiller l'intérêt pour les nouvelles technologies de commande et d'optimisation,
- ☞ offrir des outils de réflexion et d'expérimentation utiles dans le cadre académique comme professionnel.

En définitive, l'intelligence artificielle représente non seulement un champ de recherche prometteur, mais également un levier d'innovation dans l'ingénierie moderne. Les étudiants sont ainsi invités à s'approprier ces connaissances et à les mettre en œuvre dans la conception de systèmes plus performants, plus adaptatifs et mieux adaptés aux défis technologiques et sociétaux de demain.

Références bibliographiques

1. P. A. Bisgambiglia, *La logique floue et ses applications*. Hermès Science, 2000.
2. H. Bühler, *Réglage par logique floue*. Lausanne : Presses polytechniques et universitaires romandes, 1994.
3. B. Kosko, *Neural Networks and Fuzzy Systems: A Dynamical Systems Approach to Machine Intelligence*. Englewood Cliffs, NJ: Prentice-Hall, 1992.
4. L. X. Wang, *Adaptive Fuzzy Systems and Control: Design and Stability Analysis*. Englewood Cliffs, NJ: Prentice-Hall, 1994.
5. D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.
6. Pierre-Yves Glorennec, *Algorithmes d'apprentissage pour systèmes d'inférence floue*. Hermes Science Publications, 1999.
7. P. Borne, J. Rozinoer, J.-Y. Dieulot, L. Dubois, *Introduction à la commande floue*. Éditions Technip, 1998.
8. B. Bouchon-Meunier, L. Foulloy, and M. Ramdani, *Logique floue : exercices corrigés et exemples d'applications*. Cépadués-Editions, 1998.
9. B. Bouchon-Meunier, *La logique floue et ses applications*. Paris, France : Addison Wesley, 1995.
10. H. T. Nguyen, N. R. Prasad, C. L. Walker, and E. A. Walker, *A First Course in Fuzzy and Neural Control*. Chapman and Hall/CRC, 2002.
11. Fakhreddine O. Karray, Clarence De Silva, *Soft Computing and Intelligent Systems Design: Theory, Tools, and Applications*, Pearson Education, 2004.
12. Pierre Borne, Mohamed Benrejeb, Joseph Haggège, *Les réseaux de neurones – Présentation et applications*, Éditions Technip, Paris, 2007.
13. Baghdad Hadj Ali, Senouci Mohamed, *Réseaux de neurones : Théorie et pratique*, OPU, 2005.
14. G. Dreyfus, J.-M. Martinez, M. Samuelides, M. B. Gordon, F. Badran, S. Thiria, L. Hérault, *Réseaux de neurones : Méthodologie et applications*, Eyrolles, 2002.
15. Léon Personnaz, Isabelle Rivals, *Réseaux de neurones formels pour la modélisation, la commande et la classification*, CNRS Editions, 2003.
16. Stuart Russell, Peter Norvig, *Intelligence artificielle : Avec plus de 500 exercices*, Pearson Education, 2010.
17. Johann Dréo, Alain Pérowski, Patrick Siarry, Éric Taillard, *Métaheuristiques pour l'optimisation difficile : Recuit simulé, recherche avec tabous, algorithmes évolutionnaires et génétiques, colonies de fourmis...*, Eyrolles, 2003.
18. Patrick Siarry, Alliot Jean-Marc, Aupetit Sébastien, Ben Hamida Sana., *Métaheuristiques : Recuit simulé, recherche avec tabous, recherche à voisinages variables, méthodes GRASP, algorithmes*

évolutionnaires, fournis artificielles, essais particuliers et autres méthodes d'optimisation, Eyrolles, 2014.

19. R. L. Haupt and S. E. Haupt, *Practical Genetic Algorithms*. John Wiley & Sons, 2004.
20. J. Kennedy and R. Eberhart, *Particle swarm optimization*, in *Proc. IEEE Int. Conf. Neural Networks*, 1995, pp. 1942–1948.
21. A. El Dor, *Improvement of particle swarm optimization algorithms: applications in image segmentation and electronics*, Ph.D. dissertation, Université Paris-Est, France, Dec. 2012.
22. J. Holland, *Adaptation in Natural and Artificial Systems*. University of Michigan Press google schola, 1975, vol. 2, p. 29-41.
23. U. Maulik, S. Bandyopadhyay, and A. Mukhopadhyay, *Multiobjective Genetic Algorithms for Clustering: Applications in Data Mining and Bioinformatics*. Springer Science & Business Media, 2011.
24. D. A. Coley, *An Introduction to Genetic Algorithms for Scientists and Engineers*. World Scientific Publishing Company, 1999.
25. E. Poirier, *Optimisation énergétique et entraînement sans capteur de position des machines à courant alternatif*, Master thesis, Univ. De Moncton, Canada, Sept. 2001.
26. Y. Cooren, “Improvement of an adaptive algorithm of particulate swarm optimization: application in medical engineering and electronics,” Ph.D. dissertation, Université Paris-Est, France, Nov. 2008.
27. K. Saidi, A. Boumediene, and S. Massoum, *An optimal PSO-based sliding-mode control scheme for the robot manipulator*, *Elektrotehniški Vestnik*, vol. 87, no. 1–2, pp. 53–59, 2020.
28. M. Semchedine, *Contribution à la segmentation d'images médicales par les algorithmes bio-inspirés*, Ph.D. dissertation, Univ. Ferhat Abbas Sétif 1, Algeria, Jul. 2018.
29. R. B. Bouiadjra, *Commande robuste des systèmes non linéaires*, Ph.D. dissertation, Univ. Oran 1 Ahmed Ben Bella, Algeria, 2015.
30. N. Siddique, *Intelligent Control: A Hybrid Approach Based on Fuzzy Logic, Neural Networks and Genetic Algorithms*. Springer, 2014.
31. L. Reznik, *Fuzzy Controllers*, Newnes, Oxford, 1997.
32. J. Espinosa, J. Vandewalle, and V. Wertz, *Fuzzy Logic, Identification and Predictive Control*. London: Springer London, 2005.
33. K. M. Passino, S. Yurkovich and M. Reinfrank, *Fuzzy Control*. Reading, MA, USA: Addison-Wesley, 1998.
34. B. Boulebtateche, *Support de cours : Commande Intelligente I. Logique Floue*. Annaba, Algeria : Univ. Badji Mokhtar, 2010.
35. R. Moot, *Cours : Systèmes Experts*. Paris, France : Univ. Paris-Sud, 2006.

36. F. Kabanza, *Cours d'Intelligence Artificielle : Raisonnement probabiliste, inférences avec une distribution conjointe et classifieur bayésien naïf*. Univ. de Sherbrooke, Canada, 2017.
37. P. Naïm, P.-H. Willemin, P. Leray, O. Pourret, and A. Becker, *Réseaux Bayésiens*, 3rd ed. Paris, France : Eyrolles, 2007.
38. Ali Ben Mrad. *Observations probabilistes dans les réseaux bayésiens*. Intelligence artificielle [cs.AI]. Université de Valenciennes et du Hainaut-Cambresis; École nationale d'ingénieurs de Sfax (Tunisie), 2015.
39. O. François and P. Leray, *Étude comparative d'algorithmes d'apprentissage de structure dans les réseaux bayésiens*, JEDAI – Journal Électronique d'Intelligence Artificielle, 2005.
40. F. Makhoulfi, "Modélisation et commande des robots manipulateurs par les outils de l'intelligence artificielle," Ph.D. dissertation, Univ. Badji Mokhtar Annaba, Algeria, 2015.
41. J.-S. R. Jang, *Neuro-Fuzzy and Soft Computing*. Upper Saddle River, NJ, USA: Prentice Hall, 1997.
42. S. Lynch, *Dynamical Systems with Applications using MATLAB*, Boston, MA, USA: Birkhäuser, 2004.
43. A. Zilouchian and M. Jamshidi, *Intelligent Control Systems Using Soft Computing Methodologies*, Boca Raton, FL, USA: CRC Press, 2001.
44. L. Fausett, *Fundamentals of Neural Networks: Architectures, Algorithms, and Applications*, Upper Saddle River, NJ, USA: Prentice Hall, 1994.
45. H. Abdi, *Les Réseaux de Neurones*. Presses Universitaires de Grenoble, 1994.
46. D. Bystrov, *Introduction to Soft Computing*. Moscow, Russia: Lecture Notes, 2002.
47. A. I. Galushkin, *Neural Networks Theory*. Berlin, Germany: Springer, 2007.
48. D. Graupe, *Principles of Artificial Neural Networks*, 3rd Ed, Advanced Series in Circuits and Systems, World Scientific Publishing Co. Pte. Ltd., Singapore, 2013.
49. K. Saidi, *Commandes non linéaires optimales appliquées à un système mécanique articulé*, Ph.D. dissertation, Univ. de Tlemcen, Algeria, 2021.
50. A. Ferroudj, *Commande non-linéaire de la MSAP sans capteur de vitesse. Apport des méthodes de l'intelligence artificielle*, Mémoire de Magister, Univ. de Batna, Algeria, 2010.
51. R. Sadouni, *Support de cours : Commande Intelligente*. Ghardaïa, Algeria: Université de Ghardaïa, 2019.
52. D. R. Hush and B. G. Horne, *Progress in supervised learning neural networks*, *IEEE Signal Processing Magazine*, vol. 10, no. 1, pp. 8–39, Jan. 1993.